

C# — leerboek

Programming Advanced · van tekst tot database · 34 hoofdstukken · met
voorspelvragen om jezelf te testen

Hoe je dit gebruikt

Dit leerboek brengt twee dingen samen die je nodig hebt om vlot in C# te programmeren: de **bouwstenen** — wat een `List`, een `Dictionary` of een `record` is — en de **methodes** — wat je ermee kan dóen. Ze staan in één volgorde waarin elk hoofdstuk enkel steunt op wat ervoor kwam. Werk je het van voor naar achter door, dan kom je nooit iets tegen dat nog niet uitgelegd is.

Er zijn twee soorten hoofdstukken.

- Een **bouwsteen** volgt altijd dezelfde stappen: wat het is, hoe het eruitziet, wat je ermee kan, wanneer wel en wanneer niet, en de veelgemaakte fouten.
- Een **methodehoofdstuk** is geordend op wat je wil doen: *ik wil een stuk uit een tekst halen, ik wil één item uit een lijst*. Per vraag staan de methodes die ervoor bestaan, en welke je in welk geval neemt.

Elk hoofdstuk eindigt met **Voorspel de uitkomst**. Lees de code, beslis wat er op het scherm komt, en klik pas dan op het antwoord. Uitleg lezen voelt als begrijpen, maar pas als je een uitkomst juist voorspelt, weet je of het zit. Heb je er één fout, lees dan de uitleg bij het antwoord en probeer de code zelf uit in een lege consoletoepassing.

Kom je later een naam tegen en wil je weten wat hij doet, gebruik dan de index achteraan.

Volg je de lessen van Programming Advanced, dan zegt de [leerwijzer](#) welke hoofdstukken bij elke les horen.

Kleuren en labels

Label of kleur	Betekenis
METHODES	een hoofdstuk geordend op wat je wil doen
VERZAMELING	een bouwsteen die meerdere dingen bewaart
EIGEN TYPE	een bouwsteen die je zelf maakt
CONCEPT	een idee dat door alle hoofdstukken heen loopt
IN JE API	wat je nodig hebt zodra je een API bouwt
DATA	werken met een database
methode	de naam van een methode of eigenschap
resultaat	wat eruit komt
Exception	het programma crasht, of compileert niet
Wel / Niet	wanneer je een bouwsteen kiest, en wanneer niet

Open dit bestand in VS Code en druk op **Ctrl+Shift+V** om het met kleuren te zien.

OPMERKING

De voorbeelden zijn losse regels over fruit, steden, leerlingen, dieren, recepten en verkeerslichten, en hier en daar een onderdeel of een ventilator. Het zijn nooit stukken uit Bouwcontrole of uit je schooloefeningen. Welke bouwsteen je in je eigen projecten gebruikt, beslis je zelf — dit leerboek geeft je de kennis om die keuze te kunnen maken.

OPMERKING

De hoofdletters doen ertoe. In C# is het `Find`, niet `find`, en `typeof`, niet `TypeOf`. Methodes beginnen met een hoofdletter, sleutelwoorden van de taal met een kleine letter. Kleine letters vooraan zijn meestal JavaScript.

Inhoud

Deel	Hoofdstuk	Wat je leert
Vooraan	Leerwijzer per les	welke hoofdstukken bij welke les horen
	Keuzegids	welke bouwsteen je neemt, en waar je iets vindt
1. De basis	1. Tekst	lengte, hoofdletters, knippen, zoeken, vergelijken
	2. Getallen en omzetten	tekst naar getal, afronden, delen met rest
	3. Datum en tijd	datums maken, rekenen, tonen
	4. Beslissen en herhalen	if, switch, for, while
2. Verzamelingen	5. Array	een rij met vaste lengte
	6. List	een lijst die groeit en krimpt
	7. Dictionary	opzoeken op een sleutel
	8. HashSet	een verzameling zonder dubbels
	9. Queue	wie eerst komt, eerst geholpen
	10. Stack	het laatste eerst terug
3. Zoeken en rekenen	11. LINQ	zoeken, filteren, tellen en sorteren in elke verzameling
	12. IEnumerable	wat LINQ eigenlijk teruggeeft
4. Eigen types	13. class	een bouwplan voor je eigen objecten
	14. record	een pakketje gegevens dat draait om de inhoud
	15. struct	een klein pakketje dat echt gekopieerd wordt
	16. Referentie tegenover waarde	het belangrijkste idee van dit leerboek
	17. Omgaan met niets	null, ??, ?. en getallen die mogen ontbreken
	18. enum	een vaste lijst benoemde keuzes
	19. interface	een contract: wat iets moet kunnen
	20. Overerving en abstracte klassen	verder bouwen op een andere klasse, abstract, override
	21. Generics	een type met een open plaats, zoals List<T>
	22. Types herkennen	is, as, typeof, nameof
	23. Uitzonderingen	try, catch, throw: fouten opvangen en melden
	24. static, const en readonly	wat bij de klasse hoort, en wat niet meer verandert
5. In je API	25. Attributen en routes	welke URL bij welke methode uitkomt
	26. De helpers van een controller	Ok, NotFound, BadRequest en de rest
	27. Program.cs en dependency injection	opstarten, diensten registreren en opvragen
	28. Logging	meldingen in het terminalvenster
	29. Asynchroon werken	async, await, Task

Deel	Hoofdstuk	Wat je leert
	30. JSON en bestanden	lezen, schrijven, omzetten
6. Data	31. EF Core: model en database	tabellen, relaties en migraties
	32. EF Core: opvragen en bewaren	ophalen, toevoegen, wijzigen, Include
	33. DTO's en Mapster	enkel doorgeven wat de client nodig heeft
	34. Dapper	zelf SQL schrijven, met een micro-ORM
Achteraan	Veelgemaakte verwarringen	dertig paren die op elkaar lijken
	Index	alles alfabetisch

Leerwijzer per les

Welke hoofdstukken horen bij welke les van Programming Advanced? Lees ze vóór of naast de les. Deel 1 heb je in elke les nodig; de tabel noemt de hoofdstukken die in die les nieuw zijn of het meest terugkomen.

Les	Onderwerp	Hoofdstukken
01	Introductie	13, 20, 25, 26, 27
02	HTTP Verbs	6, 11, 17, 24, 25, 26
03	Repositories	17, 19, 24, 27, 28
04	Entity Framework Core	20, 27, 31
05	Asynchroon programmeren	12, 23, 29, 32
06	Relaties	2, 3, 17, 31
07	Generics & Unit of Work	17, 19, 20, 21, 32
08	Include statements	7, 13, 32, 34
09	DTO's	11, 14, 33

OPMERKING

De tabel loopt tot les 09. Komt er een les bij, zoek dan de namen die je erin tegenkomt op in de index achteraan.

Keuzegids

Welke bouwsteen neem ik?

Ik wil...	Neem	Hoofdstuk
een bedrag of prijs, exact tot op de cent	<code>decimal</code>	2
een datum of een tijdstip	<code>DateTime</code>	3
een vast aantal dingen dat nooit verandert	<code>array</code>	5
een aantal dingen waar ik bijvoeg en afhaal	<code>List</code>	6
iets terugvinden op een naam, code of nummer	<code>Dictionary</code>	7
zeker zijn dat er geen dubbels in zitten	<code>HashSet</code>	8
dingen afhandelen in de volgorde waarin ze binnenkwamen	<code>Queue</code>	9
het laatst toegevoegde als eerste terugnemen	<code>Stack</code>	10
een verzameling doorgeven zonder te zeggen welke soort	<code>IEnumerable</code>	12
een eigen ding met gegevens én gedrag	<code>class</code>	13
een pakketje gegevens dat gelijk is zodra de inhoud gelijk is	<code>record</code>	14
een klein pakketje dat bij kopiëren echt gekopieerd wordt	<code>struct</code>	15
een vaste lijst benoemde keuzes	<code>enum</code>	18
afspreken wat iets moet kunnen, zonder te zeggen hoe	<code>interface</code>	19
verwante klassen die gegevens en uitgewerkte methodes delen	<code>abstract class</code>	20
één stuk code dat werkt voor elk type dat je invult	generics, <code><T></code>	21
iets dat bij de klasse hoort, niet bij één object	<code>static</code>	24
een waarde die na het aanmaken nooit meer verandert	<code>const</code> of <code>readonly</code>	24
gegevens bewaren in een database, zonder zelf SQL te schrijven	EF Core	31
aan de client enkel geven wat hij nodig heeft	een DTO	33
zelf de SQL schrijven voor een snelle leesvraag	Dapper	34

Waar vind ik hoe ik...

Ik wil...	Hoofdstuk
een stuk uit een tekst halen, of tekst opsplitsen	1
twee teksten vergelijken zonder op hoofdletters te letten	1
tekst omzetten naar een getal zonder te crashen	2
iets verdelen: hoeveel per groep, hoeveel over	2
met datums rekenen, of een datum tonen	3
kiezen tussen meerdere mogelijkheden	4
iets een aantal keer herhalen	4
iets toevoegen aan of verwijderen uit een lijst	6
per soort tellen hoeveel er van zijn	7
één item uit een verzameling halen	11
weten of er iets in zit dat voldoet	11
filteren, sorteren, optellen, het grootste vinden	11
begrijpen waarom mijn resultaat plots veranderd is	12
begrijpen waarom mijn lijst na een methode veranderd is	16
iets doen met een waarde die null kan zijn	17
begrijpen wat = default! achter een eigenschap doet	17
een methode van een basisklasse anders laten werken	20
één klasse schrijven die voor elk type werkt	21
nagaan van welk type een object is	22
een fout opvangen, zodat mijn programma niet stopt	23
zelf een fout melden vanuit een methode	23
een lijst bewaren tussen twee verzoeken	24
een methode aan een URL koppelen	25
een waarde uit de URL in mijn methode krijgen	25
de juiste statuscode teruggeven in een API	26
mijn repository aan mijn controller geven	27
iets melden in het terminalvenster terwijl de API draait	28
een methode asynchroon maken	29
een JSON-bestand inlezen	30
een tabel maken voor mijn klasse	31
een relatie tussen twee tabellen vastleggen	31
mijn database bijwerken na een wijziging aan een klasse	31
gegevens ophalen, toevoegen, wijzigen of verwijderen in de database	32
gerelateerde gegevens meeladen	32
een entiteit omzetten naar een DTO	33

Deel 1 — De basis

Tekst, getallen, datums en beslissingen heb je in elk ander hoofdstuk nodig. Daarom komen ze eerst.

1. Tekst

METHODES

Hoe lang is het, is het leeg?

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Length</code>	aantal tekens; een eigenschap, geen methode, dus zonder haakjes	<code>"moederbord".Length</code>	<code>10</code>
<code>string.IsNullOrEmpty</code>	is het null of een lege tekst?	<code>string.IsNullOrEmpty("")</code>	<code>true</code>
<code>string.IsNullOrWhiteSpace</code>	is het null, leeg, of enkel spaties?	<code>string.IsNullOrWhiteSpace(" ")</code>	<code>true</code>

Voor invoer die van een gebruiker komt neem je bijna altijd `IsNullOrWhiteSpace`. Iemand die een spatie intikt heeft niets ingevuld, maar `IsNullOrEmpty` vindt dat wel een waarde.

Hoofdletters, kleine letters, spaties weg

Methode	Wat het doet	Voorbeeld	Resultaat
<code>ToUpper</code>	alles hoofdletters	<code>"mb-b650".ToUpper()</code>	<code>"MB-B650"</code>
<code>ToLower</code>	alles kleine letters	<code>"MB-B650".ToLower()</code>	<code>"mb-b650"</code>
<code>Trim</code>	spaties vooraan en achteraan weg	<code>" ATX ".Trim()</code>	<code>"ATX"</code>
<code>TrimStart</code>	enkel vooraan	<code>" ATX".TrimStart()</code>	<code>"ATX"</code>
<code>TrimEnd</code>	enkel achteraan	<code>"ATX ".TrimEnd()</code>	<code>"ATX"</code>
<code>PadLeft</code>	vul aan tot een lengte, vooraan	<code>"7".PadLeft(3, '0')</code>	<code>"007"</code>
<code>PadRight</code>	vul aan tot een lengte, achteraan	<code>"AM5".PadRight(6, '.')</code>	<code>"AM5..."</code>

Deze methodes veranderen je tekst niet, ze geven een nieuwe terug. `naam.Trim()` op een regel alleen doet niets; je moet het resultaat ergens opvangen.

Een stuk uit een tekst halen

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Substring</code>	vanaf een positie, eventueel een aantal tekens	<code>"moederbord".Substring(6)</code>	<code>"bord"</code>
<code>Substring</code>	met lengte erbij	<code>"moederbord".Substring(0, 6)</code>	<code>"moeder"</code>
<code>[..]</code>	hetzelfde, met de bereikoperator	<code>"moederbord"[0..6]</code>	<code>"moeder"</code>
<code>[^..]</code>	de laatste n tekens	<code>"LZ-2026-8820"[^4..]</code>	<code>"8820"</code>
<code>Split</code>	knip op een teken, geeft een array	<code>"ATX,mATX,ITX".Split(',')</code>	<code>["ATX", "mATX", "ITX"]</code>
<code>string.Join</code>	plak stukken aan elkaar met een scheider	<code>string.Join(" / ", delen)</code>	<code>"ATX / mATX / ITX"</code>
<code>Replace</code>	vervang alle voorkomens	<code>"MB-B650-01".Replace("-", "")</code>	<code>"MBB65001"</code>

LET OP

`Substring` loopt stuk zodra je buiten de tekst grijpt. `"ATX".Substring(0, 5)` geeft geen lege tekst maar een uitzondering. Controleer eerst de `Length`, of gebruik `Split` wanneer je niet weet hoe lang iets is.

Zit er iets in, en waar?

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Contains</code>	zit het erin, ja of nee	<code>"MB-B650-01".Contains("650")</code>	<code>true</code>
<code>StartsWith</code>	begint het ermee	<code>"MB-B650".StartsWith("MB")</code>	<code>true</code>
<code>EndsWith</code>	eindigt het erop	<code>"MB-B650-01".EndsWith("01")</code>	<code>true</code>
<code>IndexOf</code>	positie van het eerste voorkomen	<code>"MB-B650-01".IndexOf("-")</code>	<code>2</code>
<code>IndexOf</code>	niet gevonden	<code>"MB-B650".IndexOf("x")</code>	<code>-1</code>
<code>LastIndexOf</code>	positie van het laatste voorkomen	<code>"MB-B650-01".LastIndexOf("-")</code>	<code>7</code>

`Contains` als je enkel wil weten óf het erin zit. `IndexOf` als je daarna nog iets met de plaats wil doen, bijvoorbeeld alles ervoor of erna afsplitsen.

LET OP

`IndexOf` geeft `-1` terug als er niets gevonden is, geen `null`. Vergeet je dat te controleren en steek je die `-1` in een `Substring`, dan loopt je code stuk op een plek die er onschuldig uitziet.

Tekst vergelijken

Methode	Wat het doet	Voorbeeld	Resultaat
<code>==</code>	letterlijk gelijk, hoofdlettergevoelig	<code>"AM5" == "am5"</code>	<code>false</code>
<code>Equals</code>	idem, maar je kan de regels meegeven	<code>"AM5".Equals("am5", StringComparison.OrdinalIgnoreCase)</code>	<code>true</code>
<code>CompareTo</code>	welke komt eerst in de sortering	<code>"b".CompareTo("a")</code>	<code>1</code>

Wil je twee sockets vergelijken die de ene keer met hoofdletters en de andere keer zonder binnenkomen, dan is `Equals` met `OrdinalIgnoreCase` het nette antwoord. `ToUpper()` op allebei werkt ook, maar maakt twee nieuwe teksten om er één vergelijking mee te doen.

Tekst opbouwen met waarden erin

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>\$"..."</code>	zet waarden rechtstreeks in de tekst	<code>\$"Socket {socket} past"</code>	<code>"Socket AM5 past"</code>
<code>+</code>	plakken	<code>"Socket " + socket</code>	<code>"Socket AM5"</code>
<code>string.Format</code>	oudere vorm met plaatshouders	<code>string.Format("Socket {0}", socket)</code>	<code>"Socket AM5"</code>

De `$"..."`-vorm is wat je overal zal tegenkomen. Je mag er ook berekeningen in zetten: `$"Nog {sloten - modules} vrij"`.

Voorspel de uitkomst

Vraag 1

```
string s = "  Gent  ";
s.Trim();
Console.WriteLine($"[" + s + "]");
```

Toon het antwoord

```
[  Gent  ]
```

`Trim` verandert de tekst niet. Het geeft een nieuwe tekst terug, en die werd hier nergens opgevangen. Wil je dat het werkt, dan schrijf je `s = s.Trim();`.

Vraag 2

```
string code = "ABC-123";  
Console.WriteLine(code.Substring(code.IndexOf("-") + 1));
```

Toon het antwoord

123

Het streepje staat op positie 3. Eén verder is positie 4, en vanaf daar neemt `Substring` alles tot het einde. Dit is het patroon om alles ná een scheidingsteken te krijgen.

Vraag 3

```
Console.WriteLine("appel" == "Appel");  
Console.WriteLine("appel".Equals("Appel", StringComparison.OrdinalIgnoreCase));
```

Toon het antwoord

`False` en daarna `True`

`==` let op hoofdletters. Met `OrdinalIgnoreCase` zeg je uitdrukkelijk dat het niet mag.

Vraag 4

```
string[] delen = "rood;groen;;blauw".Split(';');  
Console.WriteLine(delen.Length);
```

Toon het antwoord

4

Tussen de twee puntkomma's staat niets, en dat niets telt ook mee als stuk: een lege tekst. De vier stukken zijn rood, groen, een lege tekst en blauw.

2. Getallen en omzetten

METHODES

Tekst omzetten naar een getal

Methode	Wat het doet	Voorbeeld	Resultaat
<code>int.Parse</code>	zet om, of loopt stuk	<code>int.Parse("120")</code>	<code>120</code>
<code>int.Parse</code>	bij onzin	<code>int.Parse("abc")</code>	uitzondering
<code>int.TryParse</code>	probeert om te zetten, zegt of het lukte	<code>int.TryParse("120", out int n)</code>	<code>true</code> , <code>n</code> is <code>120</code>
<code>int.TryParse</code>	bij onzin	<code>int.TryParse("abc", out int n)</code>	<code>false</code> , <code>n</code> is <code>0</code>
<code>double.Parse</code>	hetzelfde voor kommagetallen	<code>double.Parse("2.5")</code>	hangt af van je taalinstelling — zie hieronder

BELANGRIJK

De regel is eenvoudig: komt de tekst van een gebruiker, gebruik `TryParse`. Komt ze uit je eigen code en weet je zeker dat het een getal is, dan mag `Parse`. Een `Parse` op gebruikersinvoer is een crash die op je zit te wachten.

LET OP

`double.Parse` leest een kommagetal volgens de taalinstelling van je computer. Op een Belgische computer is de komma het decimaalteken en de punt het teken voor duizendtallen, zoals in 1.000. `double.Parse("2.5")` geeft daar 25 — zonder foutmelding. Komt je getal uit een bestand of een API, waar altijd een punt staat, geef dan `CultureInfo.InvariantCulture` mee: `double.Parse("2.5", CultureInfo.InvariantCulture)`.

Omgekeerd toont `Console.WriteLine(2.5)` op jouw computer 2,5. In dit leerboek staan kommagetallen in de schrijfwijze van C# zelf, met een punt.

Rekenen

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Math.Max</code>	de grootste van twee	<code>Math.Max(3, 7)</code>	7
<code>Math.Min</code>	de kleinste van twee	<code>Math.Min(3, 7)</code>	3
<code>Math.Abs</code>	zonder minteken	<code>Math.Abs(-5)</code>	5
<code>Math.Round</code>	afronden	<code>Math.Round(2.4)</code>	2
<code>Math.Ceiling</code>	altijd naar boven	<code>Math.Ceiling(2.1)</code>	3
<code>Math.Floor</code>	altijd naar beneden	<code>Math.Floor(2.9)</code>	2
<code>Math.Pow</code>	tot de macht	<code>Math.Pow(2, 3)</code>	8
<code>/</code>	delen van gehele getallen	<code>7 / 2</code>	3
<code>%</code>	de rest na het delen	<code>7 % 2</code>	1

LET OP

Twee dingen die bijna iedereen één keer verkeerd doet. `7 / 2` geeft 3 en niet 3.5, want twee gehele getallen delen geeft een geheel getal. En `Math.Round(2.5)` geeft 2, niet 3 — C# rondt een halve eenheid naar het dichtstbijzijnde even getal. Wil je het schoolse afronden, dan geef je `MidpointRounding.AwayFromZero` mee.

Het deelteken en het restteken samen zijn precies wat je nodig hebt om iets te verdelen: hoeveel per groep, en hoeveel blijven er over.

Kommagetallen: double tegenover decimal

C# heeft twee soorten kommagetallen die je vaak tegenkomt. Een `double` is snel, maar bewaart sommige getallen net niet exact. Een `decimal` is trager, maar rekent tot op de cent juist. Vanaf les 06 staat een prijs in de cursus als `decimal`.

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>double</code>	snel kommagetal, kan een heel klein beetje afwijken	<code>0.1 + 0.2 == 0.3</code>	<code>false</code>
<code>decimal</code>	exact kommagetal, voor geld en prijzen	<code>0.1m + 0.2m == 0.3m</code>	<code>true</code>
<code>m</code> achter een getal	maakt er een <code>decimal</code> van	<code>decimal prijs = 19.99m;</code>	compileert
zonder <code>m</code>	een kommagetal zonder letter is een <code>double</code>	<code>decimal prijs = 19.99;</code>	compileert niet
<code>(double)</code>	zet een <code>decimal</code> om naar een <code>double</code>	<code>(double)19.99m</code>	<code>19.99</code>
<code>Math.Round</code>	afronden op een aantal cijfers na de komma	<code>Math.Round(19.987m, 2)</code>	<code>19.99</code>

LET OP

Een `decimal` en een `double` mag je niet zomaar door elkaar gebruiken. `prijs * 1.21` compileert niet als `prijs` een `decimal` is, want `1.21` is een `double`. Schrijf `prijs * 1.21m`.

Voorspel de uitkomst

Vraag 1

```
Console.WriteLine(7 / 2);
Console.WriteLine(7 % 2);
```

Toon het antwoord

3 en daarna 1

Twee gehele getallen delen geeft een geheel getal: 2 past 3 keer in 7. Wat er overblijft, 1, krijg je met %.

Vraag 2

```
bool gelukt = int.TryParse("12a", out int getal);  
Console.WriteLine(gelukt);  
Console.WriteLine(getal);
```

Toon het antwoord

False en daarna 0

"12a" is geen getal. TryParse crasht niet, geeft false terug, en zet getal op nul.

Vraag 3

```
Console.WriteLine(Math.Round(3.5));  
Console.WriteLine(Math.Round(4.5));
```

Toon het antwoord

4 en daarna 4

Geen tikfout. C# rondt een halve eenheid af naar het dichtstbijzijnde **even** getal: 3,5 wordt 4, en 4,5 wordt ook 4. Wil je dat 4,5 een 5 wordt, dan geef je MidpointRounding.AwayFromZero mee.

Vraag 4

Op een computer met Belgische taalinstelling:

```
double d = double.Parse("2.5");  
Console.WriteLine(d);
```

Toon het antwoord

25

Op een Belgische computer is de punt het teken voor duizendtallen, zoals in 1.000. double.Parse leest "2.5" dus als vijfentwintig, zonder foutmelding. Met double.Parse("2.5", CultureInfo.InvariantCulture) krijg je wel twee en een half. Zie de waarschuwing hierboven.

Vraag 5

```
Console.WriteLine(0.1 + 0.2 == 0.3);  
Console.WriteLine(0.1m + 0.2m == 0.3m);
```

Toon het antwoord

False en daarna True

Een double bewaart 0,1 en 0,2 niet exact, en de som ligt een heel klein beetje naast 0,3. Een decimal rekt met tientallen, zoals jij op papier, en komt wel precies op 0,3 uit. Daarom is een prijs een decimal.

Vraag 6

```
decimal prijs = 10m;  
var metBtw = prijs * 1.21;
```

Toon het antwoord

Compileert niet.

1.21 zonder letter is een double, en C# weigert een decimal met een double te vermenigvuldigen. Het zou niet weten welk van de twee soorten het resultaat moet worden. Met 1.21m lukt het wel.

3. Datum en tijd

METHODES

OPMERKING

In les 06 en 09 krijgen je modellen een datum, zoals de dag waarop iets aangemaakt werd. Daarvoor gebruik je `DateTime`.

Een datum maken

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>new DateTime(j, m, d)</code>	een vaste datum	<code>new DateTime(2026, 10, 3)</code>	3 oktober 2026, om middernacht
<code>new DateTime(j, m, d, u, min, s)</code>	een vaste datum met uur	<code>new DateTime(2026, 10, 3, 14, 30, 0)</code>	3 oktober 2026, 14.30 uur
<code>DateTime.Now</code>	nu, op de klok van de computer	<code>DateTime.Now</code>	het huidige moment
<code>DateTime.Today</code>	vandaag, om middernacht	<code>DateTime.Today</code>	de datum van vandaag, zonder uur
<code>DateTime.UtcNow</code>	nu, in wereltijd	<code>DateTime.UtcNow</code>	in België één of twee uur vroeger dan <code>Now</code>
<code>DateOnly</code>	enkel een datum, zonder uur	<code>new DateOnly(2026, 10, 3)</code>	3 oktober 2026

Maanden tellen hier vanaf 1, niet vanaf 0 zoals de posities in een array. Januari is 1, december 12.

Een stuk eruit halen

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>Year</code>	het jaar	<code>d.Year</code>	2026
<code>Month</code>	de maand, van 1 tot 12	<code>d.Month</code>	10
<code>Day</code>	de dag van de maand	<code>d.Day</code>	3
<code>Hour, Minute</code>	het uur en de minuut	<code>d.Hour</code>	14
<code>DayOfWeek</code>	de dag van de week, als enum	<code>d.DayOfWeek</code>	Saturday
<code>Date</code>	dezelfde dag, met het uur op middernacht	<code>d.Date</code>	3 oktober 2026, 00.00 uur

`DayOfWeek` is een enum zoals in hoofdstuk 18, met Engelse namen: `Monday` tot en met `Sunday`. Dat blijft zo, ook op een Nederlandstalige computer.

Rekenen met datums

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>AddDays</code>	een aantal dagen erbij, of eraf met een minteken	<code>d.AddDays(7)</code>	een week later
<code>AddMonths</code>	een aantal maanden erbij	<code>d.AddMonths(1)</code>	een maand later
<code>AddHours</code>	een aantal uren erbij	<code>d.AddHours(-2)</code>	twee uur vroeger
<code>-</code>	het verschil tussen twee datums	<code>eind - begin</code>	een <code>TimeSpan</code>
<code>Days</code>	het aantal hele dagen in een <code>TimeSpan</code>	<code>(eind - begin).Days</code>	14
<code>TotalHours</code>	het verschil in uren, met komma	<code>(eind - begin).TotalHours</code>	336
<code><, >, ==</code>	vergelijken: welke komt eerst	<code>begin < eind</code>	true

Net als bij tekst veranderen deze methodes de datum zelf niet. `d.AddDays(1)` op een regel alleen doet niets; je moet het resultaat opvangen.

Tonen en inlezen

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>ToString("yyyy-MM-dd")</code>	toon in een vaste vorm	<code>d.ToString("yyyy-MM-dd")</code>	"2026-10-03"
<code>ToString("HH:mm")</code>	enkel het uur	<code>d.ToString("HH:mm")</code>	"14:30"
<code>DateTime.TryParse</code>	probeer een tekst om te zetten, volgens de taalinstelling	<code>DateTime.TryParse("03/10/2026", out DateTime d)</code>	op een Belgische computer: 3 oktober
<code>DateTime.ParseExact</code>	zet om volgens een vorm die jij opgeeft	<code>DateTime.ParseExact("2026-10-03", "yyyy-MM-dd", CultureInfo.InvariantCulture)</code>	3 oktober 2026

LET OP

In een opmaak betekent MM maand en mm minuten. HH is het uur van 0 tot 23, hh het uur van 1 tot 12. Eén verkeerde hoofdletter geeft een datum die er geloofwaardig uitziet en toch fout is.

En net als bij `double.Parse` in hoofdstuk 2 leest `DateTime.TryParse` volgens de taalinstelling. "03/10/2026" is in België 3 oktober, in de Verenigde Staten 10 maart. Ken je de vorm, gebruik dan `ParseExact`.

Voorspel de uitkomst

Vraag 1

```
var d = new DateTime(2026, 1, 31);  
Console.WriteLine(d.AddMonths(1).Day);
```

Toon het antwoord

28

Februari 2026 heeft geen 31ste. `AddMonths` blijft dan op de laatste dag van die maand staan, in plaats van door te schuiven naar maart.

Vraag 2

```
var begin = new DateTime(2026, 10, 1);  
var eind = new DateTime(2026, 10, 15);  
Console.WriteLine((eind - begin).Days);
```

Toon het antwoord

14

Van de eerste tot de vijftiende zijn het veertien dagen. Het verschil tussen twee datums is een `TimeSpan`, en `Days` geeft daarvan het aantal hele dagen.

Vraag 3

```
var d = new DateTime(2026, 10, 3);  
d.AddDays(1);  
Console.WriteLine(d.Day);
```

Toon het antwoord

3

`AddDays` geeft een nieuwe datum terug en laat `d` met rust, net zoals `Trim` bij tekst. Wil je dat het werkt, dan schrijf je `d = d.AddDays(1);`.

Vraag 4

```
var d = new DateTime(2026, 10, 3, 14, 5, 0);  
Console.WriteLine(d.ToString("yyyy-mm-dd"));
```

Toon het antwoord

2026-05-03

mm met kleine letters zijn de minuten, en dat is hier 5. Voor de maand had er MM moeten staan.

Vraag 5

```
var d = new DateTime(2026, 2, 29);
```

Toon het antwoord

`ArgumentOutOfRangeException` — het programma crasht.

2026 is geen schrikkeljaar, dus 29 februari bestaat niet. Het compileert wel, want de compiler rekent geen kalenders na. De fout komt pas als de regel draait.

4. Beslissen en herhalen

METHODES

OPMERKING

Ken je dit al uit een andere taal, lees dan vooral de tabellen en de waarschuwingen. Een paar dingen werken in C# net iets anders, zoals de switch-expressie.

Vergelijken en combineren

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>==</code>	is gelijk aan	<code>5 == 5</code>	<code>true</code>
<code>!=</code>	is niet gelijk aan	<code>5 != 3</code>	<code>true</code>
<code><, <=</code>	kleiner dan, kleiner dan of gelijk aan	<code>3 <= 3</code>	<code>true</code>
<code>>, >=</code>	groter dan, groter dan of gelijk aan	<code>2 > 3</code>	<code>false</code>
<code>&&</code>	en: allebei waar	<code>leeftijd >= 3 && leeftijd <= 17</code>	<code>true</code> voor 3 tot en met 17
<code> </code>	of: minstens één waar	<code>dag == "za" dag == "zo"</code>	<code>true</code> in het weekend
<code>!</code>	niet: draait om	<code>!gevonden</code>	<code>true</code> als er niets gevonden is

`&&` en `||` stoppen zodra het antwoord vastligt. Bij `s != null && s.Length > 3` wordt de lengte enkel opgevraagd als `s` niet `null` is. Dat is geen toeval: zo schrijf je een controle die niet crasht.

Iets doen als...

```
int leeftijd = 15;

if (leeftijd < 3)
{
    Console.WriteLine("Gratis");
}
else if (leeftijd < 18)
{
    Console.WriteLine("Kindertarief");
}
else
{
    Console.WriteLine("Volle prijs");
}
```

C# loopt de voorwaarden van boven naar onder af en neemt de **eerste** die klopt. De rest wordt niet meer bekeken. De volgorde van je `else if` bepaalt dus het antwoord.

Voor een keuze tussen twee waarden bestaat er een korte vorm:

```
string tekst = leeftijd >= 18 ? "volwassen" : "minderjarig";
```

Lees het als: *klopt de voorwaarde? Dan het eerste, anders het tweede.*

Kiezen uit veel mogelijkheden: switch

Vergelijk je één waarde met een reeks vaste mogelijkheden, dan is een `switch` overzichtelijker dan een lange rij `else if`:

```
string dag = "za";

switch (dag)
{
    case "za":
    case "zo":
        Console.WriteLine("Weekend");
        break;
    case "wo":
        Console.WriteLine("Halve dag");
        break;
    default:
        Console.WriteLine("Weekdag");
        break;
}
```

Moet er gewoon een waarde uitkomen, dan is de **switch-expressie** korter. Ze werkt ook met grenzen:

```
int graden = 25;

string advies = graden switch
{
    < 10 => "Neem een jas",
    < 20 => "Neem een trui",
    _ => "Een t-shirt volstaat"
};
```

De `_` betekent *al de rest*. Ook hier geldt: de eerste die past, wint.

Herhalen

```
for (int i = 0; i < 3; i++)
{
    Console.WriteLine(i);           // 0, 1, 2
}

int teller = 10;
while (teller > 0)
{
    teller -= 3;                    // 7, 4, 1, -2: dan stopt het
}

foreach (string naam in namen)
{
    Console.WriteLine(naam);       // elk item één keer
}
```

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>if / else if / else</code>	doe iets als een voorwaarde klopt	<code>if (x > 0) { ... }</code>	de eerste die klopt, wint
<code>? :</code>	kies tussen twee waarden	<code>x > 0 ? "plus" : "min"</code>	"plus" of "min"
<code>switch</code>	voer een blok uit per mogelijkheid	<code>case "za": ... break;</code>	elk blok eindigt met <code>break</code>
<code>switch</code> -expressie	geef een waarde per mogelijkheid	<code>x switch { 1 => "een", _ => "veel" }</code>	"een" of "veel"
<code>< 10 =></code>	een grens in een switch-expressie	<code>< 10 => "koud"</code>	alles onder 10
<code>and, or</code>	grenzen combineren in een switch-expressie	<code>>= 3 and <= 17 => "kind"</code>	3 tot en met 17
<code>_ =></code>	al de rest	<code>_ => "onbekend"</code>	wat nergens anders past
<code>for</code>	herhaal een vast aantal keer	<code>for (int i = 0; i < 5; i++)</code>	<code>i</code> gaat van 0 tot 4
<code>while</code>	herhaal zolang iets klopt	<code>while (teller > 0)</code>	misschien nul keer
<code>do ... while</code>	herhaal, en kijk pas achteraf	<code>do { ... } while (x > 0);</code>	minstens één keer
<code>foreach</code>	elk item van een verzameling	<code>foreach (var n in namen)</code>	elk item één keer
<code>break</code>	stop de lus nu	<code>if (gevonden) break;</code>	de lus is voorbij
<code>continue</code>	sla de rest van deze ronde over	<code>if (x < 0) continue;</code>	door naar de volgende ronde

Veelgemaakte fouten

LET OP

- `=` geeft een waarde, `==` vergelijkt. `if (x = 5)` compileert in C# niet, want een toewijzing is geen ja of nee.
- Een switch-expressie zonder `_` compileert met een waarschuwing. Past de waarde nergens, dan crasht ze terwijl het programma draait.
- `for (int i = 0; i <= lijst.Count; i++)` loopt één keer te veel. De laatste positie is `Count - 1`, dus het is `<`, niet `<=`.
- Een `while` waarvan de voorwaarde nooit vals wordt, stopt nooit. Kijk na of er in de lus iets verandert aan wat je controleert.
- Teksten vergelijken met `==` let op hoofdletters. Komt een waarde van een gebruiker, denk dan aan `Equals` met `OrdinalIgnoreCase` uit hoofdstuk 1.

Voorspel de uitkomst

Vraag 1

```
int leeftijd = 70;

if (leeftijd >= 18)
{
    Console.WriteLine("volwassen");
}
else if (leeftijd >= 65)
{
    Console.WriteLine("senior");
}
else
{
    Console.WriteLine("kind");
}
```

Toon het antwoord

volwassen

70 is groter dan 18, dus de eerste voorwaarde klopt al. De tweede wordt niet meer bekeken, ook al zou die ook kloppen. Wil je dat senioren apart behandeld worden, dan moet de controle op 65 eerst komen.

Vraag 2

```
int graden = 15;

string advies = graden switch
{
    < 10 => "jas",
    < 20 => "trui",
    _ => "t-shirt"
};

Console.WriteLine(advies);
```

Toon het antwoord

trui

15 is niet kleiner dan 10, maar wel kleiner dan 20. De tweede regel past als eerste.

Vraag 3

```
for (int i = 1; i <= 5; i++)
{
    if (i == 2)
    {
        continue;
    }

    if (i == 4)
    {
        break;
    }

    Console.Write(i);
}
```

Toon het antwoord

13

Bij 1 wordt er geschreven. Bij 2 slaat continue de rest over. Bij 3 wordt er weer geschreven. Bij 4 stopt break de hele lus, dus 4 en 5 komen er nooit. Console.Write zet alles op één regel.

Vraag 4

```
int n = 10;

do
{
    Console.WriteLine(n);
}
while (n < 5);
```

Toon het antwoord

10

Een `do-lus` voert het blok eerst uit en kijkt pas daarna naar de voorwaarde. 10 is niet kleiner dan 5, dus het blijft bij die ene keer. Met een gewone `while` was er niets op het scherm gekomen.

Vraag 5

```
int x = 5;

string woord = x switch
{
    1 => "een",
    2 => "twee"
};

Console.WriteLine(woord);
```

Toon het antwoord

`SwitchExpressionException` — het programma crasht.

Het compileert, met een waarschuwing dat niet elke waarde gedekt is. Als het programma draait en `x` is 5, past er geen enkele regel, en dan weet C# niet wat `woord` moet worden. Een laatste regel met `_` voorkomt dat.

Deel 2 — Verzamelingen

Zes manieren om meerdere dingen samen te bewaren. Ze verschillen in wat ze toelaten en in hoe snel je iets terugvindt.

De verzamelingen naast elkaar

	array	List	Dictionary	HashSet	Queue	Stack
Grootte	vast	groeit	groeit	groeit	groeit	groeit
Volgorde bewaard	ja	ja	niet gegarandeerd	niet gegarandeerd	ja, eerst in eerst uit	ja, laatst in eerst uit
Dubbels	mag	mag	sleutels nee, waarden wel	nee	mag	mag
Opvragen op	positie	positie	sleutel	—	enkel vooraan	enkel bovenaan
Snel nagaan of iets erin zit	nee	nee	ja, op sleutel	ja	nee	nee

Die laatste rij verdient uitleg. Een `List` met tienduizend items moet bij `Contains` in het slechtste geval alle tienduizend nakijken. Een `Dictionary` of `HashSet` weet meteen waar hij moet kijken, hoe groot hij ook is. Bij tien items merk je dat niet. Bij een miljoen wel.

5. Array

VERZAMELING

Wat het is

Een rij vakjes met een vaste lengte. Zoals een eierdoos: zes plaatsen, niet meer en niet minder. Je kan een ei vervangen, maar je kan er geen zevende plaats bij maken.

Hoe het eruitziet

```
string[] dagen = { "ma", "di", "wo", "do", "vr" };

Console.WriteLine(dagen[0]);           // ma
Console.WriteLine(dagen.Length);       // 5

dagen[4] = "vrijdag";                  // vervangen mag

int[] scores = new int[3];             // drie vakjes, allemaal 0

foreach (string dag in dagen)
{
    Console.WriteLine(dag);
}
```

Wanneer wel, wanneer niet

Wel — de hoeveelheid ligt vast en verandert nooit: de dagen van de week, de maanden, de vakjes van een bordspel.

Niet — zodra je iets wil bijvoegen of weghalen. Een array heeft geen `Add` en geen `Remove`.

Veelgemaakte fouten

LET OP

- De eerste positie is 0, dus de laatste is `Length - 1`. Vraag je `dagen[5]` op in een array van vijf, dan crasht je programma.
- Het is `Length`, niet `Count`. Bij een `List` is het net omgekeerd.
- `new int[3]` geeft drie nullen. `new string[3]` geeft drie keer `null`, geen drie lege teksten.

Voorspel de uitkomst

Vraag 1

```
int[] a = new int[3];  
Console.WriteLine(a[1]);
```

Toon het antwoord

0

Een nieuwe array van getallen is gevuld met de standaardwaarde, en die is voor een `int` gelijk aan nul.

Vraag 2

```
string[] s = { "a", "b", "c" };  
Console.WriteLine(s[s.Length - 1]);
```

Toon het antwoord

c

`Length` is 3, dus `Length - 1` is 2, en positie 2 is het derde vakje.

Vraag 3

```
string[] s = { "a", "b", "c" };  
Console.WriteLine(s[3]);
```

Toon het antwoord

`IndexOutOfRangeException` — het programma crasht.

Er zijn drie vakjes: 0, 1 en 2. Vakje 3 bestaat niet.

Vraag 4

```
int[] a = { 1, 2, 3 };  
int[] b = a;  
b[0] = 99;  
Console.WriteLine(a[0]);
```

Toon het antwoord

99

Dit is de belangrijkste vraag van dit hoofdstuk. `b = a` maakt geen kopie van de array. Het geeft dezelfde array een tweede naam. Verander je iets via `b`, dan zie je dat ook via `a`, want het is één en dezelfde eierdoos. Zie ook [hoofdstuk 16](#).

6. List

VERZAMELING

Wat het is

Een array die kan groeien en krimpen. Zoals een boodschappenlijstje: je schrijft erbij, je streept door, en de volgorde blijft zoals je ze opschreef.

Tussen de punthaken staat wat erin mag. `List<string>` is een lijst van teksten, `List<int>` een lijst van getallen, `List<Leerling>` een lijst van leerlingen. Iets anders erin steken compileert niet.

Hoe het eruitziet

```
List<string> fruit = new List<string>();
fruit.Add("appel");
fruit.Add("peer");
fruit.Add("kiwi");

Console.WriteLine(fruit.Count);    // 3
Console.WriteLine(fruit[1]);       // peer

fruit.Remove("peer");
Console.WriteLine(fruit[1]);       // kiwi – alles schuift een plaats op

fruit.Insert(0, "banaan");         // vooraan bijzetten

List<string> kort = new List<string> { "appel", "peer" }; // meteen gevuld
```

Je zal ook de kortere schrijfwijze `List<string> kort = ["appel", "peer"];` tegenkomen. Die doet hetzelfde.

Wat je ermee kan

Dit zijn de methodes die op een `List<T>` zelf zitten. Ze werken enkel op een lijst, niet op elke verzameling.

Toevoegen en verwijderen

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Add</code>	zet er één achteraan bij	<code>fans.Add(nieuweFan)</code>	de lijst is één langer
<code>AddRange</code>	zet er meerdere achteraan bij	<code>fans.AddRange(extraFans)</code>	de lijst groeit
<code>Insert</code>	zet er één op een vaste plaats	<code>fans.Insert(0, nieuweFan)</code>	staat nu vooraan
<code>Remove</code>	haal dit ene item eruit	<code>fans.Remove(fan)</code>	<code>true</code> als het erin zat
<code>RemoveAt</code>	haal weg op positie	<code>fans.RemoveAt(0)</code>	de eerste is weg
<code>RemoveAll</code>	haal alles weg dat voldoet	<code>fans.RemoveAll(f => f.Prijs == 0)</code>	<code>2</code> als er twee weg zijn
<code>Clear</code>	maak leeg	<code>fans.Clear()</code>	de lijst is leeg

`Remove` wil het item zelf, `RemoveAt` wil een positie, `RemoveAll` wil een voorwaarde. Drie namen die op elkaar lijken en drie verschillende dingen willen.

Erin zoeken

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Count</code>	hoeveel zitten erin; een eigenschap	<code>fans.Count</code>	4
<code>Contains</code>	zit dit exacte item erin	<code>fans.Contains(fan)</code>	true
<code>IndexOf</code>	op welke positie staat het	<code>fans.IndexOf(fan)</code>	2, of -1
<code>Exists</code>	is er minstens één die voldoet	<code>fans.Exists(f => f.Prijs > 50)</code>	true
<code>Find</code>	geef de eerste die voldoet	<code>fans.Find(f => f.Id == 3)</code>	het item, of null
<code>FindAll</code>	geef allemaal die voldoen	<code>fans.FindAll(f => f.Merk == "NZXT")</code>	een nieuwe lijst
<code>FindIndex</code>	op welke positie staat de eerste die voldoet	<code>fans.FindIndex(f => f.Id == 3)</code>	2, of -1

Ordenen

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Sort</code>	sorteer de lijst zelf	<code>getallen.Sort()</code>	de lijst staat op volgorde
<code>Reverse</code>	keer de lijst om	<code>getallen.Reverse()</code>	omgekeerde volgorde
<code>ToArray</code>	maak er een array van	<code>fans.ToArray()</code>	een array

BELANGRIJK

`Sort` en `Reverse` veranderen je lijst zelf en geven niets terug. De LINQ-methodes uit hoofdstuk 11 doen het omgekeerde: die laten je lijst met rust en geven een nieuw resultaat. Dat verschil verklaart waarom `lijst.Sort()` werkt en `var x = lijst.Sort()` niet compileert.

Wanneer wel, wanneer niet

Wel — de standaardkeuze voor "een aantal dingen". Twijfel je, begin dan met een `List`.

Niet — wanneer je telkens op dezelfde eigenschap zoekt (neem een `Dictionary`), of wanneer dubbels verboden zijn (neem een `HashSet`).

Veelgemaakte fouten

LET OP

- Het is `Count`, niet `Length`.
- Na `Remove` of `RemoveAt` schuift alles erachter een plaats op. Een positie die je vóór het verwijderen onthield, wijst daarna naar iets anders.
- Je mag een lijst niet veranderen terwijl je er met `foreach` doorheen loopt. Dat crasht, ook als je code er onschuldig uitziet.
- `List<string> lijst`; zonder `new` is geen lege lijst maar niets. `lijst.Add` crasht dan.

Voorspel de uitkomst

Vraag 1

```
var l = new List<int> { 10, 20, 30 };  
l.RemoveAt(0);  
Console.WriteLine(l[0]);
```

Toon het antwoord

20

De 10 op positie 0 is weg. De 20 schuift op naar positie 0.

Vraag 2

```
var l = new List<string> { "a", "b", "a" };  
l.Remove("a");  
Console.WriteLine(l.Count);
```

Toon het antwoord

2

`Remove` haalt enkel het **eerste** voorkomen weg. De lijst is nu `b, a`. Wil je ze allemaal weg, dan heb je `RemoveAll` nodig.

Vraag 3

```
var l = new List<int> { 1, 2, 3 };  
foreach (int x in l)  
{  
    if (x == 2)  
    {  
        l.Remove(x);  
    }  
}
```

Toon het antwoord

InvalidOperationException — het programma crasht.

foreach merkt dat de lijst veranderd is terwijl hij erdoor liep, en weigert verder te gaan. Wil je tijdens het doorlopen verwijderen, gebruik dan `RemoveAll` met een voorwaarde, of loop door een kopie.

Vraag 4

```
var l = new List<string> { "kiwi", "appel", "peer" };  
l.Sort();  
Console.WriteLine(l[0]);
```

Toon het antwoord

appel

Sort zet de lijst zelf op alfabetische volgorde. Het geeft niets terug, het verandert de lijst.

7. Dictionary

VERZAMELING

Wat het is

Een woordenboek. Je zoekt niet op paginanummer maar op het woord zelf. Elk woord staat er maar één keer in, en bij elk woord hoort een uitleg.

In C# heet het woord de **sleutel** en de uitleg de **waarde**. Tussen de punthaken staan ze allebei: `Dictionary<string, int>` heeft teksten als sleutel en getallen als waarde.

Hoe het eruitziet

```
Dictionary<string, string> hoofdsteden = new Dictionary<string, string>();

hoofdsteden.Add("België", "Brussel");
hoofdsteden["Frankrijk"] = "Parijs";           // toevoegen, of overschrijven

Console.WriteLine(hoofdsteden["België"]);      // Brussel
Console.WriteLine(hoofdsteden.Count);          // 2

if (hoofdsteden.TryGetValue("Spanje", out string? stad))
{
    Console.WriteLine(stad);
}
else
{
    Console.WriteLine("Onbekend land");         // dit wordt getoond
}

foreach (KeyValuePair<string, string> paar in hoofdsteden)
{
    Console.WriteLine($"{paar.Key}: {paar.Value}");
}
```

Bij het doorlopen krijg je telkens een paar terug: `.Key` is de sleutel, `.Value` de waarde.

Het patroon dat je het vaakst zal gebruiken: tellen

```
string[] dieren = { "kat", "hond", "kat", "vis", "kat" };
Dictionary<string, int> telling = new Dictionary<string, int>();

foreach (string dier in dieren)
{
    if (telling.ContainsKey(dier))
    {
        telling[dier]++;
    }
    else
    {
        telling[dier] = 1;
    }
}

Console.WriteLine(telling["kat"]); // 3
Console.WriteLine(telling["vis"]); // 1
```

Zodra je ergens *hoeveel van elk* moet weten, is dit het antwoord. Het werkt voor woorden, letters, kleuren, stemmen — alles waar je per soort wil tellen.

Wanneer wel, wanneer niet

Wel — je zoekt telkens op één en dezelfde eigenschap: een naam, een code, een nummer. En je wil dat snel doen, hoe groot de verzameling ook is.

Niet — wanneer de volgorde ertoe doet, of wanneer je afwisselend op verschillende eigenschappen zoekt. Een woordenboek zoekt snel op het woord, maar niet op de uitleg.

Wat je ermee kan

Methode	Wat het doet	Voorbeeld	Resultaat
[sleutel]	haal op, of loopt stuk	prijzen["dag"]	65
Add	voeg toe, of loopt stuk bij een dubbele sleutel	prijzen.Add("vip", 210)	staat erin
TryAdd	voeg toe als de sleutel vrij is	prijzen.TryAdd("vip", 210)	true of false
ContainsKey	bestaat die sleutel	prijzen.ContainsKey("vip")	true
TryGetValue	haal op en zeg of het lukte	prijzen.TryGetValue("vip", out int p)	true, p is 210
Remove	haal die sleutel weg	prijzen.Remove("vip")	true
Keys	alle sleutels	prijzen.Keys	["dag", "combi", "vip"]
Values	alle waarden	prijzen.Values	[65, 120, 210]

Dezelfde verhouding als bij Parse en TryParse: de vierkante haken lopen stuk op een sleutel die niet bestaat, TryGetValue niet. Weet je niet zeker of de sleutel er is, neem dan TryGetValue.

Twee manieren om toe te voegen

Schrijfwijze	Sleutel bestaat nog niet	Sleutel bestaat al
<code>d.Add(sleutel, waarde)</code>	voegt toe	crasht
<code>d[sleutel] = waarde</code>	voegt toe	overschrijft

Kies bewust. `Add` is een bewaker: hij zegt je dat er iets dubbel binnenkomt. De vierkante haken zijn soepel: ze overschrijven zonder te klagen.

Veelgemaakte fouten

LET OP

- hoofdsteden["Spanje"] lezen terwijl Spanje er niet in staat, crasht. Twijfel je, gebruik dan `TryGetValue` of eerst `ContainsKey`.
- Sleutels zijn hoofdlettergevoelig. "Kat" en "kat" zijn twee verschillende sleutels. Wil je dat niet, geef dan `StringComparer.OrdinalIgnoreCase` mee bij het aanmaken.
- Reken niet op de volgorde bij het doorlopen. Vandaag komen ze toevallig terug in de volgorde waarin je ze toevoegde, maar dat is geen belofte.

Voorspel de uitkomst

Vraag 1

```
var d = new Dictionary<string, int>();  
d["a"] = 1;  
d["a"] = 2;  
Console.WriteLine(d["a"]);  
Console.WriteLine(d.Count);
```

Toon het antwoord

2 en daarna 1

De tweede regel overschrijft de eerste. Er is nog altijd maar één sleutel.

Vraag 2

```
var d = new Dictionary<string, int>();  
d.Add("a", 1);  
d.Add("a", 2);
```

Toon het antwoord

`ArgumentException` — het programma crasht.

Add weigert een sleutel die er al in zit. Dat is het verschil met de vierkante haken uit vraag 1.

Vraag 3

```
var d = new Dictionary<string, int> { ["Kat"] = 4 };  
Console.WriteLine(d.ContainsKey("kat"));
```

Toon het antwoord

`False`

"Kat" met hoofdletter en "kat" zonder zijn voor een gewone `Dictionary` twee verschillende sleutels.

Vraag 4

```
var d = new Dictionary<string, int> { ["x"] = 5 };  
bool gevonden = d.TryGetValue("y", out int waarde);  
Console.WriteLine(gevonden);  
Console.WriteLine(waarde);
```

Toon het antwoord

`False` en daarna `0`

De sleutel "y" bestaat niet. `TryGetValue` crasht niet maar geeft `false` terug, en zet `waarde` op de standaardwaarde van een `int`: nul.

8. HashSet

VERZAMELING

Wat het is

Een verzameling waarin alles maar één keer voorkomt. Zoals een stickeralbum: een dubbele sticker plak je niet opnieuw in, je weet gewoon dat je hem al hebt.

Hoe het eruitziet

```
HashSet<string> bezocht = new HashSet<string>();

bezocht.Add("Gent");
bezocht.Add("Brugge");
bool nieuw = bezocht.Add("Gent");           // false – stond er al in

Console.WriteLine(bezocht.Count);           // 2
Console.WriteLine(bezocht.Contains("Brugge")); // True
```

Een `HashSet` kan ook rekenen met verzamelingen:

Methode	Wat er overblijft
<code>a.UnionWith(b)</code>	alles wat in <code>a</code> óf <code>b</code> zit
<code>a.IntersectWith(b)</code>	enkel wat in <code>a</code> én <code>b</code> zit
<code>a.ExceptWith(b)</code>	wat in <code>a</code> zit maar niet in <code>b</code>

Die drie veranderen `a` zelf.

Wanneer wel, wanneer niet

Wel — dubbels mogen niet voorkomen, of je moet vaak en snel nagaan of iets er al in zit.

Niet — wanneer de volgorde ertoe doet, of wanneer je iets op een positie wil opvragen. Een `HashSet` heeft geen `[0]`.

Veelgemaakte fouten

LET OP

- Een dubbele Add crasht niet, hij geeft gewoon `false` terug. Controleer je dat niet, dan merk je nooit dat er iets dubbel binnenkwam.
- Er bestaat geen eerste, tweede of derde. Wil je op positie opvragen, dan is dit de verkeerde bouwsteen.

Voorspel de uitkomst

Vraag 1

```
var s = new HashSet<int> { 1, 2, 2, 3, 3, 3 };  
Console.WriteLine(s.Count);
```

Toon het antwoord

3

De dubbels worden stilzwijgend genegeerd. Er blijven drie verschillende getallen over.

Vraag 2

```
var s = new HashSet<string>();  
Console.WriteLine(s.Add("a"));  
Console.WriteLine(s.Add("a"));
```

Toon het antwoord

`True` en daarna `False`

De eerste keer is "a" nieuw. De tweede keer niet meer.

Vraag 3

```
var a = new HashSet<int> { 1, 2, 3 };  
var b = new HashSet<int> { 2, 3, 4 };  
a.IntersectWith(b);  
Console.WriteLine(a.Count);
```

Toon het antwoord

2

Enkel wat in beide zit, blijft over in a: 2 en 3.

9. Queue

VERZAMELING

Wat het is

Een **wachtrij**. Wie eerst komt, wordt eerst geholpen. Je kan niet voorsteken, en je kan niemand uit het midden halen.

Hoe het eruitziet

```
Queue<string> wachtrij = new Queue<string>();

wachtrij.Enqueue("Anna");           // achteraan aansluiten
wachtrij.Enqueue("Bram");
wachtrij.Enqueue("Cleo");

Console.WriteLine(wachtrij.Peek()); // Anna – kijken wie vooraan staat
string volgende = wachtrij.Dequeue(); // Anna – en nu is ze weg
Console.WriteLine(wachtrij.Count);   // 2
```

Wanneer wel, wanneer niet

Wel — dingen afhandelen in de volgorde waarin ze binnenkwamen: printopdrachten, berichten, klanten aan een loket.

Niet — zodra je iets in het midden wil bereiken of wil overslaan.

Veelgemaakte fouten

LET OP

- Dequeue of Peek op een lege wachtrij crasht. Controleer eerst Count, of gebruik TryDequeue.
- Peek kijkt enkel, Dequeue haalt weg. Wie ze verwisselt, verwerkt de eerste persoon eindeloos opnieuw.

Voorspel de uitkomst

Vraag 1

```
var q = new Queue<int>();  
q.Enqueue(1);  
q.Enqueue(2);  
q.Enqueue(3);  
q.Dequeue();  
Console.WriteLine(q.Peek());
```

Toon het antwoord

2

De 1 stond vooraan en is weggehaald. Nu staat de 2 vooraan.

Vraag 2

```
var q = new Queue<int>();  
q.Dequeue();
```

Toon het antwoord

`InvalidOperationException` — het programma crasht.

Er staat niemand in de rij.

10. Stack

VERZAMELING

Wat het is

Een stapel bord. Het bord dat je laatst neerlegt, neem je als eerste weer op. Het onderste bord raak je pas als alles erboven weg is.

Hoe het eruitziet

```
Stack<string> stappen = new Stack<string>();

stappen.Push("typ tekst");      // bovenop Leggen
stappen.Push("maak vet");
stappen.Push("wis regel");

Console.WriteLine(stappen.Pop()); // wis regel – bovenste eraf
Console.WriteLine(stappen.Peek()); // maak vet – kijken, niet wegnemen
```

Wanneer wel, wanneer niet

Wel — ongedaan maken, de terug-knop van een browser, nagaan of haakjes correct openen en sluiten. Overal waar het laatste het eerst weer aan de beurt komt.

Niet — wanneer iedereen eerlijk op zijn beurt moet wachten. Dan is het een Queue.

Veelgemaakte fouten

LET OP

- `Pop` of `Peek` op een lege stapel crasht, net als bij een wachtrij.
- Een `Stack` en een `Queue` met dezelfde inhoud geven hun items in omgekeerde volgorde terug. Dat is het hele verschil, en het is groot.

Voorspel de uitkomst

Vraag 1

```
var s = new Stack<int>();  
s.Push(1);  
s.Push(2);  
s.Push(3);  
Console.WriteLine(s.Pop() + s.Pop());
```

Toon het antwoord

5

De eerste Pop geeft 3, de tweede geeft 2. Samen 5. De 1 ligt nog onderaan.

Vraag 2

```
var q = new Queue<char>();  
var s = new Stack<char>();  
  
foreach (char c in "abc")  
{  
    q.Enqueue(c);  
    s.Push(c);  
}  
  
Console.WriteLine(q.Dequeue());  
Console.WriteLine(s.Pop());
```

Toon het antwoord

a en daarna c

Dezelfde drie letters erin, in dezelfde volgorde. De wachtrij geeft de eerste terug, de stapel de laatste.

Deel 3 — Zoeken en rekenen

Nu je de verzamelingen kent: hoe haal je er iets uit, en wat krijg je dan eigenlijk terug?

11. LINQ

METHODES

Dit is LINQ. Deze methodes werken op elke verzameling — een lijst, een array, een resultaat van een andere LINQ-methode, en later ook op een databasetabel. Ze veranderen nooit iets aan de bron; ze geven een nieuw resultaat.

Bijna allemaal willen ze een voorwaarde in de vorm `f => f.Prijs > 50`. Lees de pijl als "geef mij, voor elke f, het antwoord op".

Ik wil één item

Methode	Wat het doet	Als er niets is	Als er meerdere zijn
<code>First</code>	de eerste die voldoet	uitzondering	neemt de eerste
<code>FirstOrDefault</code>	de eerste die voldoet	<code>null</code>	neemt de eerste
<code>Single</code>	de enige die voldoet	uitzondering	uitzondering
<code>SingleOrDefault</code>	de enige die voldoet	<code>null</code>	uitzondering
<code>Last</code>	de laatste die voldoet	uitzondering	neemt de laatste
<code>LastOrDefault</code>	de laatste die voldoet	<code>null</code>	neemt de laatste

`fans.FirstOrDefault(f => f.Id == 3)` geeft de fan met Id 3, of `null`.

BELANGRIJK

Neem `FirstOrDefault` wanneer het normaal is dat er niets gevonden wordt — een Id dat niet bestaat, bijvoorbeeld. Dan controleer je op `null` en geef je een nette melding.

Neem `SingleOrDefault` wanneer er hoogstens één mag zijn en twee een fout in je gegevens betekent. De uitzondering die je dan krijgt is geen hinder maar een alarm: er staan twee items met hetzelfde Id in je lijst.

Neem `First` alleen wanneer je zeker weet dat er iets is. Anders crasht je API met een 500 op een geval dat eigenlijk een 404 moest zijn.

LET OP

`FirstOrDefault` geeft niet altijd `null`. Bij een lijst van getallen geeft het 0, en bij een lijst van waarheidswaarden `false`. De standaardwaarde van het type dus, niet noodzakelijk niets. Bij een lijst van objecten is het wel `null`.

Ik wil weten of iets bestaat

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Any</code>	is er minstens één	<code>fans.Any()</code>	<code>true</code> als de lijst niet leeg is
<code>Any</code>	met voorwaarde	<code>fans.Any(f => f.Prijs > 50)</code>	<code>true</code> of <code>false</code>
<code>All</code>	voldoen ze allemaal	<code>fans.All(f => f.Prijs > 0)</code>	<code>true</code> of <code>false</code>
<code>Contains</code>	zit deze waarde erin	<code>maten.Contains(120)</code>	<code>true</code> of <code>false</code>

Wil je enkel weten óf er iets is, gebruik dan `Any` en niet `Count() > 0`. `Any` stopt bij het eerste item dat voldoet, `Count` loopt de hele verzameling af.

LET OP

`All` op een lege verzameling geeft `true`. Wiskundig klopt dat — er is geen enkel item dat níet voldoet — maar het is zelden wat je bedoelde. Controleer eerst of er wel iets in zit.

Ik wil een deel van de verzameling

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Where</code>	enkel wat voldoet	<code>fans.Where(f => f.Merk == "NZXT")</code>	de NZXT-fans
<code>Select</code>	maak van elk item iets anders	<code>fans.Select(f => f.Merk)</code>	enkel de merken
<code>Distinct</code>	gooi dubbels weg	<code>merken.Distinct()</code>	elk merk één keer
<code>Take</code>	de eerste n	<code>fans.Take(3)</code>	drie items
<code>Skip</code>	sla de eerste n over	<code>fans.Skip(3)</code>	de rest
<code>SelectMany</code>	plat een lijst van lijsten	<code>borden.SelectMany(b => b.Sloten)</code>	alle sloten samen

`Where` filtert, `Select` verbouwt. Dat is het hele verschil. Wil je de dure fans, dan is dat `Where`. Wil je van elke fan enkel het merk, dan is dat `Select`. Wil je de merken van de dure fans, dan is dat allebei achter elkaar.

Ik wil tellen of rekenen

Methode	Wat het doet	Voorbeeld	Resultaat
<code>Count</code>	hoeveel er zijn	<code>fans.Count()</code>	4
<code>Count</code>	hoeveel er voldoen	<code>fans.Count(f => f.Prijs > 50)</code>	2
<code>Sum</code>	tel op	<code>lijnen.Sum(l => l.Aantal * l.StukPrijs)</code>	het totaal
<code>Min</code>	de kleinste waarde	<code>fans.Min(f => f.Prijs)</code>	19.99
<code>Max</code>	de grootste waarde	<code>fans.Max(f => f.Prijs)</code>	89.99
<code>Average</code>	het gemiddelde	<code>fans.Average(f => f.Prijs)</code>	42.5
<code>MinBy</code>	het item met de kleinste waarde	<code>fans.MinBy(f => f.Prijs)</code>	de goedkoopste fan
<code>MaxBy</code>	het item met de grootste waarde	<code>fans.MaxBy(f => f.Prijs)</code>	de duurste fan

`Max` geeft het getal, `MaxBy` geeft het item. Wil je weten wat de hoogste prijs is, dan is het `Max`. Wil je weten wélke fan dat is, dan is het `MaxBy`.

Ik wil ordenen of groeperen

Methode	Wat het doet	Voorbeeld	Resultaat
<code>OrderBy</code>	sorteer oplopend	<code>fans.OrderBy(f => f.Prijs)</code>	goedkoopste eerst
<code>OrderByDescending</code>	sorteer aflopend	<code>fans.OrderByDescending(f => f.Prijs)</code>	duurste eerst
<code>ThenBy</code>	tweede sorteersleutel	<code>fans.OrderBy(f => f.Merk).ThenBy(f => f.Prijs)</code>	per merk, dan op prijs
<code>GroupBy</code>	maak groepen	<code>fans.GroupBy(f => f.Merk)</code>	een groep per merk

Ik wil het resultaat vastleggen

Methode	Wat het doet	Voorbeeld	Resultaat
<code>ToList</code>	maak er een echte lijst van	<code>fans.Where(f => f.Prijs > 50).ToList()</code>	een <code>List</code>
<code>ToArray</code>	maak er een array van	<code>fans.ToArray()</code>	een array
<code>ToDictionary</code>	maak er een opzoektabel van	<code>fans.ToDictionary(f => f.Id)</code>	een <code>Dictionary</code>

BELANGRIJK

Where, Select en OrderBy voeren op het moment dat je ze schrijft nog niets uit. Wat dat betekent, en waarom het je kan verrassen, is het onderwerp van het volgende hoofdstuk. De vuistregel voor nu: wat je aan de client van een API teruggeeft, geef je pas terug na een `ToList()`.

Voorspel de uitkomst

Vraag 1

```
var getallen = new List<int> { 5, 8, 3, 8 };
Console.WriteLine(getallen.First(g => g > 4));
Console.WriteLine(getallen.Last(g => g > 4));
```

Toon het antwoord

5 en daarna 8

Groter dan 4 zijn 5, 8 en 8. First neemt de eerste daarvan, Last de laatste.

Vraag 2

```
var getallen = new List<int> { 5, 8, 3 };
Console.WriteLine(getallen.FirstOrDefault(g => g > 10));
```

Toon het antwoord

0

Er is geen getal groter dan 10. `FirstOrDefault` geeft dan de standaardwaarde van het type terug, en bij een `int` is dat nul, niet `null`. Pas op: een 0 kan hier dus zowel "niets gevonden" als "ik vond een nul" betekenen.

Vraag 3

```
var getallen = new List<int> { 5, 8, 3, 8 };  
var x = getallen.SingleOrDefault(g => g == 8);
```

Toon het antwoord

`InvalidOperationException` — het programma crasht.

Er zijn twee achten. `SingleOrDefault` mag er nul of één vinden, twee is voor hem een fout. Dat is precies waarvoor je hem gebruikt: als alarm wanneer iets dat uniek moet zijn het niet is.

Vraag 4

```
var leeg = new List<int>();  
Console.WriteLine(leeg.All(g => g > 100));  
Console.WriteLine(leeg.Any());
```

Toon het antwoord

`True` en daarna `False`

In een lege lijst is er geen enkel getal dat niet groter is dan 100, dus `All` zegt `true`. Tegelijk zit er niets in, dus `Any` zegt `false`. Twee antwoorden die elkaar lijken tegen te spreken, en allebei correct.

Vraag 5

```
var namen = new List<string> { "Bram", "Anna", "Cleo" };  
var gesorteerd = namen.OrderBy(n => n);  
Console.WriteLine(namen[0]);
```

Toon het antwoord

`Bram`

`OrderBy` laat de oorspronkelijke lijst met rust. Het gesorteerde resultaat zit in `gesorteerd`, niet in `namen`. Vergelijk met `Sort` in hoofdstuk 6, die de lijst zelf verandert.

Vraag 6

```
var getallen = new List<int> { 1, 2, 3, 4 };  
Console.WriteLine(getallen.Where(g => g > 2).Count());  
Console.WriteLine(getallen.Select(g => g > 2).Count());
```

Toon het antwoord

2 en daarna 4

Where houdt enkel 3 en 4 over. Select houdt alles over, maar maakt van elk getal een ja of nee: false, false, true, true. Vier antwoorden, dus 4.

Vraag 7

```
var woorden = new List<string> { "kat", "olifant", "muis" };  
Console.WriteLine(woorden.Max(w => w.Length));  
Console.WriteLine(woorden.MaxBy(w => w.Length));
```

Toon het antwoord

7 en daarna olifant

Max geeft de grootste lengte. MaxBy geeft het woord dat die lengte heeft.

12. IEnumerable

VERZAMELING

Wat het is

Geen verzameling, maar een belofte: "je kan mij één voor één doorlopen". Zoals een playlist: je weet dat je nummer na nummer kan afspelen, maar niet noodzakelijk hoeveel er zijn of wat het vijfde is.

Elke verzameling uit deel 2 is óók een `IEnumerable`. Een array, een `List`, een `HashSet` — ze kunnen allemaal doorlopen worden. En alles wat LINQ teruggeeft, zoals het resultaat van `Where`, is een `IEnumerable`.

Hoe het eruitziet

```
List<int> getallen = new List<int> { 1, 2, 3, 4 };  
  
IEnumerable<int> even = getallen.Where(g => g % 2 == 0);  
  
getallen.Add(6);  
  
Console.WriteLine(even.Count());    // 3 – de 6 telt mee!
```

Dat laatste verrast iedereen de eerste keer. `even` is geen lijst met 2 en 4. Het is een vraag — *geef mij de even getallen uit die lijst* — die pas gesteld wordt op het moment dat je het antwoord nodig hebt. En op dat moment zit de 6 er al in.

Wanneer wel, wanneer niet

Wel — als parameter van een methode die enkel moet doorlopen. Wie die methode aanroept, mag dan een array, een `List` of een `HashSet` meegeven.

Niet — zodra je `Count`, een positie of `Add` nodig hebt. Maak er dan eerst met `ToList()` een echte lijst van.

Veelgemaakte fouten

LET OP

- Elke keer dat je een `IEnumerable` doorloopt, wordt de vraag opnieuw gesteld. Twee keer `Count()` op hetzelfde resultaat is twee keer het werk.
- Verandert de bron tussendoor, dan verandert het antwoord mee.
- Er is geen `[0]` en geen `.Count` zonder haakjes. Dat zijn dingen van een `List`, niet van een belofte.

Voorspel de uitkomst

Vraag 1

```
var getallen = new List<int> { 1, 2, 3 };  
var groot = getallen.Where(g => g > 1);  
getallen.Add(10);  
Console.WriteLine(groot.Count());
```

Toon het antwoord

3

2, 3 en 10. De vraag werd pas gesteld bij `Count()`, en toen zat de 10 er al in.

Vraag 2

```
var getallen = new List<int> { 1, 2, 3 };  
var groot = getallen.Where(g => g > 1).ToList();  
getallen.Add(10);  
Console.WriteLine(groot.Count());
```

Toon het antwoord

2

Enkel 2 en 3. `ToList()` stelde de vraag meteen en legde het antwoord vast. Wat er nadien bijkomt, telt niet meer mee. Vergelijk met vraag 1: één woord verschil, een ander antwoord.

Deel 4 — Eigen types

Tot hier ging het over wat C# je geeft. Nu over de dingen die je zelf maakt — en over het idee dat bepaalt hoe ze zich gedragen.

13. class

EIGEN TYPE

Wat het is

Een bouwplan. De klasse is het plan, het object is het huis dat ernaar gebouwd is. Met één plan bouw je zoveel huizen als je wil, en elk huis heeft zijn eigen bewoners, zijn eigen kleur deur.

Hoe het eruitziet

```
public class Leerling
{
    public string Naam { get; set; } = "";
    public int Leeftijd { get; set; }

    public bool IsVolwassen()
    {
        return Leeftijd >= 18;
    }
}
```

En zo gebruik je het:

```
Leerling anna = new Leerling { Naam = "Anna", Leeftijd = 17 };

Console.WriteLine(anna.Naam);           // Anna
Console.WriteLine(anna.IsVolwassen());  // False

anna.Leeftijd = 18;                     // aanpassen mag
Console.WriteLine(anna.IsVolwassen());  // True
```

Wil je afdwingen dat een leerling altijd met een naam gemaakt wordt, dan geef je de klasse een **constructor** — een methode met dezelfde naam als de klasse, die draait op het moment van `new`:

```
public class Leerling
{
    public string Naam { get; set; }

    public Leerling(string naam)
    {
        Naam = naam;
    }
}

Leerling bram = new Leerling("Bram");
```

Eigenschappen die zichzelf uitrekenen: =>

Een eigenschap hoeft geen waarde te bewaren. Met `=>` rekent ze haar waarde uit, telkens opnieuw wanneer je erom vraagt:

```
public class Persoon
{
    public string Voornaam { get; set; } = "";
    public string Achternaam { get; set; } = "";

    public string VolledigeNaam => Voornaam + " " + Achternaam;
}
```

`VolledigeNaam` heeft geen `set`: je kan ze lezen, maar niet invullen. Verander je de voornaam, dan verandert de volledige naam vanzelf mee. Dezelfde pijl kan ook een korte methode schrijven: `public bool IsVolwassen() => Leeftijd >= 18;` doet hetzelfde als de versie met `return` hierboven.

In les 07 kom je deze schrijfwijze tegen bij de eigenschappen van de Unit of Work.

Waar een klasse woont: namespace en using

Elke klasse staat in een **namespace**, een soort map voor code. Zo kunnen twee klassen dezelfde naam hebben, zolang ze in een andere namespace staan.

```
namespace School.Models;

public class Leerling
{
    // ...
}
```

Wil je die klasse in een ander bestand gebruiken, dan zeg je bovenaan met `using` in welke map C# moet kijken:

```
using School.Models;

var anna = new Leerling();
```

Je zal twee schrijfwijzen tegenkomen. `namespace School.Models;` met een puntkomma geldt voor het hele bestand. De oudere vorm zet de klasse tussen accolades: `namespace School.Models { ... }`. Ze doen hetzelfde.

Een paar namespaces staan in een nieuw project al klaar zonder dat je ze typt, zoals `System`, `System.Linq` en `System.Collections.Generic`. Daarom kan je `List` en `Where` gebruiken zonder één `using`.

Een object zonder klasse: anonieme objecten

Soms wil je even een paar waarden samen doorgeven zonder er een klasse voor te schrijven:

```
var p = new { Naam = "Anna", Leeftijd = 17 };

Console.WriteLine(p.Naam);           // Anna
// p.Naam = "Bram";                  compileert niet: enkel lezen
```

Het type heeft geen naam, daarom moet de variabele `var` zijn. In les 02 zie je dit bij `CreatedAtAction(..., new { id = laptop.Id }, ...)`: dat kleine object zegt welke waarde er in de route moet komen.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>{ get; set; }</code>	een eigenschap die je kan lezen en invullen	<code>public int Leeftijd { get; set; }</code>	<code>anna.Leeftijd = 18;</code> mag
<code>{ get; }</code>	enkel lezen; invullen kan alleen in de constructor	<code>public string Naam { get; }</code>	<code>anna.Naam = "x";</code> compileert niet
<code>=></code> (eigenschap)	een eigenschap die zichzelf uitrekent	<code>public string VolledigeNaam => ...</code>	rekent telkens opnieuw
<code>=></code> (methode)	een methode van één regel	<code>public bool IsVolwassen() => Leeftijd >= 18;</code>	zoals met <code>return</code>
<code>new Klasse { ... }</code>	maak een object en vul meteen eigenschappen in	<code>new Leerling { Naam = "Anna" }</code>	een leerling met een naam
constructor	draait bij <code>new</code> en kan iets afdwingen	<code>public Leerling(string naam) { ... }</code>	<code>new Leerling()</code> compileert niet meer
<code>namespace</code>	de map waarin een klasse woont	<code>namespace School.Models;</code>	volledige naam <code>School.Models.Leerling</code>
<code>using</code>	zeg waar C# moet zoeken	<code>using School.Models;</code>	<code>Leerling</code> is bruikbaar
<code>new { ... }</code>	een anoniem object	<code>new { id = 5 }</code>	enkel te lezen

LET OP

Krijg je de fout dat een type of namespace niet gevonden wordt, dan ontbreekt meestal een `using`. Visual Studio stelt die zelf voor: zet je cursor op de rode naam en druk op `Ctrl+.`

Wanneer wel, wanneer niet

Wel — de standaardkeuze. Iets met gegevens die in de loop van de tijd veranderen, en met gedrag: een leerling die ouder wordt, een bankrekening waarop je stort.

Niet — wanneer het object enkel gegevens draagt en twee objecten met dezelfde inhoud als gelijk moeten gelden. Dan is een `record` handiger.

Veelgemaakte fouten

LET OP

- `Leerling b = a;` maakt geen kopie. Het geeft hetzelfde object een tweede naam, net als bij een array.
- `a == b` zegt of het **hetzelfde** object is, niet of ze **dezelfde inhoud** hebben. Twee leerlingen met dezelfde naam zijn voor een klasse niet gelijk.
- Een klassevariabele zonder `new` is `null`. Ernaar vragen crasht.

Voorspel de uitkomst

Deze vragen gebruiken de eerste versie van `Leerling`, zonder constructor.

Vraag 1

```
var a = new Leerling { Naam = "Anna" };
var b = a;
b.Naam = "Bram";
Console.WriteLine(a.Naam);
```

Toon het antwoord

Bram

`a` en `b` zijn twee namen voor één en dezelfde leerling. Hernoem je haar via `b`, dan heet ze ook via `a` anders.

Vraag 2

```
var a = new Leerling { Naam = "Anna" };
var b = new Leerling { Naam = "Anna" };
Console.WriteLine(a == b);
```

Toon het antwoord

False

Twee keer `new` is twee objecten. Dezelfde naam maakt ze niet tot dezelfde leerling — net zoals twee echte mensen die Anna heten niet dezelfde persoon zijn.

Vraag 3

Deze vraag gebruikt de klasse `Persoon` van hierboven.

```
var p = new Persoon { Voornaam = "Anna", Achternaam = "Peeters" };  
p.Voornaam = "Bram";  
Console.WriteLine(p.VolledigeNaam);
```

Toon het antwoord

Bram Peeters

`VolledigeNaam` bewaart niets. Ze wordt pas samengesteld op het moment dat je erom vraagt, en dan is de voornaam al Bram.

Vraag 4

```
var p = new { Naam = "Anna" };  
p.Naam = "Bram";
```

Toon het antwoord

Compileert niet.

De eigenschappen van een anoniem object kan je enkel lezen. Moet een waarde kunnen veranderen, schrijf dan een echte klasse.

14. record

EIGEN TYPE

Wat het is

Een klasse die draait om haar inhoud. Twee records met exact dezelfde gegevens zijn gelijk. Zoals twee briefjes van tien euro: het maakt niet uit welk exemplaar je hebt, enkel wat erop staat.

Hoe het eruitziet

```
public record Punt(int X, int Y);
```

Die ene regel maakt een type met twee eigenschappen, een constructor, en een nette weergave. En zo gebruik je het:

```
var p1 = new Punt(1, 2);
var p2 = new Punt(1, 2);

Console.WriteLine(p1 == p2);    // True - zelfde inhoud, dus gelijk
Console.WriteLine(p1);          // Punt { X = 1, Y = 2 }

var p3 = p1 with { Y = 5 };     // een kopie met één waarde anders
Console.WriteLine(p3);          // Punt { X = 1, Y = 5 }
Console.WriteLine(p1);          // Punt { X = 1, Y = 2 } - ongewijzigd
```

Wanneer wel, wanneer niet

Wel — een pakketje gegevens dat je doorgeeft en niet meer verandert. In les 09 van Programming Advanced kom je records tegen als een van de twee manieren om DTO's te schrijven; zie [hoofdstuk 33](#).

Niet — wanneer het object in de loop van de tijd verandert en zijn eigen identiteit heeft. Een leerling die ouder wordt blijft dezelfde leerling; dat is een `class`.

Veelgemaakte fouten

LET OP

- De eigenschappen van `record Punt(int X, int Y)` kan je na het aanmaken niet meer aanpassen. `p1.X = 5;` compileert niet.
- `with` verandert niets aan het origineel. Het maakt een nieuw record. Wie het resultaat niet opvangt, heeft niets gedaan.

Voorspel de uitkomst

Vraag 1

```
var p1 = new Punt(1, 2);  
var p2 = p1 with { X = 9 };  
Console.WriteLine(p1.X);
```

Toon het antwoord

1

with maakte een kopie waarin X 9 is. Het origineel bleef zoals het was.

Vraag 2

```
var p1 = new Punt(1, 2);  
p1.X = 5;
```

Toon het antwoord

Compileert niet.

Bij deze korte schrijfwijze kan je de eigenschappen enkel invullen bij het aanmaken. Wil je een ander punt, maak dan een nieuw — met with of met new.

Vraag 3

```
Console.WriteLine(new Punt(3, 4));
```

Toon het antwoord

Punt { X = 3, Y = 4 }

Een record weet uit zichzelf hoe het zich moet tonen. Een gewone klasse toont enkel haar naam — dat is een van de redenen waarom records zo handig zijn om mee te testen.

15. struct

EIGEN TYPE

Wat het is

Een klein pakketje waarden dat bij kopiëren écht gekopieerd wordt. Zoals een fotokopie: wat je op de kopie schrijft, staat niet op het origineel.

Je gebruikt ze al zonder het te weten. Een `int`, een `double` en een `bool` zijn allemaal structs.

Hoe het eruitziet

```
public struct Kleur
{
    public int R;
    public int G;
    public int B;
}

Kleur a = new Kleur { R = 255, G = 0, B = 0 };
Kleur b = a;           // een echte kopie
b.R = 0;

Console.WriteLine(a.R); // 255 – a is niet veranderd
Console.WriteLine(b.R); // 0
```

Vergelijk dit met vraag 1 in hoofdstuk 13. Dezelfde drie regels, een tegengesteld resultaat.

Wanneer wel, wanneer niet

Wel — zelden zelf nodig. Een klein ding van een paar getallen dat zich gedraagt als één waarde: een kleur, een coördinaat.

Niet — bijna altijd. Twijfel je, neem een `class` of een `record`.

Veelgemaakte fouten

LET OP

- Een struct in een lijst aanpassen past de kopie aan, niet wat in de lijst staat. Dat is de klassieke valkuil, en de reden waarom je ze best klein en onveranderlijk houdt.

Voorspel de uitkomst

Vraag 1

```
int x = 5;  
int y = x;  
y = 10;  
Console.WriteLine(x);
```

Toon het antwoord

5

Een `int` is een struct. `y = x` kopieerde de waarde 5. Wat je daarna met `y` doet, raakt `x` niet. Dit gedrag ken je al sinds je eerste les — het heeft nu gewoon een naam.

16. Referentie tegenover waarde

CONCEPT

Dit is het belangrijkste idee uit dit hele leerboek. Als je één ding onthoudt, laat het dit zijn.

Er zijn twee soorten types in C#, en ze gedragen zich tegengesteld wanneer je ze aan een tweede variabele geeft.

	Referentietype	Waardetype
Voorbeelden	class, record, array, List, Dictionary, alle verzamelingen	int, double, bool, struct, enum
b = a doet	geeft hetzelfde ding een tweede naam	maakt een echte kopie
Via b iets aanpassen	zie je ook via a	raakt a niet
Vergelijking	een huissleutel: twee sleutels, één huis	een fotokopie: twee vellen, elk apart
Kan null zijn	ja	nee, tenzij je ? achter het type zet

OPMERKING

string is technisch een referentietype, maar gedraagt zich als een waarde: je kan een tekst nooit aanpassen, enkel vervangen door een nieuwe. Daarom verrast hij je nooit op de manier waarop een lijst dat doet.

Test jezelf: kijk terug naar vraag 4 in hoofdstuk 5, vraag 1 in hoofdstuk 13 en de vraag in hoofdstuk 15. Drie keer dezelfde drie regels, en het antwoord hangt enkel af van de soort type. De vraag onderaan dit hoofdstuk brengt het samen.

class, record en struct naast elkaar

	class	record	struct
Soort	referentie	referentie	waarde
a == b vergelijkt	is het hetzelfde object	is de inhoud gelijk	is de inhoud gelijk
Aanpassen na het maken	ja	met de korte schrijfwijze: nee	ja, maar liever niet
Zichzelf tonen	enkel de naam van het type	alle eigenschappen	enkel de naam van het type
Typisch gebruik	iets met gedrag en een identiteit	een pakketje gegevens	een klein getalachtig ding

Voorspel de uitkomst

Vraag 1

Dit is de vraag die alles samenbrengt. Twee methodes: de ene krijgt een lijst mee, de andere een getal.

```
void VoegToe(List<int> lijst)
{
    lijst.Add(99);
}

void Verhoog(int getal)
{
    getal = getal + 1;
}

var l = new List<int> { 1 };
int n = 1;

VoegToe(l);
Verhoog(n);

Console.WriteLine(l.Count);
Console.WriteLine(n);
```

Toon het antwoord

2 en daarna 1

Een lijst is een referentietype. VoegToe krijgt geen kopie van de lijst, maar een tweede sleutel tot dezelfde lijst. Wat de methode erin stopt, zit er daarna nog in.

Een int is een waardetype. Verhoog krijgt een fotokopie van het getal. Het verhoogt die kopie, en de kopie verdwijnt zodra de methode klaar is. n blijft 1.

Dit is de reden waarom een lijst die je aan een methode meegeeft na afloop soms "vanzelf" veranderd is.

17. Omgaan met niets

CONCEPT

In het vorige hoofdstuk zag je dat een referentietype `null` kan zijn en een waardetype niet — tenzij je er een vraagteken achter zet. Dit hoofdstuk gaat over hoe je daar veilig mee omgaat.

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>??</code>	neem dit, tenzij het null is	<code>naam ?? "onbekend"</code>	"onbekend" als <code>naam</code> null is
<code>??=</code>	zet enkel als het nog null is	<code>_repo ??= new Repository()</code>	vult één keer in
<code>?.</code>	roep enkel aan als het niet null is	<code>fan?.Merk</code>	<code>null</code> in plaats van een crash
<code>is null</code>	is het niets	<code>if (fan is null)</code>	<code>true</code> of <code>false</code>
<code>is not null</code>	is het iets	<code>if (fan is not null)</code>	<code>true</code> of <code>false</code>
<code>int?</code>	een getal dat ook niets mag zijn	<code>int? hoogte = null</code>	mag leeg blijven
<code>HasValue</code>	zit er een waarde in	<code>hoogte.HasValue</code>	<code>false</code>
<code>GetValueOrDefault</code>	de waarde, of een terugval	<code>hoogte.GetValueOrDefault()</code>	<code>0</code>
<code>!</code> achter een waarde	zeg tegen de compiler: dit is niet null	<code>naam!.Length</code>	geen waarschuwing; is het tóch null, dan crasht het
<code>= default!</code>	begin met niets, zonder waarschuwing	<code>public Stad Stad { get; set; } = default!;</code>	de eigenschap is null tot iemand ze invult

Het vraagteken achter een type betekent: deze waarde mag ontbreken. Dat is de manier waarop je in code het verschil vastlegt tussen "nul" en "niet ingevuld" — precies het onderscheid waar een controle op stuk loopt als je het niet maakt.

Het uitroepteken: vertrouw me

In een nieuw project waarschuwt de compiler zodra een referentietype zonder vraagteken `null` zou kunnen zijn, zoals een `string`-eigenschap zonder beginwaarde. Het uitroepteken zet die waarschuwing uit. Aan wat er gebeurt terwijl het programma draait, verandert het niets: het is een belofte aan de compiler, geen controle.

In de cursus zie je het op twee plaatsen:

- `= default!` achter een eigenschap, vanaf les 06. De eigenschap begint als null, maar wordt later ingevuld, bijvoorbeeld door EF Core bij het ophalen. Zie [hoofdstuk 32](#). `= null!` betekent hetzelfde.
- `src.Product!.Naam` in een regel voor Mapster, in les 09. Daar zet het enkel de waarschuwing uit. Mapster kijkt zelf na of het product er is, en laat de naam leeg als het ontbreekt. Zie [hoofdstuk 33](#).

In gewone code geldt: klopt de belofte niet, dan krijg je dezelfde `NullReferenceException` als zonder uitroepteken, alleen zonder waarschuwing vooraf.

Voorspel de uitkomst

Vraag 1

```
string? naam = null;  
Console.WriteLine(naam ?? "onbekend");
```

Toon het antwoord

onbekend

naam is null, dus ?? neemt wat rechts staat.

Vraag 2

```
string? naam = null;  
Console.WriteLine(naam?.Length);
```

Toon het antwoord

Een lege regel.

naam?.Length vraagt de lengte enkel op als naam niet null is. Anders is het resultaat zelf null, en Console.WriteLine toont null als niets. Geen crash.

Vraag 3

```
string? naam = null;  
Console.WriteLine(naam.Length);
```

Toon het antwoord

NullReferenceException — het programma crasht.

Het enige verschil met vraag 2 is het vraagteken. Zonder ?. vraag je de lengte op van iets dat er niet is. Visual Studio waarschuwt je hier wel met een groene kronkellijn, maar compileren gaat.

Vraag 4

```
int? score = null;  
Console.WriteLine(score.HasValue);  
Console.WriteLine(score.GetValueOrDefault());
```

Toon het antwoord

False en daarna 0

Er zit geen waarde in. GetValueOrDefault geeft dan de standaardwaarde van een int: nul. Merk op dat je nu wél kan zien dat de score ontbrak, via HasValue. Bij een gewone int kon dat niet.

Vraag 5

```
string? a = null;  
a ??= "eerste";  
a ??= "tweede";  
Console.WriteLine(a);
```

Toon het antwoord

eerste

??= vult enkel in als er nog niets staat. De eerste keer was a null, de tweede keer niet meer.

Vraag 6

```
string? naam = null;  
Console.WriteLine(naam!.Length);
```

Toon het antwoord

NullReferenceException — het programma crasht.

Hetzelfde als bij vraag 3, maar nu zonder groene kronkellijn. Het uitroepteken zegt de compiler dat naam niet null is. De compiler gelooft het, en zwijgt. Het programma zelf kijkt niet na, en loopt vast op de lengte van iets dat er niet is.

18. enum

EIGEN TYPE

Wat het is

Een vaste lijst benoemde keuzes. Een verkeerslicht staat op rood, oranje of groen. Niet op paars, niet op "rood" met drie o's.

Hoe het eruitziet

```
public enum Verkeerslicht
{
    Rood,
    Oranje,
    Groen
}
```

En zo gebruik je het:

```
Verkeerslicht licht = Verkeerslicht.Rood;

if (licht == Verkeerslicht.Rood)
{
    Console.WriteLine("Stop");
}

switch (licht)
{
    case Verkeerslicht.Rood:
        Console.WriteLine("Wachten");
        break;
    case Verkeerslicht.Groen:
        Console.WriteLine("Rijden");
        break;
}

Console.WriteLine(licht);           // Rood
Console.WriteLine((int)licht);     // 0
```

Tekst omzetten naar een enum gaat met `Enum.TryParse`, net als bij getallen:

```
if (Enum.TryParse("Groen", out Verkeerslicht gekozen))
{
    Console.WriteLine(gekozen);    // Groen
}
```

Wanneer wel, wanneer niet

Wel — een eigenschap heeft een klein, vast aantal mogelijke waarden die niet veranderen terwijl je programma draait: dagen van de week, een status, een richting. Het grote voordeel tegenover een `string`: een tikfout wordt een compileerfout in plaats van een fout die pas opduikt als het te laat is.

Niet — wanneer de lijst van keuzes groeit terwijl het programma draait, of door gebruikers aangevuld wordt. Een enum wijzig je enkel door je code te herschrijven.

Veelgemaakte fouten

LET OP

- Onder de motorkap is elke keuze een getal: Rood is 0, Oranje 1, Groen 2. Voeg je vooraan een keuze toe, dan schuiven alle getallen op. Staan die getallen ergens opgeslagen, dan betekenen ze plots iets anders.
- `(Verkeerslicht)7` compileert, ook al bestaat er geen zevende kleur.
- `Enum.TryParse` is standaard hoofdlettergevoelig.

Voorspel de uitkomst

Vraag 1

```
Console.WriteLine((int)Verkeerslicht.Groen);
```

Toon het antwoord

2

Tellen begint bij nul: Rood is 0, Oranje 1, Groen 2.

Vraag 2

```
Verkeerslicht l = (Verkeerslicht)7;  
Console.WriteLine(l);
```

Toon het antwoord

7

C# laat het toe, en omdat er geen naam bij 7 hoort, toont het gewoon het getal. Een enum beschermt je dus tegen tikfouten in je code, maar niet tegen elk getal dat van buiten binnenkomt.

Vraag 3

```
bool ok = Enum.TryParse("groen", out Verkeerslicht l);  
Console.WriteLine(ok);
```

Toon het antwoord

False

"groen" met kleine letter is niet Groen. Wil je dat het wel lukt, dan geef je een extra `true` mee om hoofdletters te negeren.

19. interface

EIGEN TYPE

Wat het is

Een contract. Het zegt wát iets moet kunnen, niet hóe. Zoals een stopcontact: elk toestel met de juiste stekker past erin, of het nu een lamp is of een waterkoker. Het stopcontact hoeft niet te weten wat je aansluit.

Hoe het eruitziet

```
public interface IGeluid
{
    string MaakGeluid();
}
```

Het contract zegt enkel: *wie mij belooft, kan een geluid maken*. Geen uitwerking. Die komt in de klassen die de belofte doen:

```
public class Hond : IGeluid
{
    public string MaakGeluid()
    {
        return "Woef";
    }
}

public class Kat : IGeluid
{
    public string MaakGeluid()
    {
        return "Miauw";
    }
}
```

En nu het nut ervan:

```
List<IGeluid> dieren = new List<IGeluid> { new Hond(), new Kat() };

foreach (IGeluid dier in dieren)
{
    Console.WriteLine(dier.MaakGeluid());
}
```

De lus weet niet of hij een hond of een kat voor zich heeft. Dat hoeft ook niet: hij weet dat ze allebei een geluid kunnen maken, en meer heeft hij niet nodig.

Waarom dit in de cursus zo belangrijk is

In les 03 van Programming Advanced maak je een repository met een interface ervoor. Dat is precies dit principe. De controller praat met het contract, niet met de klasse. Daardoor kan de klasse achter het contract later vervangen worden — een lijst in het geheugen wordt een database — zonder dat de controller één letter verandert.

Wanneer wel, wanneer niet

Wel — wanneer verschillende klassen hetzelfde moeten kunnen, of wanneer je wil dat een deel van je programma later vervangbaar is zonder de rest aan te passen.

Niet — voor een klasse waarvan er maar één soort ooit zal bestaan en die je nooit zal vervangen. Een contract met maar één ondertekenaar voegt weinig toe.

Moeten verwante klassen niet alleen hetzelfde kunnen, maar ook gegevens of uitgewerkte methodes delen, dan is een abstracte klasse het betere antwoord. Zie [hoofdstuk 20](#).

Veelgemaakte fouten

LET OP

- Van een interface kan je geen object maken. `new IGeluid()` compileert niet — je kan geen stopcontact aansluiten op een ander stopcontact.
- Beloofd een klasse het contract, dan moet ze **alles** uitwerken wat erin staat. Eén methode vergeten en het compileert niet.
- Volgens de afspraak begint de naam met een hoofdletter **I**.

Voorspel de uitkomst

Vraag 1

Wat verschijnt er op het scherm bij de lus hierboven?

Toon het antwoord

Woef en daarna Miauw

Elke klasse gebruikt haar eigen uitwerking, ook al weet de lus enkel dat ze `IGeluid` beloven.

Vraag 2

```
IGeluid g = new IGeluid();
```

Toon het antwoord

Compileert niet.

Een interface is een belofte zonder uitwerking. Er valt niets te bouwen.

Vraag 3

```
IGeluid g = new Kat();  
Console.WriteLine(g is Kat);
```

Toon het antwoord

True

De variabele zegt enkel "iets dat een geluid maakt", maar het object dat erin zit is nog altijd een kat. Met `is` kan je dat nagaan.

20. Overerving en abstracte klassen

EIGEN TYPE

Wat het is

Een klasse die verder bouwt op een andere. Zoals een basisrecept voor deeg: pizza en brood beginnen allebei met hetzelfde deeg, en voegen er elk hun eigen stappen aan toe. Wie het basisrecept aanpast, past het voor allebei aan.

De klasse waarop je verder bouwt, heet de **basisklasse**. De klasse die erop verder bouwt, heet de **afgeleide klasse**. Die krijgt alles mee wat de basisklasse heeft, en voegt er haar eigen dingen aan toe.

Hoe het eruitziet

```
public class Voertuig
{
    public int Wielen { get; set; }

    public string Toeter()
    {
        return "Tuut";
    }
}

public class Fiets : Voertuig
{
    public bool HeeftBel { get; set; }
}
```

Het dubbelpunt na `Fiets` betekent: *een fiets is een voertuig*. Een fiets heeft dus ook `Wielen` en kan ook `Toeter()`, zonder dat die er opnieuw in staan:

```
Fiets f = new Fiets { Wielen = 2, HeeftBel = true };

Console.WriteLine(f.Wielen);      // 2 – komt van Voertuig
Console.WriteLine(f.Toeter());    // Tuut – komt van Voertuig
Console.WriteLine(f.HeeftBel);    // True – van de fiets zelf

Voertuig v = f;                   // mag: een fiets is een voertuig
Console.WriteLine(v.Wielen);      // 2
// v.HeeftBel compileert niet: via v zie je enkel wat elk voertuig heeft
```

Een basisklasse die zelf nooit bestaat: abstract

Een "voertuig" op zich rijdt nergens rond. Wat op de weg staat, is altijd een fiets, een bus of iets anders. Met `abstract` leg je dat vast: van de basisklasse zelf kan je geen object meer maken, enkel van wat ervan erft.

```
public abstract class Voertuig
{
    public int Wielen { get; }
    protected int Kilometers { get; set; }

    protected Voertuig(int wielen)
    {
        Wielen = wielen;
    }

    public void Rij(int km)
    {
        Kilometers += km;
    }

    public virtual string Toeter()
    {
        return "Tuut";
    }

    public abstract int MaxPassagiers();
}

public class Fiets : Voertuig
{
    public Fiets() : base(2) { }

    public override string Toeter()
    {
        return "Tring";
    }

    public override int MaxPassagiers()
    {
        return 1;
    }
}

public class Bus : Voertuig
{
    public Bus() : base(6) { }

    public override int MaxPassagiers()
    {
        return 50;
    }
}
```

Wat daar gebeurt:

- `abstract` vóór `class`: een object maken van `Voertuig` zelf compileert niet meer.
- `protected Voertuig(int wielen)` is de constructor van de basisklasse. Met `: base(2)` roept de fiets hem aan met twee wielen.
- `protected int Kilometers` is zichtbaar in `Voertuig` en in elke klasse die ervan erft, maar niet daarbuiten.
- `virtual` geeft `Toeter` een standaardversie die een afgeleide klasse mag vervangen. De fiets doet dat met `override`, de bus niet.
- `abstract` vóór een methode betekent: geen uitwerking. Elke afgeleide klasse moet ze zelf uitwerken, met `override`.

En het nut ervan:

```
List<Voertuig> vloot = new List<Voertuig> { new Fiets(), new Bus() };

foreach (Voertuig v in vloot)
{
    Console.WriteLine($"{v.Wielen} wielen, {v.Toeter()}, max {v.MaxPassagiers()}");
}
// 2 wielen, Tring, max 1
// 6 wielen, Tuut, max 50
```

De lus weet niet of ze een fiets of een bus voor zich heeft. Dat hoeft ook niet: elk voertuig heeft wielen, kan toeteren en weet hoeveel passagiers erin passen. Het is hetzelfde idee als bij de interface in [hoofdstuk 19](#), met één verschil: de basisklasse geeft ook gegevens en uitgewerkte methodes mee.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>: Basisklasse</code>	erf van een klasse	<code>class Fiets : Voertuig</code>	een fiets heeft alles van een voertuig
<code>: Basisklasse, IContract</code>	erf van één klasse en beloof daarnaast contracten	<code>class Fiets : Voertuig, IGeluid</code>	eerst de klasse, dan de interfaces
<code>base(...)</code>	roep de constructor van de basisklasse aan	<code>public Fiets() : base(2) { }</code>	<code>Wielen</code> is <code>2</code>
<code>protected</code>	zichtbaar in de klasse en in wie ervan erft, niet daarbuiten	<code>protected int Kilometers</code>	<code>fiets.Kilometers</code> compileert niet
<code>virtual</code>	deze methode mag vervangen worden	<code>public virtual string Toeter()</code>	de bus gebruikt de standaardversie
<code>override</code>	vervang een methode die <code>virtual</code> of <code>abstract</code> is	<code>public override string Toeter()</code>	<code>"Tring"</code>
<code>base.Methode()</code>	gebruik de versie van de basisklasse, binnen je eigen versie	<code>return base.Toeter() + "!"</code> ;	<code>"Tuut!"</code>
<code>abstract class</code>	een klasse waarvan je geen object kan maken	<code>new Voertuig(4)</code>	compileert niet
<code>abstract</code> (methode)	een methode zonder uitwerking; elke afgeleide klasse moet ze uitwerken	<code>public abstract int MaxPassagiers();</code>	vergeet je ze, dan compileert het niet
<code>sealed</code>	van deze klasse mag niemand meer erven	<code>public sealed class Bus : Voertuig</code>	<code>class Dubbeldekker : Bus</code> compileert niet
<code>is</code>	is dit object van dat type, of ervan afgeleid	<code>new Fiets() is Voertuig</code>	<code>true</code>
<code>GetType</code>	het echte type van het object, niet dat van de variabele	<code>Voertuig v = new Fiets();</code> en dan <code>v.GetType().Name</code>	<code>"Fiets"</code>

Abstracte klasse tegenover interface

	Abstracte klasse	Interface
Zegt	wat iets is	wat iets kan
Bewaart gegevens	ja	nee — ze kan enkel zeggen dát er een eigenschap moet zijn
Uitgewerkte methodes	ja, naast de abstracte	nee, op een zeldzame uitzondering na die je in deze cursus niet tegenkomt
Constructor	ja	nee
Hoeveel per klasse	één	zoveel je wil
Een object van maken	nee	nee
Typisch gebruik	verwante klassen die gegevens en werk delen: een fiets en een bus	klassen die hetzelfde moeten kunnen, ook als ze niets met elkaar te maken hebben: een hond en een deurbel maken allebei geluid

Waar je dit in de cursus tegenkomt

Les	Wat je ziet	Wat er gebeurt
01	een controller die erft van <code>ControllerBase</code>	je controller erft <code>Ok</code> , <code>NotFound</code> en de rest uit hoofdstuk 26
04	<code>: base(options)</code> in je context	de opties gaan door naar de constructor van <code>DbContext</code> ; zie hoofdstuk 31
04	<code>protected override void OnModelCreating(...)</code>	je vervangt een methode die <code>DbContext</code> daarvoor openzet
07	een specifieke repository die erft van de generieke	ze krijgt alle gewone methodes mee; <code>protected</code> maakt de context ook voor haar zichtbaar

Wanneer wel, wanneer niet

Wel — verwante klassen die echt hetzelfde zijn en gegevens of uitgewerkte methodes delen. De test: klinkt *een X is een Y* juist? Een fiets is een voertuig: ja. Maak de basisklasse `abstract` zodra ze op zichzelf niets betekent.

Niet — enkel om een paar methodes te hergebruiken tussen klassen die niets met elkaar te maken hebben. Een leerling is geen klaslokaal, ook al hebben ze allebei een naam. En gaat het enkel om wat iets kan, dan is een interface lichter.

Veelgemaakte fouten

LET OP

- Wat `private` is in de basisklasse, kan de afgeleide klasse niet zien. Heeft ze het nodig, maak het dan `protected` — niet `public`, want dan kan iedereen erbij.
- Heeft de basisklasse enkel een constructor met parameters, dan moet elke afgeleide klasse : `base(...)` aanroepen. Vergeet je dat, dan compileert het niet.
- `override` kan enkel op een methode die `virtual` of `abstract` is. Schrijf je in de afgeleide klasse gewoon een methode met dezelfde naam, dan vervang je niets: je verstoort de oude alleen. Visual Studio waarschuwt je, maar het compileert. Zie vraag 4.
- Een klasse erft van hoogstens één klasse. Interfaces mag je er zoveel bij zetten als je wil.
- Via een variabele van de basisklasse zie je enkel wat de basisklasse heeft, ook als het object meer kan. Met `is` uit [hoofdstuk 22](#) ga je na wat het echt is.

Voorspel de uitkomst

Vragen 1, 2, 3 en 6 gebruiken de tweede versie van `Voertuig`, met `abstract`, en de klassen `Fiets` en `Bus` die erbij horen.

Vraag 1

```
Voertuig v = new Fiets();  
Console.WriteLine(v.Toeter());
```

Toon het antwoord

Tring

De variabele zegt enkel "een voertuig", maar het object is een fiets. Omdat `Toeter` `virtual` is en de fiets ze met `override` vervangt, telt de versie van het object, niet die van de variabele.

Vraag 2

```
Voertuig v = new Bus();  
Console.WriteLine(v.Toeter());
```

Toon het antwoord

Tuut

De bus vervangt `Toeter` niet. Ze gebruikt dus de standaardversie uit `Voertuig`. Een `virtual` methode mag je vervangen, je moet niet.

Vraag 3

```
Voertuig v = new Voertuig(4);
```

Toon het antwoord

Compileert niet.

Voertuig is abstract. Van de basisklasse zelf kan je geen object maken, enkel van een klasse die ervan erft: een fiets of een bus.

Vraag 4

```
public class Basis
{
    public string Wie()
    {
        return "basis";
    }
}

public class Afgeleid : Basis
{
    public string Wie()
    {
        return "afgeleid";
    }
}

Basis b = new Afgeleid();
Afgeleid a = new Afgeleid();
Console.WriteLine(b.Wie());
Console.WriteLine(a.Wie());
```

Toon het antwoord

basis en daarna afgeleid

Twee keer hetzelfde object, twee verschillende antwoorden. Zonder `virtual` en `override` vervangt `Afgeleid` de methode niet, maar verstopt ze haar enkel. Welke versie je krijgt, hangt dan af van de variabele, niet van het object. Zet `virtual` bij `Basis` en `override` bij `Afgeleid`, en je krijgt twee keer `afgeleid`.

Vraag 5

```
public class A
{
    public A()
    {
        Console.WriteLine("A");
    }
}

public class B : A
{
    public B()
    {
        Console.WriteLine("B");
    }
}

new B();
```

Toon het antwoord

A en daarna B

Eerst de fundering, dan de muren. Vóór de constructor van B draait die van de basisklasse. Staat er geen : base(. . .), dan roept C# zelf de constructor zonder parameters van A aan.

Vraag 6

```
Voertuig v = new Fiets();
Console.WriteLine(v is Voertuig);
Console.WriteLine(v.GetType() == typeof(Voertuig));
Console.WriteLine(v.GetType().Name);
```

Toon het antwoord

True, False en daarna Fiets

is vraagt: *is dit een voertuig, of iets dat ervan erft?* Een fiets is dat. GetType() geeft het echte type van het object, en dat is precies Fiets, niet Voertuig. Wil je weten of iets tot een familie behoort, gebruik dan is.

21. Generics

EIGEN TYPE

OPMERKING

Zelf generics schrijven komt aan bod in les 07 van Programming Advanced. Het staat hier al, zodat je het klaar hebt wanneer je het nodig hebt.

Wat het is

Een type met een open plaats. Zoals een doos met een etiket *inhoud*: _____. De doos werkt hetzelfde voor schoenen, boeken of speelgoed; pas wanneer je hem gebruikt, vul je in wat erin gaat.

Je gebruikt het al sinds [hoofdstuk 6](#). `List<T>` is één klasse die werkt voor `List<string>`, `List<int>` en `List<Leerling>`. De `T` is de open plaats. In dit hoofdstuk maak je zelf zo'n type.

Hoe het eruitziet

```
public class Doos<T>
{
    private T _inhoud;

    public Doos(T inhoud)
    {
        _inhoud = inhoud;
    }

    public T Open()
    {
        return _inhoud;
    }
}
```

Overall waar `T` staat, komt straks het type dat je invult:

```
Doos<string> brief = new Doos<string>("Proficiat!");
Doos<int> getal = new Doos<int>(42);

string tekst = brief.Open();           // Proficiat!
int n = getal.Open();                  // 42 – een echt getal, je kan ermee rekenen

// new Doos<int>("42") compileert niet: dat is een tekst, geen getal
```

Ook een losse methode kan een open plaats hebben:

```
T Eerste<T>(List<T> lijst)
{
    return lijst[0];
}

string s = Eerste(new List<string> { "kiwi", "peer" }); // kiwi
int g = Eerste(new List<int> { 7, 3 });                 // 7
```

C# leidt zelf af wat `T` is uit wat je meegeeft. Je hoeft `Eerste<string>(...)` niet uit te schrijven, al mag het.

De open plaats beperken: where

Zonder meer weet C# niets over `T`. Het kan een getal zijn, een tekst of een leerling, dus je mag er niets mee doen wat niet voor alle types werkt. Met `where` beperk je wat er ingevuld mag worden — en daar krijg je iets voor terug:

```
public class Garage<T> where T : Voertuig
{
    private List<T> _voertuigen = new List<T>();

    public void Zet(T voertuig)
    {
        _voertuigen.Add(voertuig);
    }

    public int TotaalWielen()
    {
        return _voertuigen.Sum(v => v.Wielen); // mag: elke T is een voertuig
    }
}

Garage<Fiets> stalling = new Garage<Fiets>(); // mag: een fiets is een voertuig
```

`Voertuig` en `Fiets` komen uit hoofdstuk 20. Zonder `where T : Voertuig` zou `v.Wielen` niet compileren, want dan kan `T` ook een tekst zijn.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>class Naam<T></code>	een klasse met een open plaats	<code>class Doos<T></code>	<code>Doos<int></code> , <code>Doos<string></code> , ...
<code>Methode<T>(...)</code>	een methode met een open plaats	<code>T Eerste<T>(List<T> lijst)</code>	<code>Eerste(namen)</code> geeft een tekst
<code>interface INaam<T></code>	een contract met een open plaats	<code>interface IHouder<T></code>	elk type belooft het voor zijn eigen <code>T</code>
<code>class X<T> : IY<T></code>	geef de open plaats door aan het contract	<code>class Doos<T> : IHouder<T></code>	de doos belooft het contract, voor elke <code>T</code>
<code>class X : IY<int></code>	vul de open plaats meteen in	<code>class Spaarpot : IHouder<int></code>	enkel voor getallen
<code><T1, T2></code>	meerdere open plaatsen	<code>class Paar<TLinks, TRechts></code>	zoals <code>Dictionary<TKey, TValue></code>
<code>where T : class</code>	<code>T</code> moet een referentietype zijn	<code>class Doos<T> where T : class</code>	<code>Doos<int></code> compileert niet
<code>where T : struct</code>	<code>T</code> moet een waardetype zijn	<code>class Doos<T> where T : struct</code>	<code>Doos<string></code> compileert niet
<code>where T : Basisklasse</code>	<code>T</code> moet die klasse zijn, of ervan erven	<code>where T : Voertuig</code>	binnenin mag je <code>Wielen</code> gebruiken
<code>where T : IContract</code>	<code>T</code> moet dat contract beloven	<code>where T : IGeluid</code>	binnenin mag je <code>MaakGeluid()</code> aanroepen
<code>where T : new()</code>	<code>T</code> moet een constructor zonder parameters hebben	<code>where T : new()</code>	binnenin mag je <code>new T()</code> schrijven
<code>default(T)</code>	de standaardwaarde van <code>T</code>	<code>default(int)</code>	<code>0</code> ; bij een klasse <code>null</code> , bij een bool <code>false</code>
<code>typeof(T)</code>	het type dat ingevuld werd	<code>typeof(T).Name</code> in een <code>Doos<int></code>	<code>"Int32"</code>

Meerdere beperkingen zet je achter elkaar, gescheiden door een komma: `where T : class, new()`.

OPMERKING

`string` is een klasse, en dus een referentietype. Bij `where T : class` mag je `string` dus wél invullen. Enkel waardetypes zoals `int`, `double` en `bool` vallen erbuiten.

Waarom dit in je cursus zo belangrijk is

In les 07 van Programming Advanced schrijf je één repository die werkt voor al je entiteiten, in plaats van één per entiteit. De open plaats is de entiteit; de cursus noemt ze `TEntity` in plaats van `T`. De beperking `where TEntity : class` staat erbij omdat EF Core enkel met klassen werkt. Specifieke repositories erven daarna van die generieke, zoals in [hoofdstuk 20](#). Hoe zo'n repository aan de juiste tabel komt, staat in [hoofdstuk 32](#).

Wanneer wel, wanneer niet

Wel — dezelfde code werkt voor meerdere types, en enkel het type verschilt. Zonder generics zou je dezelfde klasse per type kopiëren.

Niet — wanneer de code per type iets anders moet doen. Schrijf je in een generieke methode `if (typeof(T) == typeof(int))`, dan heb je geen open plaats nodig maar aparte methodes, of overerving.

Veelgemaakte fouten

LET OP

- Zonder `where` kan je binnen de klasse niets specifiek met `T` doen. `voertuig.Wielen` compileert pas met `where T : Voertuig`, ook al stop je er in de praktijk enkel voertuigen in.
- `return null;` compileert niet als `T` ook een getal kan zijn. Gebruik `default`, of beperk `T` met `where T : class`.
- `Doos<Fiets>` is geen `Doos<Voertuig>`, ook al is een fiets een voertuig. Hetzelfde geldt voor `List<Fiets>` en `List<Voertuig>`. Zie vraag 4.
- Volgens de afspraak begint de naam van een open plaats met een `T`: `T`, `TSleutel`, `TEntity`.

Voorspel de uitkomst

De vragen gebruiken `Doos<T>` en `Garage<T>` van hierboven, en `Voertuig` en `Fiets` uit hoofdstuk 20.

Vraag 1

```
var a = new Doos<int>(5);
var b = new Doos<string>("5");
Console.WriteLine(a.Open() + 1);
Console.WriteLine(b.Open() + 1);
```

Toon het antwoord

6 en daarna 51

Dezelfde klasse, twee verschillende types. De eerste doos geeft een echt getal terug, en $5 + 1$ is 6. De tweede geeft een tekst terug, en een tekst + 1 plakt de 1 erachter.

Vraag 2

```
Console.WriteLine(default(int));
Console.WriteLine(default(bool));
Console.WriteLine(default(string) == null);
```

Toon het antwoord

0, False en daarna True

Elk type heeft een standaardwaarde: nul voor getallen, false voor een bool en null voor referentietypes. Het is dezelfde waarde die `FirstOrDefault` uit hoofdstuk 11 teruggeeft wanneer het niets vindt.

Vraag 3

```
var g = new Garage<string>();
```

Toon het antwoord

Compileert niet.

`Garage<T>` eist met `where T : Voertuig` dat T een voertuig is. Een tekst is dat niet. De fout valt bij het bouwen, niet pas wanneer het programma draait.

Vraag 4

```
Doos<Voertuig> d = new Doos<Fiets>(new Fiets());
```

Toon het antwoord

Compileert niet.

Voor C# zijn `Doos<Fiets>` en `Doos<Voertuig>` twee aparte types, zonder verband. Dat een fiets een voertuig is, maakt een doos met een fiets nog geen doos met een voertuig. Wat wel mag: een `Doos<Voertuig>` maken en daar een fiets in stoppen, met `new Doos<Voertuig>(new Fiets())`.

Vraag 5

```
string Soort<T>(T waarde)
{
    return typeof(T).Name;
}

Console.WriteLine(Soort(42));
Console.WriteLine(Soort("hallo"));
```

Toon het antwoord

`Int32` en daarna `String`

C# leidt T af uit wat je meegeeft: eerst een `int`, daarna een `string`. Dat de namen er anders uitzien, komt doordat `int` en `string` korte namen zijn voor `Int32` en `String`. Het zijn dezelfde types.

22. Types herkennen

METHODES

De voorbeelden gebruiken `IGeluid`, `Hond` en `Kat` uit hoofdstuk 19.

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>typeof</code>	het type zelf, als je de naam kent	<code>typeof(Kat)</code>	het type
<code>GetType</code>	het type van een bestaand object	<code>dier.GetType()</code>	het type
<code>is</code>	is dit object van dat type	<code>if (dier is Kat)</code>	<code>true</code> of <code>false</code>
<code>is</code> met naam	idem, en geef het meteen die naam	<code>if (dier is Kat k)</code>	<code>k</code> is bruikbaar
<code>as</code>	behandel als dat type, of <code>null</code>	<code>dier as Kat</code>	het object of <code>null</code>
<code>nameof</code>	de naam van iets als tekst	<code>nameof(GetFanById)</code>	<code>"GetFanById"</code>

`nameof` lijkt onnozel tot je het nodig hebt. Het voordeel tegenover `"GetFanById"` tussen aanhalingstekens is dat je code niet meer compileert wanneer je die methode hernoemt. Een typfout in een naam vind je zo bij het bouwen in plaats van bij het testen.

Voorspel de uitkomst

Vraag 1

```
IGeluid dier = new Hond();
Console.WriteLine(dier is Kat);
```

Toon het antwoord

`False`

De variabele zegt enkel "iets dat een geluid maakt". Wat erin zit, is een hond.

Vraag 2

```
IGeluid dier = new Hond();
Kat? k = dier as Kat;
Console.WriteLine(k is null);
```

Toon het antwoord

`True`

`as` probeert het dier als kat te behandelen. Lukt dat niet, dan krijg je `null` in plaats van een crash. Daarom controleer je na een `as` altijd op `null`.

Vraag 3

```
Console.WriteLine(nameof(Kat));  
Console.WriteLine(typeof(Kat).Name);
```

Toon het antwoord

`Kat` en daarna `Kat`

Twee wegen naar dezelfde tekst. `nameof` wordt al bij het compileren ingevuld, `typeof(...).Name` pas als het programma draait. Voor een naam in een melding of in `CreatedAtAction` is `nameof` de gewoonte.

23. Uitzonderingen

CONCEPT

Wat het is

Een **noodrem**. Wanneer een methode iets gevraagd wordt dat ze niet kan — een tekst omzetten die geen getal is, een sleutel opzoeken die er niet is — dan stopt ze meteen en gooit ze een **uitzondering**. Die gaat terug naar wie de methode aanriep, en naar wie die aanriep, tot iemand ze opvangt. Vangt niemand ze op, dan stopt het hele programma.

Je bent ze in dit leerboek al vaak tegengekomen, in het rood. Dit hoofdstuk gaat over wat je ermee doet.

Hoe het eruitziet

```
try
{
    int leeftijd = int.Parse("twaalf");
    Console.WriteLine($"Leeftijd: {leeftijd}");
}
catch (FormatException)
{
    Console.WriteLine("Dat is geen getal.");
}
finally
{
    Console.WriteLine("Klaar.");
}
```

`int.Parse` gooit een `FormatException`. De rest van het `try`-blok wordt overgeslagen, het `catch`-blok vangt de uitzondering op, en `finally` draait altijd — fout of geen fout. Op het scherm komt `Dat is geen getal.` en daarna `Klaar.`

Je kan ook zelf een uitzondering gooien, wanneer je methode iets gevraagd wordt dat ze niet kan of mag doen:

```
void ZetThermostaat(int graden)
{
    if (graden < 5 || graden > 30)
    {
        throw new ArgumentException("Kies tussen 5 en 30 graden.");
    }

    Console.WriteLine($"Thermostaat op {graden} graden.");
}

try
{
    ZetThermostaat(40);
}
catch (ArgumentException ex)
{
    Console.WriteLine(ex.Message);    // Kies tussen 5 en 30 graden.
}
```

`throw` werkt zoals `return`: de methode stopt daar. Het verschil is dat er geen waarde teruggaat maar een uitzondering, en dat wie de methode aanriep ze moet opvangen — of zelf ook stopt.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>try</code>	probeer dit blok	<code>try { ... }</code>	loopt tot de eerste uitzondering
<code>catch (Soort)</code>	vang deze soort op	<code>catch (FormatException)</code>	het programma loopt verder na het blok
<code>catch (Soort ex)</code>	vang op en geef ze een naam	<code>catch (FormatException ex)</code>	<code>ex</code> is bruikbaar in het blok
<code>catch (Exception)</code>	vang elke soort op	<code>catch (Exception ex)</code>	vangt alles, ook wat je niet verwachtte
<code>finally</code>	draait altijd, fout of geen fout	<code>finally { ... }</code>	ook na een <code>return</code> in het <code>try</code> -blok
<code>throw new</code>	gooi zelf een uitzondering	<code>throw new ArgumentException("Te warm.")</code>	de methode stopt hier
<code>throw;</code>	gooi dezelfde uitzondering verder, binnen een <code>catch</code>	<code>catch (Exception) { ...; throw; }</code>	wie hoger staat, krijgt ze alsnog
<code>Message</code>	de uitleg bij de uitzondering	<code>ex.Message</code>	"Te warm."
<code>GetType</code>	welke soort het is	<code>ex.GetType().Name</code>	"FormatException"

De uitzonderingen die je al kent

Uitzondering	Wanneer	Hoofdstuk
<code>FormatException</code>	<code>int.Parse</code> op een tekst die geen getal is	2
<code>IndexOutOfRangeException</code>	een positie buiten een array	5
<code>ArgumentOutOfRangeException</code>	een positie buiten een <code>List</code> , of <code>Substring</code> buiten de tekst	1, 6
<code>ArgumentException</code>	<code>Add</code> met een sleutel die al in een <code>Dictionary</code> zit	7
<code>KeyNotFoundException</code>	<code>[sleutel]</code> die niet bestaat	7
<code>InvalidOperationException</code>	een lijst veranderen in een <code>foreach</code> , <code>Dequeue</code> of <code>Pop</code> op iets leegs, <code>First</code> zonder resultaat, <code>Single</code> met twee	6, 9, 10, 11
<code>NullReferenceException</code>	iets opvragen van <code>null</code>	17
<code>FileNotFoundException</code>	<code>File.ReadAllText</code> op een bestand dat niet bestaat	30
<code>JsonException</code>	<code>JsonSerializer.Deserialize</code> op tekst die geen geldige JSON is	30
<code>NotImplementedException</code>	een methode die nog niet uitgewerkt is	les 05 van de cursus

Ontbreekt niet alleen het bestand maar ook de map waarin het moet staan, dan krijg je een `DirectoryNotFoundException` in plaats van een `FileNotFoundException`.

Voorkomen tegenover opvangen

Veel uitzonderingen kan je voorkomen door eerst te kijken. Dat is bijna altijd beter: het zegt duidelijker wat je bedoelt, en een `catch` die te veel opvangt, verbergt fouten die je had moeten zien.

In plaats van opvangen	Kijk vooraf met	Hoofdstuk
<code>FormatException</code> van <code>Parse</code>	<code>TryParse</code>	2
<code>KeyNotFoundException</code>	<code>TryGetValue</code> of <code>ContainsKey</code>	7
<code>InvalidOperationException</code> van <code>First</code>	<code>FirstOrDefault</code> en een controle op <code>null</code>	11
<code>NullReferenceException</code>	<code>?.</code> of <code>is null</code>	17

Opvangen blijft nodig bij alles wat van buiten je programma komt: een bestand, een netwerk, een database. Of een bestand er nog staat op het moment dat je het leest, en of de inhoud geldige JSON is, kan je vooraf niet zeker weten.

De volgorde van je catch-blokken

Alle uitzonderingen erven van `Exception` — overerving uit hoofdstuk 20. Daarom vangt `catch (Exception)` elke soort op. Zet je meerdere `catch`-blokken onder elkaar, dan loopt C# ze van boven naar onder af en neemt het het eerste dat past. De specifieke soorten komen dus eerst, de algemene als laatste:

```
try
{
    int aantal = int.Parse(invoer);
    Console.WriteLine(100 / aantal);
}
catch (FormatException)
{
    Console.WriteLine("Geef een getal.");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Nul mag niet.");
}
catch (Exception ex)
{
    Console.WriteLine($"Onverwacht: {ex.Message}");
}
```

In je API

Een uitzondering die in een controller niemand opvangt, legt niet de hele server plat. ASP.NET Core vangt ze op het laatste moment op en stuurt **500 Internal Server Error** terug — dat zag je in vraag 3 van hoofdstuk 26. Voor de client betekent 500: *de server heeft een fout gemaakt*. Lag de fout eigenlijk bij de client — een Id dat niet bestaat, een ongeldige waarde — dan hoort daar een 404 of een 400 bij.

Wanneer wel, wanneer niet

Wel — `try` en `catch` bij alles wat van buiten je programma komt en wat je vooraf niet volledig kan nakijken: bestanden, netwerk, een database, JSON van elders. En `throw` wanneer je methode iets gevraagd wordt dat niet kan, en ze zelf niet weet wat ze dan moet doen.

Niet — als vervanger van een controle die je vooraf kan doen. En nooit om een fout stil te laten verdwijnen.

Veelgemaakte fouten

LET OP

- Een lege `catch { }` slikt de fout in. Het programma loopt verder alsof er niets gebeurd is, met verkeerde of ontbrekende gegevens, en niemand weet waarom.
- `catch (Exception)` vangt ook de fouten die je niet verwachtte, zoals een `NullReferenceException` uit je eigen code. Vang op wat je verwacht en waar je iets zinnigs mee kan doen.
- Een variabele die je in het `try`-blok maakt, bestaat niet meer na het blok. Heb je hem nadien nodig, maak hem dan vóór de `try`.
- Code die in hetzelfde blok na een `throw` staat, wordt nooit uitgevoerd.

Voorspel de uitkomst

Vraag 1

```
try
{
    Console.WriteLine("A");
    int.Parse("x");
    Console.WriteLine("B");
}
catch (FormatException)
{
    Console.WriteLine("C");
}
finally
{
    Console.WriteLine("D");
}
Console.WriteLine("E");
```

Toon het antwoord

A, C, D en daarna E

A komt nog op het scherm. Daarna gooit `int.Parse` een uitzondering, en de rest van het `try`-blok — de B — wordt overgeslagen. Het `catch`-blok toont C, `finally` toont D, en omdat de uitzondering opgevangen is, loopt het programma gewoon verder met E.

Vraag 2

```
try
{
    var d = new Dictionary<string, int>();
    Console.WriteLine(d["x"]);
}
catch (FormatException)
{
    Console.WriteLine("opgevangen");
}
```

Toon het antwoord

KeyNotFoundException — het programma crasht.

De sleutel "x" bestaat niet, en dat geeft een **KeyNotFoundException**. Het **catch**-blok vangt enkel een **FormatException** op. Een andere soort gaat er gewoon langs.

Vraag 3

```
try
{
    int.Parse("x");
}
catch (Exception)
{
    Console.WriteLine("algemeen");
}
catch (FormatException)
{
    Console.WriteLine("formaat");
}
```

Toon het antwoord

Compileert niet.

FormatException erft van **Exception**. Het eerste blok vangt dus al alles op, en het tweede kan nooit aan de beurt komen. C# weigert dat. Draai de volgorde om: de specifieke soort eerst, de algemene als laatste.

Vraag 4

```
try
{
    throw new InvalidOperationException("Rij is leeg");
}
catch (Exception ex)
{
    Console.WriteLine(ex.GetType().Name);
    Console.WriteLine(ex.Message);
}
```

Toon het antwoord

`InvalidOperationException` en daarna `Rij is leeg`

`catch (Exception ex)` vangt elke soort op, maar `ex` blijft wat het echt is. Net als bij de fiets in hoofdstuk 20 geeft `GetType()` het echte type, niet dat van de variabele.

Vraag 5

```
string Test()
{
    try
    {
        return "uit try";
    }
    finally
    {
        Console.WriteLine("finally");
    }
}

Console.WriteLine(Test());
```

Toon het antwoord

`finally` en daarna `uit try`

De `return` legt het antwoord klaar, maar vóór de methode echt stopt, draait `finally` nog. Pas daarna komt het antwoord bij `Console.WriteLine` aan. Dat is wat *altijd* betekent.

Vraag 6

```
void Binnen()  
{  
    throw new ArgumentException("mis");  
}  
  
void Midden()  
{  
    Binnen();  
    Console.WriteLine("Midden klaar");  
}  
  
try  
{  
    Midden();  
}  
catch (ArgumentException ex)  
{  
    Console.WriteLine($"Opgevangen: {ex.Message}");  
}
```

Toon het antwoord

Opgevangen: mis

Binnen gooit, en Midden vangt niets op. De uitzondering gaat dus meteen verder naar boven, en Midden klaar komt nooit op het scherm. Pas het catch-blok helemaal bovenaan vangt ze op.

24. static, const en readonly

CONCEPT

Wat het is

Wat bij de klasse hoort, en wat bij één object. Denk aan een school. Elk klaslokaal heeft zijn eigen bord: wat erop staat, verschilt per lokaal. Maar er is maar één schoolbel, en als die gaat, hoort iedereen hem.

Een gewoon veld is het bord: elk object heeft er zijn eigen exemplaar van. Een `static` veld is de bel: er is er één, en die hoort bij de klasse zelf.

Daarnaast zijn er twee manieren om te zeggen dat iets niet meer mag veranderen: `const` en `readonly`.

Hoe het eruitziet

```
public class Teller
{
    public static int Totaal = 0;    // één voor alle tellers samen
    public int Eigen = 0;          // één per teller

    public void Tik()
    {
        Totaal++;
        Eigen++;
    }
}

var a = new Teller();
var b = new Teller();

a.Tik();
a.Tik();
b.Tik();

Console.WriteLine(Teller.Totaal); // 3 – via de klasse, niet via a of b
Console.WriteLine(a.Eigen);       // 2
Console.WriteLine(b.Eigen);       // 1
```

Je kent `static` al zonder het te weten. `Math.Max`, `int.Parse` en `Console.WriteLine` roep je op via de naam van de klasse, zonder eerst een object te maken. Dat zijn `static` methodes.

En zo zien `const` en `readonly` eruit:

```
public class Bestelling
{
    public const int MaxAantal = 10;    // ligt vast bij het compileren

    private readonly DateTime _aangemaakt; // ligt vast na de constructor

    public Bestelling()
    {
        _aangemaakt = DateTime.Now;    // hier mag het nog
    }

    public void Wijzig()
    {
        // _aangemaakt = DateTime.Now;    compileert niet
    }
}
```

Waarom dit in je cursus zo belangrijk is

Les 02. De lijst in de controller is daar `private static List<Laptop>`. ASP.NET Core maakt voor elk verzoek een nieuw controller-object. Zonder `static` hoort de lijst bij dat ene object en begint elk verzoek met een verse lijst: wat je met een POST toevoegde, is bij de volgende GET verdwenen. Met `static` hoort de lijst bij de klasse, en blijft ze bestaan zolang de applicatie draait.

Vanaf les 03. Een repository of logger die je via de constructor krijgt, bewaar je in een `private readonly` veld. Zo kan niemand dat veld later in de klasse nog vervangen. Hoe dat precies werkt, staat in [hoofdstuk 27](#).

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>static</code> veld	één waarde voor de hele klasse	<code>public static int Totaal;</code>	alle objecten delen dezelfde
<code>static</code> methode	oproepen zonder object	<code>Teller.Reset()</code>	zoals <code>Math.Max</code>
<code>static class</code>	een klasse met enkel <code>static</code> leden	<code>public static class Omrekenen</code>	<code>new Omrekenen()</code> compileert niet
Klasse.Lid	een <code>static</code> lid gebruik je via de klassenaam	<code>Teller.Totaal</code>	3
<code>const</code>	een vaste waarde, al bekend bij het compileren	<code>const int MaxAantal = 10;</code>	kan nooit veranderen
<code>readonly</code>	invullen bij de declaratie of in de constructor, daarna nooit meer	<code>private readonly ILogger _logger;</code>	toewijzen in een andere methode compileert niet
<code>static readonly</code>	één vaste waarde voor de hele klasse, die geen <code>const</code> kan zijn	<code>static readonly DateTime Start = DateTime.Now;</code>	wordt één keer ingevuld

Wanneer wel, wanneer niet

Wel — `static` voor wat echt bij de klasse hoort: een teller over alle objecten, een hulpmethode die geen gegevens van een object nodig heeft. `const` voor een waarde die nooit verandert, zoals een maximum. `readonly` voor een veld dat je één keer invult en daarna met rust laat, zoals alles wat je via de constructor krijgt.

Niet — `static` voor gegevens die bij één object horen. De `static` lijst uit les 02 is een tijdelijke oplossing: zodra er een database is, verdwijnt ze.

Veelgemaakte fouten

LET OP

- Een `static` methode kan niet aan de gewone velden van de klasse. Ze hoort bij de klasse, niet bij één object, dus ze weet niet wélk object je bedoelt.
- `readonly` beschermt het veld, niet wat erin zit. Een `readonly` lijst kan je niet vervangen door een andere lijst, maar je kan er wel nog items aan toevoegen.
- `const` werkt enkel met waarden die al vastliggen bij het compileren: getallen, tekst, `true` en `false`. `const List<int>` of `const DateTime` compileert niet. Gebruik dan `static readonly`.
- Een `static` lid roep je op via de klassenaam. `a.Totaal` compileert niet, `Teller.Totaal` wel.

Voorspel de uitkomst

Vraag 1

Deze vraag gebruikt de klasse `Teller` van hierboven.

```
var a = new Teller();
var b = new Teller();
var c = new Teller();

a.Tik();
b.Tik();
b.Tik();

Console.WriteLine(Teller.Totaal);
Console.WriteLine(c.Eigen);
```

Toon het antwoord

3 en daarna 0

Er werd drie keer getikt, en het totaal is van iedereen samen. `c` heeft zelf nooit getikt, dus zijn eigen teller staat nog op nul.

Vraag 2

```
public class Mand
{
    private List<string> _items = new List<string>();
    private static List<string> _gedeeld = new List<string>();

    public void Voeg(string item)
    {
        _items.Add(item);
        _gedeeld.Add(item);
    }

    public string Tel()
    {
        return _items.Count + " en " + _gedeeld.Count;
    }
}

new Mand().Voeg("appel");
new Mand().Voeg("peer");
Console.WriteLine(new Mand().Tel());
```

Toon het antwoord

0 en 2

Elke `new Mand()` is een nieuwe mand met een lege eigen lijst. De gedeelde lijst hoort bij de klasse en onthoudt alles. Dit is precies wat er in een controller gebeurt: elk verzoek krijgt een nieuw object, en enkel wat static is, blijft.

Vraag 3

```
public class Klas
{
    private readonly List<string> _namen = new List<string>();

    public void Voeg(string naam)
    {
        _namen.Add(naam);
    }

    public int Aantal()
    {
        return _namen.Count;
    }
}

var k = new Klas();
k.Voeg("Anna");
k.Voeg("Bram");
Console.WriteLine(k.Aantal());
```

Toon het antwoord

2

`readonly` zegt enkel dat `_namen` altijd naar dezelfde lijst wijst. Wat er in die lijst zit, mag gewoon veranderen. `_namen = new List<string>();` in `Voeg` had niet gecompileerd.

Vraag 4

```
public class Rekenhulp
{
    private int _laatste = 0;

    public static int Dubbel(int x)
    {
        _laatste = x * 2;
        return _laatste;
    }
}
```

Toon het antwoord

Compileert niet.

`Dubbel` is static en hoort bij de klasse. `_laatste` is een gewoon veld en hoort bij één object. Als je `Rekenhulp.Dubbel(5)` oproept zonder object, is er geen `_laatste` om in te schrijven.

Deel 5 — In je API

Wat je nodig hebt zodra je van een consoletoepassing naar een API overstapt.

25. Attributen en routes

IN JE API

Wat het is

Een etiket op je code. Een attribuut staat tussen vierkante haken boven een klasse, een methode of een parameter. Het verandert niets aan wat de code doet, maar het vertelt het framework iets erover. `[HttpGet]` boven een methode zegt: *deze methode antwoordt op een GET-verzoek*.

Voor een API is het belangrijkste wat attributen regelen de **route**: welke URL bij welke methode uitkomt.

Hoe het eruitziet

```
[ApiController]
[Route("api/[controller]")]
public class ReceptController : ControllerBase
{
    [HttpGet]
    public ActionResult<List<Recept>> GetAll() { /* ... */ }

    [HttpGet("{id}")]
    public ActionResult<Recept> GetById(int id) { /* ... */ }

    [HttpGet("keuken/{keuken}")]
    public ActionResult<List<Recept>> GetPerKeuken(string keuken) { /* ... */ }

    [HttpPost]
    public ActionResult<Recept> VoegToe(Recept recept) { /* ... */ }

    [HttpPut("{id}")]
    public IActionResult Wijzig(int id, Recept recept) { /* ... */ }

    [HttpDelete("{id}")]
    public IActionResult Verwijder(int id) { /* ... */ }
}
```

De route van de klasse en die van de methode worden aan elkaar geplakt:

Verzoek	Komt uit bij	Met
GET api/recept	GetAll	—
GET api/recept/5	GetById	id is 5
GET api/recept/keuken/italiaans	GetPerKeuken	keuken is "italiaans"
POST api/recept	VoegToe	recept komt uit de JSON in de body
PUT api/recept/5	Wijzig	id uit de route, recept uit de body
DELETE api/recept/5	Verwijder	id is 5

`[controller]` wordt vervangen door de naam van de klasse zonder het woord *Controller*: van `ReceptController` blijft `Recept` over. Hoofdletters maken in een route niet uit: `api/Recept` en `api/recept` komen bij dezelfde methode uit. De waarde zelf behoudt wel haar hoofdletters: bij `keuken/Italiaans` is `keuken` gelijk aan `"Italiaans"`.

Waar een parameter vandaan komt

Met `[ApiController]` op de klasse beslist ASP.NET Core zelf waar elke parameter vandaan komt:

1. Staat de naam tussen accolades in de route, dan komt hij uit de route.
2. Is het een object, zoals een `Recept`, dan komt het uit de body.
3. Al de rest komt uit de querystring: het stuk na het vraagteken, zoals `?zoek=pasta`.

Je kan het ook zelf zeggen met `[FromRoute]`, `[FromBody]` of `[FromQuery]` vóór de parameter. Met `[ApiController]` heb je dat zelden nodig.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>[ApiController]</code>	zet het API-gedrag aan: parameters herkennen, automatisch 400 bij ongeldige invoer	boven de klasse	—
<code>[Route("api/[controller]")]</code>	de basisroute van de klasse	op <code>ReceptController</code>	<code>api/recept</code>
<code>[Route("api/keuken")]</code>	een vaste route, los van de klassenaam	op <code>ReceptController</code>	<code>api/keuken</code>
<code>[HttpGet]</code>	antwoordt op GET, op de basisroute	<code>GetAlle()</code>	GET <code>api/recept</code>
<code>[HttpGet("{id}")]</code>	met een routeparameter; de naam tussen accolades moet die van de parameter zijn	<code>GetById(int id)</code>	GET <code>api/recept/5</code>
<code>[HttpGet("{id:int}")]</code>	routeparameter met een beperking: enkel een geheel getal	<code>GetById(int id)</code>	GET <code>api/recept/abc</code> geeft 404
<code>[HttpGet("vast/{x}")]</code>	een vast stuk en een parameter	<code>keuken/{keuken}</code>	GET <code>api/recept/keuken/thais</code>
<code>[HttpGet("{a}/{b}")]</code>	twee parameters in één route	<code>Bereken(int a, int b)</code>	GET <code>api/recept/2/3</code>
<code>[HttpPost]</code>	antwoordt op POST	<code>VoegToe(Recept r)</code>	POST <code>api/recept</code>
<code>[HttpPut("{id}")]</code>	antwoordt op PUT	<code>Wijzig(int id, Recept r)</code>	PUT <code>api/recept/5</code>
<code>[HttpDelete("{id}")]</code>	antwoordt op DELETE	<code>Verwijder(int id)</code>	DELETE <code>api/recept/5</code>
<code>[FromBody]</code>	haal deze parameter uit de body	<code>VoegToe([FromBody] Recept r)</code>	de JSON wordt een <code>Recept</code>
<code>[FromQuery]</code>	haal deze parameter uit de querystring	<code>Zoek([FromQuery] string zoek)</code>	<code>api/recept?zoek=pasta</code>

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>[FromRoute]</code>	haal deze parameter uit de route	<code>GetById([FromRoute] int id)</code>	zoals zonder attribuut

Veelgemaakte fouten

LET OP

- Twee methodes met hetzelfde verb en dezelfde route compileren, maar zodra een verzoek binnenkomt, weet ASP.NET Core niet welke het moet nemen. Dat geeft **500** met een `AmbiguousMatchException`. `{id}` en `{naam}` op dezelfde plaats tellen als dezelfde route.
- De naam tussen accolades moet exact die van de parameter zijn. `[HttpGet("{id}")]` met `GetById(int nummer)` vult nummer niet uit de route, en dan blijft het op 0.
- Een route in een methode-attribuut die begint met `/` negeert de route van de klasse. `[HttpGet("/zoek")]` is gewoon zoek, zonder `api/recept` ervoor.
- Een waarde uit de URL behoudt haar hoofdletters. Vergelijk je ze met een vaste tekst, denk dan aan `Equals` met `OrdinalIgnoreCase` uit hoofdstuk 1.

Voorspel de uitkomst

Bij een API voorspel je wat de client terugkrijgt. Deze vragen gebruiken de `ReceptController` van hierboven.

Vraag 1

```
GET api/recept/abc
```

Toon het antwoord

400 Bad Request

De route `{id}` past: er staat iets na `api/recept/`. Maar `abc` kan geen `int` worden. `[ApiController]` merkt dat en stuurt zelf een 400 terug, nog voor `GetById` draait.

Vraag 2

Dezelfde controller, maar nu met `[HttpGet("{id:int}")]` boven `GetById`.

```
GET api/recept/abc
```

Toon het antwoord

404 Not Found

Door de beperking `:int` past de route niet meer bij `abc`. Er is dus geen enkele methode voor dit adres, en dat is een 404. Vergelijk met vraag 1: een kleine beperking, een andere statuscode.

Vraag 3

Iemand voegt deze methode toe aan de `ReceptController`:

```
[HttpGet("{naam}")]
public ActionResult<Recept> GetByNaam(string naam) { /* ... */ }

GET api/recept/5
```

Toon het antwoord

500 Internal Server Error

`api/recept/{id}` en `api/recept/{naam}` zijn voor de route hetzelfde: een stuk tekst na `api/recept/`. Er passen nu twee methodes, en ASP.NET Core weigert te kiezen. Het compileerde nochtans zonder fout.

Vraag 4

```
GET api/RECEPT/keuken/Thais
```

Toon het antwoord

`GetPerKeuken` draait, met `keuken` gelijk aan `"Thais"`.

De hoofdletters in `RECEPT` maken voor de route niets uit. De waarde `Thais` komt wel letterlijk in de parameter, met hoofdletter. Vergelijk je die met `"thais"`, dan geeft `==` `vals`.

26. De helpers van een controller

IN JE API

Deze erf je van `ControllerBase`. Ze maken een antwoord met de juiste statuscode.

Methode	Statuscode	Voorbeeld
<code>Ok</code>	200 OK	<code>Ok(fans)</code>
<code>CreatedAtAction</code>	201 Created	<code>CreatedAtAction(nameof(GetFanById), new { id = fan.Id }, fan)</code>
<code>NoContent</code>	204 No Content	<code>NoContent()</code>
<code>BadRequest</code>	400 Bad Request	<code>BadRequest("De prijs moet groter zijn dan 0.")</code>
<code>NotFound</code>	404 Not Found	<code>NotFound(\$"Geen fan met Id {id}.")</code>
<code>StatusCode</code>	wat je zelf opgeeft	<code>StatusCode(500)</code>

Ze nemen allemaal een tekst of een object mee, en dat is wat de client in de body ziet. Een `NotFound()` zonder tekst is geldig maar zegt niets; een `NotFound()` met een zin erbij scheelt de volgende persoon tien minuten zoeken.

`ActionResult<T>` tegenover `IActionResult`

Een methode die een van deze helpers teruggeeft, heeft een terugkeertype nodig dat zowel een statuscode als gegevens kan dragen. In de cursus kom je er twee tegen: `ActionResult<T>` vanaf les 02, en `IActionResult` vanaf les 06, bij PUT en DELETE.

	<code>ActionResult<T></code>	<code>IActionResult</code>
Zegt wat er terugkomt	ja: een T	nee
<code>return recept;</code>	mag, wordt 200 OK	compileert niet — schrijf <code>return Ok(recept);</code>
<code>return NotFound();</code>	mag	mag
Scalar toont het type van het antwoord	ja	nee
Typisch gebruik	GET en POST, die iets teruggeven	PUT en DELETE, die enkel een statuscode teruggeven

LET OP

`ActionResult<T>` kan een waarde enkel rechtstreeks teruggeven als `T` een klasse is, geen interface. Bij `ActionResult<IEnumerable<Recept>>` compileert `return resultaat;` niet als resultaat zelf als `IEnumerable` getypt is. Maak er eerst met `ToList()` een lijst van, of kies meteen `ActionResult<List<Recept>>`.

Voorspel de uitkomst

Bij een API voorspel je geen tekst op het scherm, maar wat de client terugkrijgt: een statuscode en een body.

Vraag 1

Een methode eindigt met:

```
return NotFound("Onbekend");
```

Toon het antwoord

404 Not Found, met in de body de tekst **Onbekend**.

Vraag 2

Na een geslaagde DELETE eindigt de methode met:

```
return NoContent();
```

Toon het antwoord

204 No Content, met een lege body.

Dat is geen vergetelheid. Er valt na een verwijdering niets meer te tonen, en de statuscode zegt al dat het gelukt is.

Vraag 3

Een methode zoekt met `First` en het gevraagde `Id` bestaat niet:

```
var dier = dieren.First(d => d.Id == id);  
return Ok(dier);
```

Toon het antwoord

500 Internal Server Error

`First` crasht zodra er niets gevonden wordt, en die crash wordt nergens opgevangen. De server meldt dan een fout aan zijn kant — terwijl de fout eigenlijk bij de client lag, die een `Id` vroeg dat niet bestaat. Met `FirstOrDefault`, een controle op null en een `NotFound` was dit een nette 404 geworden. Kijk nog eens naar hoofdstuk 11.

Vraag 4

Twee methodes, allebei met een recept `r` dat gevonden werd:

```
public ActionResult<Recept> GetEen(int id)
{
    // ...
    return r;
}

public IActionResult GetAnder(int id)
{
    // ...
    return r;
}
```

Toon het antwoord

De eerste geeft **200 OK** met het recept. De tweede **compileert niet**.

`ActionResult<Recept>` weet dat er een recept terug mag komen en maakt er zelf een 200 van.
`IActionResult` kent geen recepten, enkel antwoorden. Daar moet je het recept zelf inpakken met `Ok(r)`.

27. Program.cs en dependency injection

IN JE API

Wat het is

Het startpunt en de schakelkast van je API. `Program.cs` draait één keer, bij het opstarten. Het doet vier dingen, altijd in deze volgorde:

1. een **bouwer** maken;
2. bij die bouwer **registreren** wat je API nodig heeft: de diensten;
3. de app **bouwen**;
4. vastleggen welke weg elk verzoek aflegt, en de app **starten**.

Hoe het eruitziet

```
var builder = WebApplication.CreateBuilder(args);

// 2. Diensten registreren
builder.Services.AddControllers();
builder.Services.AddOpenApi();
builder.Services.AddScoped<IReceptRepository, ReceptRepository>();

// 3. Bouwen
var app = builder.Build();

// 4. De weg van een verzoek, en starten
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();
    app.MapScalarApiReference();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Alles wat met `builder.Services` begint, staat vóór `builder.Build()`. Alles wat met `app.` begint, staat erna.

Dependency injection: vragen in plaats van zelf maken

Een controller heeft vaak iets anders nodig, zoals een repository. Hij maakt dat niet zelf met `new`. Hij **vraagt** erom in zijn constructor:

```
public class ReceptController : ControllerBase
{
    private readonly IReceptRepository _repository;

    public ReceptController(IReceptRepository repository)
    {
        _repository = repository;
    }
}
```

Komt er een verzoek binnen, dan ziet ASP.NET Core dat de controller een `IReceptRepository` nodig heeft. Het zoekt in de registraties welke klasse daarbij hoort, maakt er een object van en geeft dat mee. De controller kent enkel het contract uit [hoofdstuk 19](#). Zo kan je later een andere klasse registreren, bijvoorbeeld een die een database gebruikt, zonder één letter in de controller te veranderen.

Het `private readonly` veld uit [hoofdstuk 24](#) bewaart wat je kreeg, zodat geen andere methode het nog kan vervangen.

Hoe lang een dienst leeft

Registratie	Hoe vaak er een nieuw object komt	Typisch voor
<code>AddTransient<I, K>()</code>	elke keer dat iemand erom vraagt	kleine diensten zonder eigen gegevens
<code>AddScoped<I, K>()</code>	één per verzoek	repositories en de databasecontext; de cursus gebruikt dit
<code>AddSingleton<I, K>()</code>	één voor de hele levensduur van de app	iets dat iedereen moet delen

BELANGRIJK

Bij `AddScoped` krijgt elk verzoek een nieuwe repository. Bewaart die repository haar gegevens in een gewone lijst, dan begint elk verzoek met een verse lijst: wat je met een POST toevoegde, is bij de volgende GET weg. Dat is hetzelfde probleem als bij de controller in [hoofdstuk 24](#). Met een database speelt het niet, want die onthoudt de gegevens zelf.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>WebApplication.CreateBuilder</code>	maak de bouwer	<code>var builder = WebApplication.CreateBuilder(args);</code>	de bouwer, met de standaardinstellingen
<code>AddControllers</code>	zet controllers aan	<code>builder.Services.AddControllers();</code>	zonder dit bestaan je controllers niet
<code>AddOpenApi</code>	beschrijf je API voor Scalar	<code>builder.Services.AddOpenApi();</code>	—
<code>AddScoped</code>	registreer: één object per verzoek	<code>AddScoped<IReceptRepository, ReceptRepository>()</code>	wie het contract vraagt, krijgt de klasse
<code>AddTransient</code>	registreer: elke keer een nieuw object	<code>AddTransient<IKlok, Klok>()</code>	—
<code>AddSingleton</code>	registreer: één object voor de hele app	<code>AddSingleton<ITeller, Teller>()</code>	—
<code>GetConnectionString</code>	lees een verbinding uit <code>appsettings.json</code>	<code>builder.Configuration.GetConnectionString("Standaard")</code>	de tekst onder <code>ConnectionString</code> s
<code>Build</code>	bouw de app; daarna liggen de registraties vast	<code>var app = builder.Build();</code>	de app
<code>IsDevelopment</code>	draait de app op je eigen computer	<code>if (app.Environment.IsDevelopment())</code>	<code>true</code> tijdens het ontwikkelen
<code>MapOpenApi, MapScalarApiReference</code>	maak de beschrijving en de Scalar-pagina bereikbaar	<code>app.MapScalarApiReference();</code>	de Scalar-pagina is bereikbaar
<code>UseHttpsRedirection</code>	stuur http door naar https	<code>app.UseHttpsRedirection();</code>	—
<code>UseAuthorization</code>	controleer wie wat mag	<code>app.UseAuthorization();</code>	belangrijk vanaf de beveiliging
<code>MapControllers</code>	koppel de routes van je controllers	<code>app.MapControllers();</code>	zonder dit geeft elke route 404
<code>Run</code>	start de app en blijf luisteren	<code>app.Run();</code>	wat erna staat, draait pas als de app stopt

OPMERKING

In nieuwere code zie je soms een kortere schrijfwijze, een *primary constructor*: `public class ReceptController(IReceptRepository repository) : ControllerBase`. De parameter staat dan meteen achter de klassenaam, en het veld en de constructor vallen weg. Het werkt op dezelfde manier.

Veelgemaakte fouten

LET OP

- Vraagt een controller iets dat niet geregistreerd is, dan compileert alles. Pas wanneer er een verzoek binnenkomt, krijg je een `InvalidOperationException: Unable to resolve service for type...` De client ziet **500**.
- Een registratie na `builder.Build()` komt te laat. Zet ze er altijd vóór.
- Vergeet je `app.MapControllers()`, dan start de app gewoon, maar geeft elke route 404.
- `new ReceptRepository()` in een controller werkt, maar breekt het hele idee. De controller hangt dan weer vast aan één klasse.

Voorspel de uitkomst

Vraag 1

Een repository bewaart recepten in een gewone lijst, geen `static`. Ze is geregistreerd met `AddScoped`. Je doet een POST die één recept toevoegt, en daarna een GET die alle recepten opvraagt. De lijst begon leeg.

Hoeveel recepten geeft de GET terug? En als de repository met `AddSingleton` geregistreerd was?

Toon het antwoord

Met `AddScoped`: 0. Met `AddSingleton`: 1.

`AddScoped` geeft elk verzoek een nieuwe repository, met een nieuwe, lege lijst. Het recept van de POST zat in de repository van dat ene verzoek. Bij `AddSingleton` is er één repository voor de hele app, en die onthoudt het recept.

Vraag 2

In `Program.cs` ontbreekt de regel met `AddScoped<IReceptRepository, ReceptRepository>()`. De `ReceptController` vraagt in zijn constructor een `IReceptRepository`. De app start, en je doet GET `api/recept`.

Toon het antwoord

500 Internal Server Error

De app start gewoon, want niemand heeft de controller al nodig. Bij het eerste verzoek wil ASP.NET Core er een maken, vindt geen klasse voor `IReceptRepository`, en geeft een `InvalidOperationException`.

Vraag 3

```
app.MapControllers();  
  
app.Run();  
  
Console.WriteLine("De API draait.");
```

Toon het antwoord

Zolang de API draait, verschijnt er niets.

`app.Run()` keert pas terug wanneer de app stopt. Wil je iets melden bij het opstarten, dan moet het vóór `app.Run()` staan.

28. Logging

IN JE API

Wat het is

Een logboek dat je API zelf bijhoudt. Terwijl de API draait, schrijft hij in het terminalvenster wat er gebeurt: welk verzoek er binnenkwam, wat er niet gevonden werd, wat er misliep. Zo zie je achteraf wat er gebeurd is, ook als niemand op dat moment keek.

Elke melding krijgt een **niveau**, van een detail dat enkel tijdens het ontwikkelen nuttig is, tot een fout die meteen aandacht vraagt.

Hoe het eruitziet

```
public class ReceptController : ControllerBase
{
    private readonly ILogger<ReceptController> _logger;

    public ReceptController(ILogger<ReceptController> logger)
    {
        _logger = logger;
    }

    [HttpGet("{id}")]
    public ActionResult<Recept> GetById(int id)
    {
        _logger.LogInformation("Recept {Id} opgevraagd", id);

        // ...

        _logger.LogWarning("Recept {Id} niet gevonden", id);
        return NotFound();
    }
}
```

De logger vraag je in de constructor, net als een repository in [hoofdstuk 27](#). Registreren hoeft niet: ASP.NET Core heeft hem al klaarstaan. Tussen de punthaken staat de klasse die logt. Die naam komt mee in het logboek, zodat je ziet waar een melding vandaan komt.

De niveaus

Methode	Wanneer	Standaard zichtbaar
<code>LogTrace</code>	de kleinste details	nee
<code>LogDebug</code>	details die enkel tijdens het ontwikkelen helpen	nee
<code>LogInformation</code>	de normale werking: een verzoek, een geslaagde actie	ja
<code>LogWarning</code>	iets onverwachts, maar de app werkt verder: een Id dat niet bestaat	ja
<code>LogError</code>	iets is misgelopen: een actie lukte niet	ja
<code>LogCritical</code>	de app zelf zit in de problemen	ja

Welke niveaus je ziet, staat in `appsettings.json`, onder `Logging` en `LogLevel`. Een nieuw project zet `Default` op `Information`: alles vanaf dat niveau verschijnt, `Debug` en `Trace` niet.

Twee schrijfwijzen

In les 03 schrijft de cursus de melding met een `$` ervoor, zoals in hoofdstuk 1: de waarden staan meteen in de tekst. Dat werkt.

Microsoft raadt een andere vorm aan, met een **sjabloon**: de tekst bevat plaatshouders tussen accolades, en de waarden komen erachter, gescheiden door komma's. Bijvoorbeeld `_logger.LogInformation("Recept {Id} opgevraagd", id);`. In het terminalvenster zie je hetzelfde. Het verschil is dat de logger de waarden apart bewaart, zodat een loggingprogramma later kan zoeken op *alle meldingen over recept 5*. Visual Studio stelt daarom soms voor om de eerste vorm om te zetten.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>ILogger<T></code>	een logger voor klasse T, via de constructor	<code>ILogger<ReceptController> logger</code>	de logger staat klaar
<code>LogInformation</code>	meld de normale werking	<code>_logger.LogInformation("Alle recepten opgevraagd");</code>	<code>info: ... Alle recepten opgevraagd</code>
<code>LogWarning</code>	meld iets onverwachts	<code>_logger.LogWarning("Recept {Id} niet gevonden", id);</code>	<code>warn: ... Recept 5 niet gevonden</code>
<code>LogError</code>	meld een fout	<code>_logger.LogError("Verwijderen van {Id} mislukt", id);</code>	<code>fail: ...</code>
<code>LogError</code> met uitzondering	meld een fout en geef de uitzondering mee	<code>_logger.LogError(ex, "Bestand niet gelezen");</code>	de melding, met de details van <code>ex</code> eronder
<code>{Naam}</code>	plaatshouder in een sjabloon	<code>"{Aantal} recepten", aantal</code>	<code>12 recepten</code>

Veelgemaakte fouten

LET OP

- In een sjabloon telt de **volgorde** van de waarden, niet de naam tussen de accolades. De eerste waarde komt in de eerste plaatshouder, wat er ook tussen de accolades staat.
- Een LogDebug die je nooit ziet, is meestal geen fout in je code. Het niveau staat in `appsettings.json` hoger ingesteld.
- Zet nooit wachtwoorden of andere geheime gegevens in een logboek. Wie het logboek kan lezen, kan ze dan ook lezen.

Voorspel de uitkomst

Vraag 1

```
_logger.LogInformation("{Peren} peren en {Appels} appels", 3, 8);
```

Toon het antwoord

3 peren en 8 appels

De eerste waarde gaat naar de eerste plaatshouder, de tweede naar de tweede. Dat de namen Peren en Appels heten, speelt geen rol bij welke waarde waar komt.

Vraag 2

In `appsettings.json` staat `"Default": "Information"`. Een methode bevat:

```
_logger.LogDebug("Begin van de methode");  
_logger.LogWarning("Geen recepten gevonden");
```

Wat verschijnt er in het terminalvenster?

Toon het antwoord

Enkel `Geen recepten gevonden`, als waarschuwing.

Debug ligt lager dan Information, dus die melding valt weg. Warning ligt hoger en komt erdoor.

29. Asynchroon werken

IN JE API

Vorm	Wat het doet
<code>async</code>	markeert een methode die mag wachten
<code>await</code>	wacht op het resultaat zonder de server te blokkeren
<code>Task</code>	een belofte dat er ooit iets klaar is
<code>Task<T></code>	een belofte dat er ooit een <code>T</code> klaar is

De asynchrone tegenhangers die je in de latere lessen tegenkomt dragen dezelfde naam met `Async` erachter: `ToListAsync`, `FirstOrDefaultAsync`, `FindAsync`, `SaveChangesAsync`, `AnyAsync`, `CountAsync`. Ze doen hetzelfde als hun gewone versie; het verschil zit in wat de server intussen mag doen.

OPMERKING

Deze komen pas aan bod in les 05 van Programming Advanced. Ze staan hier zodat je ze herkent wanneer je ze in een voorbeeld ziet staan, niet om ze nu al te gebruiken. Hoe je ze met een database gebruikt, staat in [hoofdstuk 32](#).

Hoe het eruitziet

```
async Task<int> TellLangzaamAsync()
{
    await Task.Delay(1000);    // wacht een seconde, zonder de thread vast te houden
    return 42;
}

int n = await TellLangzaamAsync();
Console.WriteLine(n);        // 42, na een seconde
```

In een controller ziet dat er zo uit:

```
[HttpGet]
public async Task<ActionResult<List<Recept>>> GetAll()
{
    List<Recept> recepten = await _repository.GetAllAsync();
    return recepten;
}
```

Drie dingen veranderen tegenover de gewone versie: `async` voor het terugkeertype, het terugkeertype zelf in een `Task<...>`, en `await` voor elke aanroep die tijd kost.

Async is besmettelijk

Les 05 noemt het het domino-effect. Gebruik je ergens `await`, dan moet die methode `async` worden en een `Task` teruggeven. Wie die methode oproept, moet dan zelf ook `await` gebruiken, en dus ook `async` worden. Zo schuift het naar boven: van de repository, via de interface, tot in de controller.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>async Task</code>	een asynchrone methode die niets teruggeeft	<code>async Task BewaarAsync()</code>	—
<code>async Task<T></code>	een asynchrone methode die een T teruggeeft	<code>async Task<int> TelAsync()</code>	<code>await TelAsync()</code> geeft een <code>int</code>
<code>await</code>	wacht op een <code>Task</code> en pak het resultaat uit	<code>int n = await TelAsync();</code>	<code>n</code> is een getal
<code>Task.Delay</code>	wacht een aantal milliseconden, zonder te blokkeren	<code>await Task.Delay(500);</code>	een halve seconde later gaat het verder
<code>Async</code> achter de naam	de afspraak voor asynchrone methodes	<code>GetAllAsync, SaveChangesAsync</code>	je ziet meteen dat er een <code>await</code> bij hoort
<code>.Result, .Wait()</code>	wacht door de thread vast te houden	<code>TelAsync().Result</code>	werkt, maar blokkeert; niet doen
<code>async void</code>	een asynchrone methode zonder <code>Task</code>	<code>async void Bewaar()</code>	fouten erin kan niemand opvangen; niet doen

Veelgemaakte fouten

LET OP

- Een vergeten `await` compileert vaak gewoon, met een waarschuwing. Je code gaat dan verder voor het werk klaar is.
- `.Result` en `.Wait()` maken van asynchrone code weer gewone, wachtende code. Alle winst is weg, en in sommige toepassingen loopt het programma vast, omdat twee stukken op elkaar wachten (een *deadlock*). Les 05 zegt het kort: hou het bij `async` en `await`.
- `async void` geeft geen `Task` terug. Wie de methode oproept, kan er dus niet op wachten en geen fout opvangen. Gebruik altijd `async Task` of `async Task<T>`.

Voorspel de uitkomst

Stel dat er een methode bestaat met deze kop:

```
Task<int> TelAsync()
```

Vraag 1

```
var x = await TelAsync();
```

Van welk type is `x`?

Toon het antwoord

```
int
```

`await` wacht tot de belofte ingelost is en geeft je wat erin zit: het getal.

Vraag 2

```
var x = TelAsync();
```

Van welk type is `x` nu?

Toon het antwoord

```
Task<int>
```

Zonder `await` krijg je de belofte zelf, niet het getal. Probeer je er daarna mee te rekenen, dan compileert het niet. Een vergeten `await` is een van de meest voorkomende fouten zodra je asynchroon begint te werken.

Vraag 3

```
async Task SchrijfAsync()
{
    Console.WriteLine("B");
    await Task.Delay(100);
    Console.WriteLine("D");
}

Console.WriteLine("A");
Task t = SchrijfAsync();
Console.WriteLine("C");
await t;
Console.WriteLine("E");
```

Toon het antwoord

A, B, C, D en daarna E

SchrijfAsync begint meteen en schrijft B. Bij de eerste await die echt moet wachten, geeft de methode de beurt terug. Daardoor komt C vóór D. Pas bij await t wacht het hoofdprogramma tot de methode helemaal klaar is, en dus komt E als laatste.

Vraag 4

```
async Task<int> GeefAsync()
{
    await Task.Delay(10);
    return 5;
}

int x = GeefAsync() + 1;
```

Toon het antwoord

Compileert niet.

Zonder await is GeefAsync() een Task<int>, een belofte, en een belofte plus 1 bestaat niet. Met await GeefAsync() + 1 lukt het wel: eerst het getal uitpakken, dan optellen.

30. JSON en bestanden

IN JE API

Methode	Wat het doet
<code>JsonSerializer.Serialize</code>	maak van een object een JSON-tekst
<code>JsonSerializer.Deserialize</code>	maak van een JSON-tekst een object
<code>File.ReadAllText</code>	lees een heel bestand als tekst
<code>File.WriteAllText</code>	schrijf een tekst naar een bestand
<code>File.Exists</code>	bestaat dat bestand
<code>Path.Combine</code>	plak padstukken correct aan elkaar

Ontbreekt het bestand, of staat er geen geldige JSON in, dan krijg je een uitzondering. Hoe je die opvangt, staat in [hoofdstuk 23](#).

LET OP

`JsonSerializer.Deserialize` is standaard hoofdlettergevoelig. Staat er "krakenId" in je bestand en heet je eigenschap `KrakenId`, dan blijft die leeg zonder één foutmelding. Dat gedrag zet je om met `PropertyNameCaseInsensitive`. Dat ASP.NET Core zelf wél soepel is met hoofdletters geldt enkel voor verzoeken die het framework zelf uitleest, niet voor een bestand dat je zelf inleest.

Voorspel de uitkomst

Vraag 1

```
var p = new { Naam = "Anna", Leeftijd = 17 };
Console.WriteLine(JsonSerializer.Serialize(p));
```

Toon het antwoord

```
{"Naam": "Anna", "Leeftijd": 17}
```

Alles op één regel, en de namen precies zoals in je code, met hoofdletter. Dat een API zijn JSON met kleine letters teruggeeft, komt doordat ASP.NET Core dat voor jou instelt; `JsonSerializer` zelf doet het niet.

Vraag 2

```
public class Persoon
{
    public string Naam { get; set; } = "";
}

var p = JsonSerializer.Deserialize<Persoon>("{\"naam\":\"Anna\"}");
Console.WriteLine("[ " + p!.Naam + " ]");
```

Toon het antwoord

[]

In de JSON staat naam met kleine letter, in de klasse Naam met hoofdletter. Standaard is dat voor JsonSerializer een andere naam, dus Naam blijft leeg. Er komt geen enkele foutmelding. Met `PropertyNameCaseInsensitive = true` in de opties lost je dit op.

Deel 6 — Data

Van gegevens in het geheugen naar gegevens in een database: bewaren, opvragen, en enkel doorgeven wat nodig is.

31. EF Core: model en database

DATA

Wat het is

Hoe het eruitziet

Twee entiteiten: een keuken heeft veel recepten, een recept hoort bij één keuken.

```
public class Keuken
{
    public int Id { get; set; }
    public string Naam { get; set; } = default!;

    public List<Recept> Recepten { get; set; } = new();
}

public class Recept
{
    public int Id { get; set; }
    public string Titel { get; set; } = default!;
    public decimal Kostprijs { get; set; }

    public int KeukenId { get; set; }           // de vreemde sleutel
    public Keuken Keuken { get; set; } = default!; // de navigatie-eigenschap
}
```

De context, met de verfijning in `OnModelCreating`:

```
public class ReceptContext : DbContext
{
    public ReceptContext(DbContextOptions<ReceptContext> options) : base(options) { }

    public DbSet<Keuken> Keukens { get; set; }
    public DbSet<Recept> Recepten { get; set; }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        modelBuilder.Entity<Recept>(entity =>
        {
            entity.Property(r => r.Titel).IsRequired().HasMaxLength(100);
            entity.Property(r => r.Kostprijs).HasPrecision(8, 2);

            entity.HasOne(r => r.Keuken)
                .WithMany(k => k.Recepten)
                .HasForeignKey(r => r.KeukenId)
                .OnDelete(DeleteBehavior.Restrict);
        });
    }
}
```

De registratie in `Program.cs`, met de verbinding uit `appsettings.json`:

```
var connectionString = builder.Configuration.GetConnectionString("Standaard");
builder.Services.AddDbContext<ReceptContext>(options => options.UseNpgsql(connectionString));

"ConnectionStrings": {
  "Standaard": "Host=localhost;Port=5432;Database=ReceptenDb;Username=postgres;Password=..."
}
```

`AddDbContext` registreert de context zoals `AddScoped` uit [hoofdstuk 27](#): één context per verzoek. Een repository vraagt hem gewoon in haar constructor.

Wat EF Core zelf afleidt

Veel hoeft je niet te zeggen. EF Core volgt **conventies**: vaste afspraken over namen en types.

Je schrijft	EF Core maakt ervan
<code>DbSet<Recept></code> <code>Recepten</code> in de context	een tabel <code>Recepten</code>
een eigenschap <code>Id</code> of <code>ReceptId</code>	de primaire sleutel; bij een <code>int</code> telt de database zelf op
<code>string</code>	een verplichte kolom van onbeperkte lengte
<code>string?</code> , <code>int?</code>	een kolom die leeg mag blijven
<code>decimal</code>	een kolom voor kommagetallen, exact, zonder afrondingsfouten
<code>int</code> <code>KeukenId</code> naast <code>Keuken</code> <code>Keuken</code>	een vreemde sleutel naar de tabel <code>Keukens</code>
<code>int</code> <code>KeukenId</code>	een verplichte relatie: elk recept heeft een keuken
<code>int?</code> <code>KeukenId</code>	een optionele relatie: een recept mag zonder keuken

Alles wat afwijkt van die afspraken, of wat je strenger wil, zet je in `OnModelCreating`. Die schrijfwijze, met methodes achter elkaar, heet de **Fluent API**. `override` en `base` ken je uit [hoofdstuk 20](#).

Relaties

Een **navigatie-eigenschap** is een eigenschap die naar een ander object wijst, zoals `Recept.Keuken`, of naar een lijst objecten, zoals `Keuken.Recepten`. De **vreemde sleutel** is het getal in de tabel dat die koppeling bewaart, zoals `KeukenId`.

Soort	Voorbeeld	Waar de vreemde sleutel staat	Navigatie-eigenschappen
één op één	een leerling en zijn kluisje	aan één kant, die je zelf kiest	aan elke kant één object
één op veel	een keuken en haar recepten	aan de <i>veel</i> -kant: <code>Recept.KeukenId</code>	<code>Recept.Keuken</code> en de lijst <code>Keuken.Recepten</code>
veel op veel	recepten en ingrediënten	in een aparte koppeltabel, met twee vreemde sleutels	elke kant een lijst koppelobjecten

Bij veel op veel maakt de cursus de koppeltabel als een eigen klasse, met twee één-op-veel-relaties. Zo kan de koppeling zelf ook gegevens dragen, zoals de hoeveelheid van een ingrediënt in een recept.

Een relatie in de Fluent API lees je vanuit de entiteit die je configureert:

Regel	Lees als
<code>HasOne(r => r.Keuken)</code>	een recept heeft één keuken
<code>.WithMany(k => k.Recepten)</code>	die keuken heeft veel recepten
<code>.HasForeignKey(r => r.KeukenId)</code>	de koppeling zit in <code>KeukenId</code>
<code>.onDelete(DeleteBehavior.Restrict)</code>	een keuken met recepten mag niet weg

Verwijderen: Cascade of Restrict

	Cascade	Restrict
Standaard bij	een verplichte relatie, zoals <code>int KeukenId</code>	nooit, je kiest het zelf
Een keuken met recepten verwijderen	de recepten verdwijnen mee	<code>SaveChanges</code> weigert, met een uitzondering
Het gevaar	gegevens verdwijnen zonder dat iemand het merkt	geen; je ruimt eerst zelf op

Les 06 vertelt het verhaal van klant Gerrit, die verwijderd wil worden. Met **Cascade** gaan al zijn bestellingen stilletjes mee. Met **Restrict** houdt de database tegen.

Migraties

Een migratie is een script dat beschrijft hoe de database moet veranderen om weer bij je klassen te passen. Elk script heeft een `Up`, die de wijziging uitvoert, en een `Down`, die ze terugdraait.

Package Manager Console (Visual Studio)	Terminal	Wat het doet
Add-Migration Naam	<code>dotnet ef migrations add Naam</code>	maak een nieuw script in de map <code>Migrations</code>
Update-Database	<code>dotnet ef database update</code>	voer de scripts uit die de database nog niet kent
Remove-Migration	<code>dotnet ef migrations remove</code>	verwijder het laatste script, zolang de database het nog niet uitvoerde

De vaste cyclus, telkens je een entiteit toevoegt of aanpast:

1. pas je klasse aan, en voeg indien nodig een `DbSet` toe;
2. `Add-Migration` met een naam die zegt wat er verandert;
3. kijk het script na: heeft EF Core begrepen wat je bedoelde?
4. `Update-Database`, met Docker aan.

Je hebt drie packages nodig: `Npgsql.EntityFrameworkCore.PostgreSQL`, `Microsoft.EntityFrameworkCore.Design` en, voor de Package Manager Console, `Microsoft.EntityFrameworkCore.Tools`. EF Core zelf komt mee met het eerste. Les 04 installeert het ook apart, en dat kan geen kwaad. In de terminal installeer je eenmalig ook het programma zelf, met `dotnet tool install --global dotnet-ef`.

LET OP

In les 04 staat op een paar plaatsen `dotnet Entity Framework migrations add` en `dotnet Entity Framework database update`. De commando's heten `dotnet ef migrations add` en `dotnet ef database update`. De lange vorm herkent de terminal niet.

Startgegevens met HasData

Met `HasData` geef je rijen mee die de migratie meteen in de tabel zet. Zo heeft je database na `Update-Database` al gegevens om mee te testen. Twee regels: je vult het `Id` zelf in, ook al telt de database het anders zelf op, en een relatie leg je via de vreemde sleutel, niet via de navigatie-eigenschap.

```
modelBuilder.Entity<Keuken>().HasData(  
    new Keuken { Id = 1, Naam = "Italiaans" },  
    new Keuken { Id = 2, Naam = "Thais" });
```

LET OP

PostgreSQL telt de `Ids` met een teller die niet weet dat jouw startgegevens 1 en 2 al gebruiken. De eerste keuken die je daarna via de API toevoegt, krijgt ook `Id 1`, en dat geeft een fout over een dubbele sleutel: een **500**. De documentatie van `Npgsql` noemt twee uitwegen: de teller laten beginnen boven je startgegevens, met `HasIdentityOptions`, of je startgegevens negatieve `Ids` geven.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>DbContext</code>	de basisklasse van je context	<code>class ReceptContext : DbContext</code>	de brug naar de database
<code>DbSet<T></code>	één tabel	<code>public DbSet<Recept> Recepten { get; set; }</code>	de tabel Recepten
<code>: base(options)</code>	geef de opties door aan <code>DbContext</code>	in de constructor van de context	de context weet welke database
<code>AddDbContext</code>	registreer de context, één per verzoek	<code>builder.Services.AddDbContext<ReceptContext>(...)</code>	repositories kunnen hem vragen
<code>UseNpgsql</code>	gebruik PostgreSQL, met deze verbinding	<code>options.UseNpgsql(connectionString)</code>	—
<code>OnModelCreating</code>	de plaats voor de Fluent API	<code>protected override void OnModelCreating(ModelBuilder modelBuilder)</code>	—
<code>Entity<T></code>	configureer één entiteit	<code>modelBuilder.Entity<Recept>(entity => { ... })</code>	—
<code>ToTable</code>	kies zelf de naam van de tabel	<code>entity.ToTable("Recepten")</code>	—
<code>HasKey</code>	kies zelf de primaire sleutel	<code>entity.HasKey(r => r.Code)</code>	nodig als de sleutel geen <code>Id</code> heet
<code>Property</code>	configureer één eigenschap	<code>entity.Property(r => r.Titel)</code>	—
<code>IsRequired</code>	de kolom mag niet leeg zijn	<code>.IsRequired()</code>	—
<code>HasMaxLength</code>	een maximale lengte voor tekst	<code>.HasMaxLength(100)</code>	hoogstens 100 tekens
<code>HasPrecision</code>	het aantal cijfers van een kommagetal	<code>.HasPrecision(8, 2)</code>	8 cijfers, waarvan 2 na de komma
<code>HasOne + WithMany</code>	één op veel, gelezen vanaf de <i>veel</i> -kant	<code>entity.HasOne(r => r.Keuken).WithMany(k => k.Recepten)</code>	—
<code>HasOne + WithOne</code>	één op één	<code>.HasOne(l => l.Kluisje).WithOne(k => k.Leerling)</code>	—
<code>HasForeignKey</code>	welke eigenschap de vreemde sleutel is	<code>.HasForeignKey(r => r.KeukenId)</code>	bij één op één met het type erbij: <code>HasForeignKey<Kluisje>(k => k.LeerlingId)</code>
<code>OnDelete</code>	wat er gebeurt als de andere kant verdwijnt	<code>.OnDelete(DeleteBehavior.Restrict)</code>	—
<code>HasData</code>	startgegevens via een migratie	<code>modelBuilder.Entity<Keuken>().HasData(...)</code>	rijen in de tabel na Update-Database

Veelgemaakte fouten

LET OP

- Een klasse aangepast, maar geen migratie gemaakt of Update-Database vergeten? Alles compileert, maar bij het eerste verzoek meldt PostgreSQL dat een kolom of tabel niet bestaat. De client ziet **500**.
- Heet de sleutel geen Id, en ook niet de naam van de klasse met Id erachter, en staat er geen HasKey, dan stopt Add-Migration met *The entity type '...' requires a primary key to be defined*.
- Pas de database nooit met de hand aan. De volgende migratie gaat ervan uit dat de database er nog uitziet zoals de vorige migratie hem achterliet.
- Draait Docker niet, dan kan Update-Database de database niet bereiken.
- Zet `base.OnModelCreating(modelBuilder)` altijd bovenaan. Bij een gewone DbContext merk je het verschil niet, maar erft je context later van een klasse die zelf tabellen configureert, zoals bij de beveiliging, dan krijg je zonder die regel fouten over ontbrekende sleutels.

Voorspel de uitkomst

Vraag 1

De context bevat `public DbSet<Ingredient> Ingredienten { get; set; }`, en verder niets over ingrediënten.

```
public class Ingredient
{
    public int Nummer { get; set; }
    public string Naam { get; set; } = default!;
}

Add-Migration Ingredienten
```

Toon het antwoord

Er komt geen migratie. EF Core meldt dat Ingredient een primaire sleutel nodig heeft.

EF Core zoekt een eigenschap die Id of IngredientId heet. Nummer past bij geen van beide. Hernoem de eigenschap, of wijs ze aan met HasKey.

Vraag 2

Recept heeft `public int KeukenId`, en in `OnModelCreating` staat niets over `OnDelete`. De keuken met Id 1 heeft drie recepten. Je verwijdert die keuken en roept `SaveChangesAsync` op.

Hoeveel recepten van die keuken staan er daarna nog in de database?

Toon het antwoord

0

`int KeukenId` kan niet leeg zijn, dus de relatie is verplicht. Bij een verplichte relatie kiest EF Core standaard Cascade: de recepten gaan mee met hun keuken. Geen foutmelding, geen waarschuwing.

Vraag 3

Hetzelfde, maar nu met `.OnDelete(DeleteBehavior.Restrict)` op de relatie. Het verwijderen gebeurt in een DELETE-methode van een controller, zonder `try` en `catch`.

Toon het antwoord

500 Internal Server Error, en de keuken staat nog in de database, met haar drie recepten.

`SaveChangesAsync` gooit een uitzondering, omdat er nog recepten aan de keuken hangen. Er wordt dus niets verwijderd. Wil je de client een duidelijker antwoord geven, kijk dan vóór het verwijderen of er nog recepten zijn, en geef een **400 Bad Request** met uitleg.

Vraag 4

Je voegt `public int Bereidingstijd { get; set; }` toe aan `Recept`. Je maakt een migratie met `Add-Migration`, maar vergeet `Update-Database`. Dan start je de API en doe je `GET api/recept`.

Toon het antwoord

500 Internal Server Error

De migratie staat klaar in je project, maar de database kent ze nog niet. EF Core vraagt de kolom `Bereidingstijd` op, en PostgreSQL meldt dat die niet bestaat. Het script maken en het script uitvoeren zijn twee aparte stappen.

32. EF Core: opvragen en bewaren

DATA

Wat het is

Een **winkelmandje**. Wat je erin legt, eruit haalt of omruilt, is nog niet betaald. Pas aan de kassa gaat alles in één keer door. Bij EF Core is de context het mandje: hij noteert wat je toevoegt, wijzigt en verwijdert. Dat noteren heet de **change tracker**. De kassa is `SaveChangesAsync`: pas dan maakt EF Core er SQL van en stuurt hij alles naar de database.

Opvragen werkt anders. Een vraag gaat meteen naar de database, op het moment dat je het antwoord opeist.

De voorbeelden gebruiken de `ReceptContext` uit [hoofdstuk 31](#), bewaard in een veld `_context`.

Hoe het eruitziet

```
// opvragen
List<Recept> alle = await _context.Recepten.ToListAsync();
Recept? vijf = await _context.Recepten.FindAsync(5);
List<Recept> goedkoop = await _context.Recepten
    .Where(r => r.Kostprijs < 5)
    .OrderBy(r => r.Titel)
    .ToListAsync();

// bewaren
_context.Recepten.Add(nieuwRecept);
await _context.SaveChangesAsync();
```

`Where`, `OrderBy` en `Select` ken je uit [hoofdstuk 11](#). Ze werken hier net zo, met één verschil: EF Core vertaalt ze naar SQL, en de database doet het filteren. De asynchrone methodes zoals `ToListAsync` staan in de namespace `Microsoft.EntityFrameworkCore`. Zonder die `using` bovenaan kent je code ze niet.

Opvragen: de vraag vertrekt pas als je het antwoord opeist

`_context.Recepten.Where(...)` geeft een `IQueryable<Recept>`. Net als de `IEnumerable` uit [hoofdstuk 12](#) is dat een vraag die nog niet gesteld is. Het verschil: een `IQueryable` wordt vertaald naar SQL, zodat de database het werk doet in plaats van je programma.

Bouwt de vraag verder	Stuurt de vraag naar de database
<code>Where</code> , <code>OrderBy</code> , <code>Select</code> , <code>Include</code> , <code>Take</code> , <code>Skip</code>	<code>ToListAsync</code> , <code>FirstOrDefaultAsync</code> , <code>FindAsync</code> , <code>CountAsync</code> , <code>AnyAsync</code> , een <code>foreach</code>

Elke keer dat je dezelfde `IQueryable` uitvoert, vertrekt er een nieuwe vraag naar de database. Les 05 noemt dit *deferred execution*: uitgestelde uitvoering.

OPMERKING

Geeft een repository een lijst terug, en filter je daarna in de controller met `Where`, dan gebeurt dat filteren in het geheugen, op alles wat de database stuurde. Zet een filter dus in de vraag zelf, vóór `ToListAsync`.

Bewaren: wat de context noteert

Je doet	De context noteert	Bij SaveChangesAsync
Add(recept)	nieuw	een INSERT
een opgevraagd object aanpassen	gewijzigd, vanzelf	een UPDATE van de gewijzigde kolommen
Update(recept), of Attach met State = EntityState.Modified	gewijzigd	een UPDATE van alle kolommen
Remove(recept)	te verwijderen	een DELETE
niets	ongewijzigd	niets

SaveChangesAsync geeft het aantal rijen terug dat het schreef. Een nieuw object heeft na Add nog geen Id: de database kiest dat getal pas bij het bewaren. Daarom staat een CreatedAtAction met het nieuwe Id altijd ná SaveChangesAsync.

FindAsync zoekt op de primaire sleutel, en kijkt eerst of de context dat object al kent. Is dat zo, dan vraagt hij niets aan de database.

Gerelateerde gegevens: Include en ThenInclude

Vraag je een recept op, dan komt alleen het recept mee. recept.Keuken blijft null. EF Core laadt gerelateerde gegevens niet vanzelf: je zegt het met Include.

Stel dat een recept ook een lijst ReceptIngredienten heeft, de koppelklasse uit hoofdstuk 31, en dat elk koppelobject zijn Ingredient kent:

```
Recept? recept = await _context.Recepten
    .Include(r => r.Keuken)
    .Include(r => r.ReceptIngredienten)
    .ThenInclude(ri => ri.Ingredient)
    .FirstOrDefaultAsync(r => r.Id == id);
```

Include vertrekt altijd van het recept. ThenInclude gaat één stap verder vanaf de vorige Include: van het koppelobject naar zijn ingrediënt. Alles samen wordt één vraag aan de database.

Zonder Include zou je per recept een aparte vraag moeten stellen om de keuken op te halen: één vraag voor de lijst, en dan nog eens N vragen, één per recept. Les 08 noemt dat het **N+1-probleem**. Het gebeurt ongemerkt bij *lazy loading*, waarbij gegevens pas geladen worden zodra je de eigenschap gebruikt. In EF Core staat lazy loading standaard uit.

OPMERKING

Bovenaan les 08 staat dat EF Core standaard werkt "via het Lazy Loading principe". Daar bedoelt de les: gelinkte gegevens komen niet vanzelf mee. Verderop in de les, en in de documentatie van Microsoft, betekent *lazy loading* iets anders: vanzelf laden zodra je de eigenschap aanraakt. Dat doet EF Core standaard niet. Zonder Include blijft de eigenschap gewoon null.

Kringlopen in JSON

Een recept wijst naar zijn keuken, en die keuken heeft een lijst met haar recepten, waaronder hetzelfde recept. Geef je zo'n object rechtstreeks terug uit een controller, dan probeert de JSON-omzetter die kring eindeloos te volgen. Hij stopt met een `JsonException: A possible object cycle was detected`.

Les 08 lost dat snel op met één regel in `Program.cs`: `.AddJsonOptions(o => o.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles)` achter `AddControllers()`, met `using System.Text.Json.Serialization;` bovenaan. Les 08 schrijft de volledige naam uit, dan heb je die `using` niet nodig. Waar de kring zich sluit, komt dan `null`.

BELANGRIJK

Les 08 zegt het zelf: `IgnoreCycles` is een snelle oplossing, maar op het examen verwacht men DTO's. Die staan in hoofdstuk 33.

In een generieke repository en een Unit of Work

Les 07 maakt één repository voor alle entiteiten, met de open plaats uit hoofdstuk 21. Omdat de repository niet weet welke tabel ze krijgt, schrijft ze niet `_context.Recepten` maar `_context.Set<TEntity>()`: de `DbSet` van het type dat ingevuld werd.

De repositories roepen zelf geen `SaveChangesAsync` meer op. Dat doet de **Unit of Work**: één klasse die alle repositories aanbiedt en ze allemaal dezelfde context meegeeft. Omdat ze één context delen, noteert die ene context alle wijzigingen. Eén `SaveChangesAsync` op de Unit of Work bewaart ze dan samen, of geen enkele.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>ToListAsync</code>	voer de vraag uit, alles in een lijst	<code>await _context.Recepten.ToListAsync()</code>	een <code>List<Recept></code>
<code>FindAsync</code>	zoek op de primaire sleutel	<code>await _context.Recepten.FindAsync(5)</code>	het recept, of <code>null</code>
<code>FirstOrDefaultAsync</code>	de eerste die voldoet	<code>.FirstOrDefaultAsync(r => r.Titel == "Soep")</code>	het recept, of <code>null</code>
<code>AnyAsync</code>	is er minstens één	<code>.AnyAsync(r => r.KeukenId == 3)</code>	<code>true</code> of <code>false</code>
<code>CountAsync</code>	hoeveel zijn er	<code>.CountAsync(r => r.Kostprijs > 10)</code>	een getal
<code>Add</code>	noteer als nieuw	<code>_context.Recepten.Add(recept)</code>	het Id is nog <code>0</code>
<code>Update</code>	noteer als gewijzigd	<code>_context.Recepten.Update(recept)</code>	—
<code>Attach</code>	laat de context een object volgen	<code>_context.Set<TEntity>().Attach(entity)</code>	gevolgd als ongewijzigd
<code>Entry(...).State</code>	zet zelf de toestand van een object	<code>_context.Entry(entity).State = EntityState.Modified;</code>	gevolgd als gewijzigd
<code>Remove</code>	noteer om te verwijderen	<code>_context.Recepten.Remove(recept)</code>	—
<code>SaveChangesAsync</code>	stuur alle genoteerde wijzigingen naar de database	<code>int n = await _context.SaveChangesAsync();</code>	het aantal geschreven rijen
<code>Include</code>	laad een navigatie-eigenschap mee	<code>.Include(r => r.Keuken)</code>	<code>recept.Keuken</code> is ingevuld

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>ThenInclude</code>	ga één stap dieper vanaf de vorige Include	<code>.ThenInclude(ri => ri.Ingredient)</code>	—
<code>Set<T></code>	de DbSet van een type dat je pas later kent	<code>_context.Set<TEntity>()</code>	in een generieke repository
<code>IQueryable<T></code>	een vraag die naar SQL vertaald wordt	<code>var vraag = _context.Recepten.Where(...);</code>	nog niets uitgevoerd

Veelgemaakte fouten

LET OP

- Add, Update of Remove zonder `SaveChangesAsync` erachter: er gebeurt niets in de database, en je krijgt geen enkele foutmelding.
- Het `Id` van een nieuw object gebruiken vóór `SaveChangesAsync`. Het staat dan nog op 0.
- `recept.Keuken.Naam` zonder `Include`: de keuken is niet geladen, en je krijgt een `NullReferenceException`.
- `Include` werkt niet samen met `FindAsync`. Wil je gerelateerde gegevens mee, gebruik dan `FirstOrDefaultAsync` met een voorwaarde op het `Id`.
- Een eigen C#-methode in een `Where` kan EF Core niet naar SQL vertalen. Het compileert, maar bij het uitvoeren krijg je *The LINQ expression ... could not be translated*.
- Een `await` vergeten bij een EF-methode, en meteen een tweede oproepen? Dan werkt de context aan twee dingen tegelijk, en dat weigert hij met een uitzondering. Zet `await` voor elke asynchrone EF-methode.

Voorspel de uitkomst

Vraag 1

De tabel `Recepten` is net aangemaakt en nog leeg.

```
var r = new Recept { Titel = "Soep", KeukenId = 1 };
_context.Recepten.Add(r);
Console.WriteLine(r.Id);
await _context.SaveChangesAsync();
Console.WriteLine(r.Id);
```

Toon het antwoord

0 en daarna 1

Na `Add` is het recept enkel genoteerd. Het `Id` kiest de database, en die heeft het recept nog niet gezien. Na `SaveChangesAsync` vult EF Core het `Id` in met wat de database koos.

Vraag 2

De tabel `Recepten` bevat vier recepten.

```
_context.Recepten.Add(new Recept { Titel = "Curry", KeukenId = 2 });  
int aantal = await _context.Recepten.CountAsync();  
Console.WriteLine(aantal);
```

Toon het antwoord

4

`CountAsync` stelt de vraag aan de database, en daar staat de curry nog niet in. Het recept ligt nog in het mandje, want `SaveChangesAsync` is niet opgeroepen.

Vraag 3

```
var r = await _context.Recepten.FindAsync(2);  
r!.Titel = "Tomatensoep";  
await _context.SaveChangesAsync();
```

Er staat nergens `Update`. Heet recept 2 nu in de database *Tomatensoep*?

Toon het antwoord

Ja.

De context heeft het recept zelf opgehaald, en volgt het sindsdien. Hij merkt dat de titel veranderd is, en `SaveChangesAsync` stuurt een UPDATE. Update heb je enkel nodig voor een object dat de context niet zelf ophaalde, zoals een object dat uit de body van een verzoek komt.

Vraag 4

In een nieuwe context, waarin nog niets geladen werd:

```
var r = await _context.Recepten.FirstOrDefaultAsync(x => x.Id == 2);  
Console.WriteLine(r!.Keuken.Naam);
```

Toon het antwoord

`NullReferenceException` — het programma crasht.

Het recept is gevonden, maar zijn keuken niet meegeladen. `r.Keuken` is `null`, ook al staat er = default! achter de eigenschap. Met `.Include(x => x.Keuken)` vóór `FirstOrDefaultAsync` lukt het wel.

Vraag 5

```
var duur = _context.Recepten.Where(x => x.Kostprijs > 10);  
int aantal = await duur.CountAsync();  
List<Recept> lijst = await duur.ToListAsync();
```

Hoeveel keer vraagt deze code iets aan de database?

Toon het antwoord

2

duur is een vraag, geen antwoord. CountAsync stelt ze één keer, ToListAsync stelt ze opnieuw. Komt er tussen die twee een recept bij, dan kan aantal zelfs verschillen van lijst.Count. Heb je beide nodig, vraag dan de lijst op en tel die.

Vraag 6

Een GET-methode geeft een recept terug, opgehaald met `.Include(x => x.Keuken)`. De klasse Keuken heeft een lijst Recepten. In Program.cs staat niets over ReferenceHandler.

Toon het antwoord

500 Internal Server Error

Het recept wijst naar de keuken, en de keuken heeft het recept in haar lijst Recepten: EF Core vult die lijst aan met wat hij al geladen heeft. De JSON-omzetter loopt in een kring en stopt met een JsonException. Met IgnoreCycles krijg je wel een antwoord, met null waar de kring zich sluit. Met een DTO heb je het probleem niet.

33. DTO's en Mapster

DATA

Wat het is

De etalage, niet het magazijn. Je entiteit is het magazijn: alles wat je bewaart, met alle relaties en velden die niemand buiten je API hoeft te zien. Een **DTO**, een *Data Transfer Object*, is de etalage: een klasse of record met enkel wat de client te zien krijgt, in de vorm die hem past.

Een DTO lost vier dingen tegelijk op:

- geen kringlopen in je JSON, want de DTO wijst niet terug naar zijn ouder (zie [hoofdstuk 32](#));
- je stuurt niet meer dan nodig;
- velden die binnen moeten blijven, komen er gewoon niet in;
- je database mag veranderen zonder dat de client het merkt.

Van een entiteit een DTO maken is veel overtypen: `Titel = recept.Titel`, en zo verder voor elke eigenschap.

Mapster doet dat voor je.

Hoe het eruitziet

```
public class ReceptDto
{
    public int Id { get; set; }
    public string Titel { get; set; } = default!;
    public string KeukenNaam { get; set; } = default!;
}
```

Les 09 toont ook de kortere vorm als `record`, uit [hoofdstuk 14](#):

```
public record ReceptDto(int Id, string Titel, string KeukenNaam);
```

Omzetten, na `dotnet add package Mapster` en `using Mapster;`:

```
ReceptDto dto = recept.Adapt<ReceptDto>();
List<ReceptDto> dtos = recepten.Adapt<List<ReceptDto>>();
```

`Adapt` is een *extension method*: een methode die Mapster toevoegt aan elk object, zodat je ze kan oproepen alsof ze er altijd al stond.

Hoe Mapster een eigenschap vult

Mapster overloopt elke eigenschap van de DTO en zoekt in de bron:

1. een eigenschap met **exact dezelfde naam**, hoofdletters inbegrepen: `Titel` uit `Titel`;
2. een naam die aan elkaar geplakt is uit een eigenschap en de eigenschap daarvan: `KeukenNaam` uit `Keuken.Naam`. Dat heet **platslaan** (*flattening*);
3. vindt hij niets, dan laat hij de eigenschap op haar standaardwaarde staan, zonder foutmelding.

Wat in de bron staat en niet in de DTO, laat hij gewoon liggen.

OPMERKING

Platslaan werkt enkel als de keuken ook geladen is. Zonder `Include` is de keuken `null`. Mapster vangt dat op, zonder crash, en laat `KeukenNaam` gewoon leeg.

Eigen regels: een MapperProfile

Past een naam niet, of moet een waarde ergens anders vandaan komen, dan schrijf je een regel. Les 09 zet die regels samen in één klasse die `IRegister` belooft. Stel een `ReceptDetailDto` met een tekst `Herkomst` en een lijst `Ingredienten`:

```
public class MapperProfile : IRegister
{
    public void Register(TypeAdapterConfig config)
    {
        config.NewConfig<Recept, ReceptDetailDto>()
            .Map(dest => dest.Herkomst, src => src.Keuken.Naam)
            .Map(dest => dest.Ingredienten, src => src.ReceptIngredienten);
    }
}
```

Lees `.Map(dest => dest.Herkomst, src => src.Keuken.Naam)` als: *vul in het doel `Herkomst` met de naam van de keuken uit de bron*. Wat wel overeenkomt, blijft Mapster zelf doen. Voor de lijst `Ingredienten` zoekt Mapster op zijn beurt een regel om één koppelobject om te zetten.

Die klasse doet pas iets als `Program.cs` haar laat zoeken, met één regel:

```
TypeAdapterConfig.GlobalSettings.Scan(Assembly.GetExecutingAssembly());
```

`Scan` doorzoekt je project naar klassen die `IRegister` beloven, en leest hun regels. Bovenaan heb je `using Mapster`; en `using System.Reflection`; nodig.

OPMERKING

Les 09 raadt aan om in een profiel geen berekeningen te doen, zoals een totaalprijs. Code in een profiel is moeilijk te testen en te debuggen. Hou het profiel bij overzetten; het rekenwerk hoort in een controller, een service of een hulpklasse.

Wanneer wel, wanneer niet

Wel — alles wat je API teruggeeft aan de client. Voor wat binnenkomt maak je vaak een aparte DTO zonder `Id`, want dat kiest de database.

Niet — binnen je eigen code, tussen repository en database. Daar werk je met de entiteiten zelf.

Wat je ermee kan

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>Adapt<T></code>	maak een nieuw object van type <code>T</code>	<code>recept.Adapt<ReceptDto>()</code>	een <code>ReceptDto</code>
<code>Adapt</code> op een lijst	zet elk item om	<code>recepten.Adapt<List<ReceptDto>>()</code>	een <code>List<ReceptDto></code>
<code>bron.Adapt(doel)</code>	kopieer in een object dat al bestaat	<code>dto.Adapt(recept)</code>	<code>recept</code> krijgt de waarden van <code>dto</code>
<code>IRegister</code>	de interface voor een klasse met eigen regels	<code>class MapperProfile : IRegister</code>	—
<code>Register</code>	de methode waarin je de regels zet	<code>public void Register(TypeAdapterConfig config)</code>	—
<code>NewConfig</code>	begin een regel voor dit paar	<code>config.NewConfig<Recept, ReceptDto>()</code>	vervangt een eerdere regel voor hetzelfde paar
<code>ForType</code>	maak of vul een regel aan	<code>config.ForType<Recept, ReceptDto>()</code>	een eerdere regel blijft gelden
<code>Map</code>	vul deze doel-eigenschap met dit	<code>.Map(dest => dest.Herkomst, src => src.Kuiken.Naam)</code>	—
<code>Ignore</code>	laat deze doel-eigenschap met rust	<code>.Ignore(dest => dest.Id)</code>	<code>Id</code> wordt niet overschreven
<code>Scan</code>	zoek alle <code>IRegister</code> -klassen en lees hun regels	<code>GlobalSettings.Scan(...)</code>	één regel in <code>Program.cs</code> , zie hierboven

Veelgemaakte fouten

LET OP

- Een naam die net anders is, zoals `Naam` in de bron en `Titel` in de DTO, geeft geen fout. De eigenschap blijft gewoon leeg. Kijk bij een lege eigenschap eerst naar de namen.
- De `Scan`-regel vergeten in `Program.cs`: alles compileert en draait, maar je eigen regels worden nooit gelezen. Enkel wat vanzelf overeenkomt, wordt ingevuld.
- Twee keer `NewConfig` voor hetzelfde paar: de tweede vervangt de eerste helemaal. Wil je aanvullen, gebruik dan `ForType`, of zet alles in één regel.
- DTO's gebruiken én `IgnoreCycles` laten staan. Les 09 haalt die regel weer weg: een DTO zonder terugwijzende eigenschappen heeft hem niet nodig.

Voorspel de uitkomst

Deze vragen gebruiken gewone klassen met eigenschappen, zonder database.

Vraag 1

```
public class Dier
{
    public int Id { get; set; }
    public string Naam { get; set; } = "";
    public string Soort { get; set; } = "";
}

public class DierDto
{
    public string Naam { get; set; } = "";
    public string Ras { get; set; } = "";
}

var dto = new Dier { Id = 3, Naam = "Bo", Soort = "hond" }.Adapt<DierDto>();
Console.WriteLine($"[" + dto.Naam + "] [" + dto.Ras + "]");
```

Toon het antwoord

[Bo] []

`Naam` staat aan beide kanten, dus die wordt gekopieerd. `Ras` vindt geen bron en houdt zijn beginwaarde, een lege tekst. `Id` en `Soort` hebben geen plaats in de DTO en vallen weg. Nergens een foutmelding.

Vraag 2

```
public class Stad
{
    public string Naam { get; set; } = "";
}

public class Leerling
{
    public string Naam { get; set; } = "";
    public Stad Stad { get; set; } = new();
}

public class LeerlingDto
{
    public string Naam { get; set; } = "";
    public string StadNaam { get; set; } = "";
}

var anna = new Leerling { Naam = "Anna", Stad = new Stad { Naam = "Gent" } };
var dto = anna.Adapt<LeerlingDto>();
Console.WriteLine(dto.StadNaam);
```

Toon het antwoord

Gent

StadNaam bestaat niet in Leerling, maar Stad gevolgd door Naam wel. Mapster slaat dat vanzelf plat. Een eigen regel heb je pas nodig als de naam dat patroon niet volgt, zoals bij Woonplaats.

Vraag 3

LeerlingDto krijgt er twee teksten bij: Woonplaats en Klas. Leerling krijgt een eigenschap Klasgroep, met daarin een Code. Het profiel bevat:

```
config.NewConfig<Leerling, LeerlingDto>()
    .Map(dest => dest.Woonplaats, src => src.Stad.Naam);

config.NewConfig<Leerling, LeerlingDto>()
    .Map(dest => dest.Klas, src => src.Klasgroep.Code);
```

Welke van de twee wordt ingevuld?

Toon het antwoord

Enkel Klas.

De tweede NewConfig voor hetzelfde paar gooit de eerste regel weg. Woonplaats blijft leeg. Zet beide Map-regels achter één NewConfig, of begin de tweede met ForType.

Vraag 4

Het profiel van vraag 3 is verbeterd en zet beide regels achter één `NewConfig`. Maar in `Program.cs` ontbreekt de regel met `Scan`.

Wat staat er in `Naam`, `Woonplaats` en `Klas` van de DTO?

Toon het antwoord

`Naam` is ingevuld, `Woonplaats` en `Klas` blijven leeg.

Zonder `Scan` leest niemand het profiel. Mapster doet enkel wat het zelf kan: eigenschappen met dezelfde naam kopiëren, en platslaan. Er is geen enkele foutmelding die je op het spoor zet.

34. Dapper

DATA

OPMERKING

Les 08 zegt het duidelijk: op het examen ligt de focus op EF Core. Dapper hoeft je niet zelf te kunnen schrijven, maar je moet kunnen uitleggen wat het is, en waarom en wanneer iemand ervoor kiest. Dit hoofdstuk helpt je de code te lezen en het verschil te begrijpen.

Wat het is

Zelf de taal van de database spreken. EF Core is een tolk: jij schrijft C#, hij maakt er SQL van. Met Dapper schrijf je de SQL zelf, en Dapper doet enkel de laatste stap: de rijen die terugkomen omzetten in objecten. Zo'n kleine tussenpersoon heet een **micro-ORM**. EF Core is een volledige ORM.

	EF Core	Dapper
SQL	maakt het zelf, uit LINQ	schrijf je zelf
Wijzigingen bijhouden	ja, de change tracker	nee
Tabellen en migraties	ja	nee; de database moet al bestaan
Gerelateerde gegevens	Include	een JOIN in je SQL, en zelf de objecten koppelen
Sterk in	schrijven, en de structuur van je database	snel lezen, ingewikkelde rapporten

Grotere toepassingen combineren ze vaak. EF Core doet het schrijven, met change tracking en een Unit of Work. Dapper doet het lezen waar elke milliseconde telt. Les 08 noemt dat het hybride patroon, naar het principe *Command Query Separation*.

Hoe het eruitziet

```
using Dapper;

string sql = """
    SELECT * FROM "Recepten"
    WHERE "Id" = @Id
    """;

Recept? recept = await _dbConnection.QueryFirstOrDefaultAsync<Recept>(sql, new { Id = id });
```

`_dbConnection` is een `IDbConnection`: een verbinding met de database, die de repository in haar constructor krijgt. Dapper voegt zijn methodes toe aan die verbinding.

De drie aanhalingstekens openen een *raw string*: een tekst, op één regel of over meerdere regels, waarin gewone aanhalingstekens gewoon mogen staan. Dat komt van pas, want PostgreSQL wil de namen van tabellen en kolommen die EF Core maakte tussen aanhalingstekens. Zie de waarschuwing verderop.

Parameters: altijd met @

@Id in de SQL is een plaatshouder. De waarde komt uit het **anonieme object** `new { Id = id }` uit [hoofdstuk 13](#): de naam na @ moet de naam van een eigenschap in dat object zijn. Dapper geeft de waarde apart door aan de database, nooit als stuk tekst in de SQL.

Plak je een waarde zelf in de SQL, met een \$ en accolades zoals in [hoofdstuk 1](#), dan kan wie die waarde aanlevert, eigen SQL meesturen. Dat heet **SQL-injectie**. Vraag 1 toont hoe weinig daarvoor nodig is.

Een rij in twee objecten: multi-mapping

Met een `JOIN` komen een recept en zijn keuken samen op één rij. Dapper moet weten waar het recept ophoudt en de keuken begint:

```
string sql = """
    SELECT r.*, k.*
    FROM "Recepten" r
    INNER JOIN "Keukens" k ON r."KeukenId" = k."Id"
    WHERE r."Id" = @Id
    """;

var resultaat = await _dbConnection.QueryAsync<Recept, Keuken, Recept>(
    sql,
    (recept, keuken) =>
    {
        recept.Keuken = keuken;
        return recept;
    },
    param: new { Id = id },
    splitOn: "Id");
```

- Tussen de punthaken staan de types per stuk van de rij, en als laatste wat je terugkrijgt.
- `splitOn: "Id"` zegt: zodra je opnieuw een kolom Id tegenkomt, begint het volgende object. Id is ook wat Dapper aanneemt als je niets opgeeft.
- De lambda koppelt de stukken aan elkaar, zoals `Include` dat bij EF Core voor je doet.

Bij één op veel staat de ouder op elke rij opnieuw, één keer per kind. Les 08 houdt daarom een `Dictionary` bij uit [hoofdstuk 7](#), met het `Id` als sleutel, zodat elke ouder maar één keer in het resultaat komt.

Wat je ermee kan

In de voorbeelden is `db` de verbinding, `sql` de tekst van de query en `param` een anoniem object met de parameters, zoals `new { Id = 5 }`. Voor elke oproep staat nog een `await`.

Vorm	Wat het doet	Voorbeeld	Resultaat
<code>QueryAsync<T></code>	alle rijen, elk als object	<code>db.QueryAsync<Recept>(sql)</code>	een <code>IEnumerable<Recept></code>
<code>QueryFirstOrDefaultAsync<T></code>	de eerste rij, of niets	<code>db.QueryFirstOrDefaultAsync<Recept>(sql, param)</code>	het recept, of <code>null</code>
<code>QueryFirstAsync<T></code>	de eerste rij, die er moet zijn	<code>db.QueryFirstAsync<Recept>(sql, param)</code>	geen rij: <code>InvalidOperationException</code>
<code>QuerySingleOrDefaultAsync<T></code>	precies één rij, of niets	<code>db.QuerySingleOrDefaultAsync<Recept>(sql, param)</code>	meer dan één rij: <code>InvalidOperationException</code>
<code>ExecuteAsync</code>	voer een INSERT, UPDATE of DELETE uit	<code>db.ExecuteAsync(sql, param)</code>	het aantal geraakte rijen
<code>ExecuteScalarAsync<T></code>	voer uit en geef één waarde terug	<code>db.ExecuteScalarAsync<int>(sql)</code>	één getal of tekst
<code>@Naam</code> + <code>new { Naam = ... }</code>	een parameter	<code>WHERE "Titel" = @Titel</code> met <code>new { Titel = titel }</code>	veilig voor SQL-injectie
<code>QueryAsync<A, B, A></code>	één rij verdelen over twee objecten	met een lambda en <code>splitOn</code>	het object A, met B eraan gekoppeld
<code>splitOn</code>	de kolom waar het volgende object begint	<code>splitOn: "Id"</code>	—

Dapper koppelt elke kolom aan de eigenschap met dezelfde naam. Anders dan bij Mapster maken hoofdletters daarbij niet uit: een kolom `titel` vult `Titel`. Vindt hij geen eigenschap, dan slaat hij de kolom over, zonder foutmelding.

Veelgemaakte fouten

LET OP

- **Een waarde in de SQL plakken** met \$ en accolades. Gebruik altijd @Naam en een anoniem object.
- **Namen zonder aanhalingstekens.** Tabellen die EF Core in PostgreSQL maakte, heten exact Recepten, met hoofdletter. PostgreSQL maakt van elke naam zonder aanhalingstekens kleine letters, en zoekt dan recepten. Die bestaat niet. Schrijf "Recepten" en "KeukenId".
- **SCOPE_IDENTITY() in PostgreSQL.** Het voorbeeld in les 08 haalt het nieuwe Id op met SELECT CAST(SCOPE_IDENTITY() as int). Dat is SQL Server. In PostgreSQL zet je RETURNING "Id" achter de INSERT, en lees je dat getal met ExecuteScalarAsync<int>.
- **Een IDbConnection vragen die niet geregistreerd is.** Les 08 toont niet hoe die in Program.cs komt. Je registreert ze met AddScoped, met een lambda die een NpgsqlConnection maakt met je connection string. Zonder registratie krijg je dezelfde fout als in [hoofdstuk 27](#).

Voorspel de uitkomst

db is een verbinding met een database waarin EF Core de tabel "Recepten" maakte. Die bevat twaalf recepten, en geen enkel recept heeft een vreemde titel.

Vraag 1

```
string titel = "' OR '1'='1";
string sql = $"SELECT * FROM \"Recepten\" WHERE \"Titel\" = '{titel}'";
var recepten = await db.QueryAsync<Recept>(sql);
Console.WriteLine(recepten.Count());
```

Toon het antwoord

12

Na het invullen staat er WHERE "Titel" = ' ' OR '1'='1'. Het tweede deel is altijd waar, dus elke rij voldoet. Met WHERE "Titel" = @Titel en new { Titel = titel } zoekt de database letterlijk naar die vreemde titel, en vindt ze er 0.

Vraag 2

```
int n = await db.ExecuteAsync(
    "DELETE FROM \"Recepten\" WHERE \"Id\" = @Id",
    new { Id = 999 });
Console.WriteLine(n);
```

Toon het antwoord

0

Er is geen recept 999, dus er wordt niets verwijderd. Dat is geen fout: de database meldt gewoon dat nul rijen geraakt zijn. In een controller is dat het moment voor een NotFound.

Vraag 3

```
var r = await db.QueryFirstOrDefaultAsync<Recept>(
    """SELECT "Id", "Titel" AS naam FROM "Recepten" WHERE "Id" = 1""");
Console.WriteLine($"[" + r!.Titel + "]");
```

Toon het antwoord

```
[]
```

De kolom heet in het antwoord naam, en Recept heeft geen eigenschap met die naam. Dapper slaat de kolom over. Titel krijgt dus niets, en blijft null. Het Id is wel ingevuld.

Vraag 4

```
var alle = await db.QueryAsync<Recept>("SELECT * FROM Recepten");
```

Toon het antwoord

Een uitzondering van PostgreSQL: *relation "recepten" does not exist*. In een API wordt dat een 500.

Zonder aanhalingstekens maakt PostgreSQL er recepten van, met kleine letters. EF Core maakte de tabel als "Recepten". Voor PostgreSQL zijn dat twee verschillende namen.

Vraag 5

```
var r = await db.QueryFirstOrDefaultAsync<Recept>(
    """SELECT * FROM "Recepten" WHERE "Id" = @Id""",
    new { Id = 999 });
Console.WriteLine(r is null);
```

Toon het antwoord

```
True
```

Geen rij gevonden, dus null, net als FirstOrDefault in hoofdstuk 11. Met QueryFirstAsync was het een InvalidOperationException geweest.

Veelgemaakte verwarringen

Dertig paren die op elkaar lijken en iets anders doen. Dit is het deel waar je het vaakst naar terug zal bladeren.

Length, Count en Count()

Een array en een tekst zeggen `Length`. Een `List` zegt `Count`, zonder haakjes, en weet het meteen. `Count()` met haakjes is LINQ: dat werkt op elke verzameling, maar loopt ze daarvoor helemaal af. Op een lijst neem je de eigenschap. Hoofdstukken [5](#), [6](#) en [11](#).

Array tegenover List

Een array heeft een vaste lengte, een `List` groeit mee. In de praktijk kies je bijna altijd de `List`, tenzij de hoeveelheid echt vastligt. Hoofdstukken [5](#) en [6](#).

List tegenover Dictionary

Zoek je in een `List` telkens met `Find` of `FirstOrDefault` op dezelfde eigenschap — een naam, een code — dan wil je eigenlijk een `Dictionary` met die eigenschap als sleutel. De `List` kijkt elk item na tot hij het vindt, de `Dictionary` weet meteen waar hij moet zijn. Zoek je afwisselend op verschillende eigenschappen, of doet de volgorde ertoe, dan blijft de `List` de betere keuze. Hoofdstukken [6](#) en [7](#).

Queue tegenover Stack

Allebei laten ze je enkel aan één kant bij. Stel jezelf de vraag: *wie komt er als volgende aan de beurt?* Wie het langst wacht — `Queue`. Wie het laatst kwam — `Stack`. Hoofdstukken [9](#) en [10](#).

Remove, RemoveAt en RemoveAll

`Remove` wil het item, `RemoveAt` wil een positie, `RemoveAll` wil een voorwaarde. Ze geven ook iets anders terug: `Remove` een ja of nee, `RemoveAll` het aantal dat weg is, `RemoveAt` niets. En `Remove` haalt enkel het eerste voorkomen weg. Hoofdstuk [6](#).

Parse tegenover TryParse

`Parse` gelooft je en crasht als je ongelijk hebt. `TryParse` vraagt het na en geeft `false`. Alles wat van buiten komt, gaat door `TryParse`. Hoofdstuk [2](#).

TryParse tegenover try/catch

Allebei voorkomen ze dat je programma stopt op een tekst die geen getal is. `TryParse` kijkt vooraf en geeft `false`; `try/catch` laat de fout gebeuren en vangt ze daarna op. Kan je vooraf kijken, doe dat dan: het is sneller en zegt duidelijker wat je bedoelt. Houd `try/catch` voor wat je vooraf niet kan nakijken, zoals een bestand. Hoofdstukken [2](#) en [23](#).

= tegenover ==

`=` geeft een variabele een waarde, `==` vraagt of twee waarden gelijk zijn. In een `if` hoort altijd `==`. C# weigert `if (x = 5)`, want een toewijzing is geen ja of nee. Hoofdstuk [4](#).

decimal tegenover double

Allebei zijn het kommagetallen. Een `double` is snel, maar wijkt soms een heel klein beetje af. Een `decimal` rekent tot op de cent juist. Voor geld en prijzen neem je `decimal`, en dan schrijf je een `m` achter elk getal: `19.99m`. Hoofdstuk [2](#).

== tegenover Equals bij tekst

Allebei vergelijken ze letterlijk, hoofdletters inbegrepen. Het verschil is dat `Equals` een tweede argument aanvaardt waarmee je die regel kan versoepelen. Zodra hoofdletters er niet toe mogen doen, heb je `Equals` nodig. Hoofdstuk [1](#).

Find tegenover FirstOrDefault

Op een lijst doen ze hetzelfde. `Find` bestaat enkel op een `List` en een array; `FirstOrDefault` werkt op elke verzameling, ook op een `databasequery`. Omdat je later met databases werkt en `Find` daar niet bestaat, went `FirstOrDefault` beter aan. Hoofdstukken [6](#) en [11](#).

First tegenover Single

`First` zegt: er is er minstens één, geef mij de eerste. `Single` zegt: er is er precies één, en als dat niet klopt is er iets grondig mis. Gebruik `Single` op een `Id`, en `First` op een sortering. Hoofdstuk [11](#).

Where tegenover Select

`Where` houdt items over, `Select` verbouwt ze. Krijg je minder items terug dan je erin stak, dan heb je gefilterd — dat is `Where`. Krijg je er evenveel terug maar van een ander soort, bijvoorbeeld enkel de namen, dan heb je verbouwd — dat is `Select`. Hoofdstuk [11](#).

Max tegenover MaxBy

`Max` geeft de waarde, `MaxBy` geeft het item dat die waarde draagt. Dezelfde verhouding geldt voor `Min` en `MinBy`. Hoofdstuk [11](#).

Any() tegenover Count() > 0

Hetzelfde antwoord, maar `Any` stopt bij het eerste item dat voldoet. Op een lijst van vijf maakt dat niets uit; op een databasetabel van vijftigduizend wel. Hoofdstuk [11](#).

Wat een methode teruggeeft, en wat ze verandert

`lijst.Sort()` verandert je lijst en geeft niets terug. `lijst.OrderBy(...)` laat je lijst met rust en geeft een nieuw resultaat. Hetzelfde geldt voor `Trim` en `ToUpper` bij tekst: die veranderen niets, ze geven een nieuwe tekst terug. Compileert je regel niet, of lijkt hij niets te doen, dan heb je meestal de verkeerde van de twee te pakken. Hoofdstukken [1](#), [6](#) en [11](#).

IEnumerable tegenover List

Een `IEnumerable` is een vraag die nog gesteld moet worden. Een `List` is een antwoord dat al klaarligt. `Where` voert niets uit tot je het resultaat doorloopt; verandert je bron intussen, dan verandert je antwoord mee. `ToList()` stelt de vraag meteen en legt het antwoord vast. Hoofdstuk [12](#).

Abstracte klasse tegenover interface

Een abstracte klasse zegt wat iets **is**, en geeft gegevens en uitgewerkte methodes mee. Een interface zegt wat iets **kan**, en geeft niets mee. Een klasse erft van hoogstens één abstracte klasse, maar kan zoveel interfaces beloven als ze wil. Delen je klassen gegevens, dan is het een abstracte klasse; delen ze enkel een vaardigheid, dan een interface. Hoofdstukken [19](#) en [20](#).

private tegenover protected

Allebei verbergen ze iets voor de buitenwereld. Het verschil zit bij wie erft: **private** is enkel zichtbaar in de klasse zelf, **protected** ook in elke klasse die ervan erft. Kies **private**, tenzij een afgeleide klasse het echt nodig heeft. Hoofdstuk [20](#).

static tegenover een gewoon veld

Een gewoon veld heeft elk object apart. Een **static** veld is er één keer, voor de hele klasse. Daarom onthoudt een **static** lijst in een controller wat er toegevoegd werd, terwijl een gewone lijst bij elk verzoek opnieuw leeg begint. Hoofdstuk [24](#).

const tegenover readonly

Allebei veranderen ze niet meer. Een **const** ligt al vast bij het compileren en kan enkel een getal, een tekst of een **bool** zijn. Een **readonly** veld vul je één keer in, bij de declaratie of in de constructor. Het kan dus ook iets zijn dat pas bekend is als het programma draait, zoals een repository. Hoofdstuk [24](#).

{id} tegenover {id:int}

Allebei halen ze een waarde uit de route. Zonder beperking past de route bij elke tekst, en geeft een tekst die geen getal is een 400. Met **:int** past de route enkel bij een geheel getal, en geeft al de rest een 404. Hoofdstuk [25](#).

ActionResult<T> tegenover IActionResult

ActionResult<T> zegt wat er terugkomt, en laat je dat ook rechtstreeks teruggeven. **IActionResult** kent enkel antwoorden, dus daar pak je je gegevens zelf in met **Ok(...)**. Geeft een methode gegevens terug, neem dan **ActionResult<T>**. Geeft ze enkel een statuscode terug, dan volstaat **IActionResult**. Hoofdstuk [26](#).

AddScoped tegenover AddSingleton

AddScoped maakt één object per verzoek, **AddSingleton** één voor de hele app. Bewaart een dienst zelf gegevens in het geheugen, dan is een scoped dienst ze na elk verzoek kwijt, terwijl een singleton ze onthoudt. **AddTransient** maakt elke keer een nieuw object. Hoofdstuk [27](#).

await tegenover .Result

Allebei geven ze je wat er in een **Task** zit. **await** laat de thread intussen vrij voor ander werk. **.Result** houdt hem vast tot het antwoord er is, en kan een programma zelfs laten vastlopen. In een API gebruik je altijd **await**. Hoofdstuk [29](#).

IEnumerable tegenover IQueryable

Allebei zijn het vragen die nog niet gesteld zijn. Een `IEnumerable` voert je programma zelf uit, op wat al in het geheugen zit. Een `IQueryable` wordt naar SQL vertaald, en de database voert hem uit. Filter je pas na `ToListAsync`, dan komt eerst alles uit de database, en filtert je programma daarna. Hoofdstukken [12](#) en [32](#).

Add tegenover SaveChangesAsync

Add legt iets in het mandje, `SaveChangesAsync` rekent af. Na Add staat er nog niets in de database, en heeft het object nog geen `Id`. Hetzelfde geldt voor `Update` en `Remove`. Hoofdstuk [32](#).

Include tegenover ThenInclude

`Include` vertrekt altijd van het object dat je opvraagt. `ThenInclude` gaat één stap verder vanaf de vorige `Include`. De keuken van een recept is een `Include`. De ingrediënten achter de koppelobjecten van een recept zijn een `Include`, gevolgd door een `ThenInclude`. Hoofdstuk [32](#).

Cascade tegenover Restrict

Allebei zeggen ze wat er gebeurt als je iets verwijdert waar nog andere rijen aan hangen. `Cascade` neemt die rijen mee, zonder iets te vragen. `Restrict` weigert, en laat jou eerst opruimen. Bij een verplichte relatie is `Cascade` de standaard. Hoofdstuk [31](#).

-1, null en 0

`IndexOf` geeft `-1` wanneer er niets gevonden is. `FirstOrDefault` geeft `null` bij objecten, maar `0` bij getallen. `TryGetValue` zet zijn uitkomst op de standaardwaarde. Drie manieren om "niets" te zeggen, en elk vraagt een andere controle. Hoofdstukken [1](#), [7](#), [11](#) en [17](#).

Index

Alfabetisch, met het hoofdstuk waar de uitleg staat.

Naam	Hoofdstuk
<code>abstract</code>	20
<code>ActionResult<T></code>	26
<code>Adapt</code>	33
<code>Add</code> (database)	32
<code>Add</code> (dictionary)	7
<code>Add</code> (lijst)	6
<code>Add-Migration</code>	31
<code>AddDays</code>	3
<code>AddDbContext</code>	31
<code>AddMonths</code>	3
<code>AddRange</code>	6
<code>AddScoped</code>	27
<code>AddSingleton</code>	27
<code>AddTransient</code>	27
<code>All</code>	11
anoniem object	13
<code>Any</code>	11
<code>AnyAsync</code>	32
<code>[ApiController]</code>	25
array	5
<code>as</code>	22
<code>async</code>	29
<code>Attach</code>	32
attribuut	25
<code>Average</code>	11
<code>await</code>	29
<code>BadRequest</code>	26
<code>base</code>	20
basisklasse	20
<code>break</code>	4

Naam	Hoofdstuk
<code>Build</code>	27
<code>Cascade</code>	31
<code>catch</code>	23
change tracker	32
<code>class</code>	13
<code>Clear</code>	6
<code>CompareTo</code>	1
<code>const</code>	24
constructor	13
<code>Contains</code> (lijst)	6
<code>Contains</code> (tekst)	1
<code>ContainsKey</code>	7
<code>continue</code>	4
<code>Count</code> (eigenschap)	6
<code>Count</code> (LINQ)	11
<code>CountAsync</code>	32
<code>CreatedAtAction</code>	26
<code>CultureInfo.InvariantCulture</code>	2
Dapper	34
<code>DateOnly</code>	3
<code>DateTime</code>	3
<code>DayOfWeek</code>	3
<code>DbContext</code>	31
<code>DbSet</code>	31
<code>decimal</code>	2
<code>default</code>	21
dependency injection	27
<code>Dequeue</code>	9
<code>Dictionary</code>	7
<code>Distinct</code>	11

Naam	Hoofdstuk
do	4
dotnet ef	31
double.Parse	2
DTO	33
EndsWith	1
Enqueue	9
EntityState	32
Entry	32
enum	18
Enum.TryParse	18
Equals	1
Exception	23
ExceptWith	8
ExecuteAsync	34
ExecuteScalarAsync	34
Exists	6
File.Exists	30
File.ReadAllText	30
File.WriteAllText	30
finally	23
Find	6
FindAll	6
FindAsync	32
FindIndex	6
First	11
FirstOrDefault	11
FirstOrDefaultAsync	32
Fluent API	31
for	4
ForType	33
[FromBody]	25
[FromQuery]	25
generics	21
GetConnectionString	27
GetType	22
GetValueOrDefault	17

Naam	Hoofdstuk
GroupBy	11
HasData	31
HasForeignKey	31
HashSet	8
HasKey	31
HasMaxLength	31
HasOne	31
HasPrecision	31
HasValue	17
[HttpDelete]	25
[HttpGet]	25
[HttpPost]	25
[HttpPut]	25
IActionResult	26
IEnumerable	12
if	4
Ignore (Mapster)	33
IgnoreCycles	32
ILogger	28
Include	32
IndexOf (lijst)	6
IndexOf (tekst)	1
Insert	6
int.Parse	2
int.TryParse	2
interface	19
IntersectWith	8
IQueryable	32
IRegister	33
is	22
IsEmptyOrNull	1
IsNullOrWhiteSpace	1
IsRequired	31
JsonSerializer.Deserialize	30
JsonSerializer.Serialize	30
Keys	7

Naam	Hoofdstuk
KeyValuePair	7
Last	11
LastIndexOf	1
LastOrDefault	11
lazy loading	32
Length	1
List	6
LogError	28
LogInformation	28
LogWarning	28
Map (Mapster)	33
MapControllers	27
Mapster	33
Math.Abs	2
Math.Ceiling	2
Math.Floor	2
Math.Max	2
Math.Min	2
Math.Pow	2
Math.Round	2
Max	11
MaxBy	11
Message	23
micro-ORM	34
MidpointRounding	2
migratie	31
Min	11
MinBy	11
N+1-probleem	32
nameof	22
namespace	13
navigatie-eigenschap	31
new() (beperking)	21
NewConfig	33
NoContent	26
NotFound	26

Naam	Hoofdstuk
Ok	26
OnDelete	31
OnModelCreating	31
OrderBy	11
OrderByDescending	11
overerving	20
override	20
PadLeft	1
PadRight	1
ParseExact	3
Path.Combine	30
Peek	9, 10
platslaan	33
Pop	10
primaire sleutel	31
protected	20
Push	10
QueryAsync	34
QueryFirstOrDefaultAsync	34
Queue	9
readonly	24
record	14
referentietype	16
Remove (database)	32
Remove (dictionary)	7
Remove (lijst)	6
Remove-Migration	31
RemoveAll	6
RemoveAt	6
Replace	1
Restrict	31
.Result	29
Reverse	6
[Route]	25
routeparameter	25
SaveChangesAsync	32

Naam	Hoofdstuk
Scan	33
sealed	20
Select	11
SelectMany	11
Set<T>	32
Single	11
SingleOrDefault	11
Skip	11
Sort	6
Split	1
splitOn	34
SQL-injectie	34
Stack	10
StartsWith	1
static	24
StatusCode	26
string.Format	1
string.Join	1
struct	15
Substring	1
Sum	11
switch	4
Take	11
Task.Delay	29
ThenBy	11
ThenInclude	32
throw	23
TimeSpan	3
ToArray	6
ToDictionary	11
ToList	11
ToListAsync	32
ToLower	1
ToTable	31
ToUpper	1
Trim	1

Naam	Hoofdstuk
TrimEnd	1
TrimStart	1
try	23
TryAdd	7
TryDequeue	9
TryGetValue	7
typeof	22
uitzondering	23
UnionWith	8
Unit of Work	32
Update (database)	32
Update-Database	31
UseNpgsql	31
using	13
Values	7
virtual	20
vreemde sleutel	31
waardetype	16
Where	11
where (beperking)	21
while	4
with	14
WithMany	31
WithOne	31
%	2
?.	17
?? en ??=	17
&& en 	4
? :	4
=>	13
! en = default!	17
"""	34